# Comparing Parallel Functional Array Languages: Programming and Performance

David van Balen[g], Tiziano De Matteis[f], Clemens Grelck[c,*], Troels Henriksen[a,*], Aaron W. Hsu[d,*], Gabriele K. Keller[g,*], Thomas Koopman[h], Trevor L. McDonell[j], Cosmin Oancea[a,*], Sven-Bodo Scholz[h,*], Artjoms Sinkarovs[i], Tom Smeding[g], Phil Trinder[b,*], Ivo Gabe de Wolff[g], Alexandros Nikolaos Ziogas[e,*]

[a] *University of Copenhagen, Universitetsparken 5, DK-2100, Copenhagen, Denmark*
[b] *University of Glasgow, 17 Lilybank Gardens, Glasgow, G12 8QQ, United Kingdom*
[c] *Friedrich Schiller University Jena, Inselplatz 5, Jena, 07743, Germany*
[d] *Dyalog Limited, Minchens Court, Minchens Lane, Bramley, Hampshire, RG26 5BH, United Kingdom*
[e] *ETH Zürich, Gloriastrasse 35, Zürich, 8092, Switzerland*
[f] *Vrije Universiteit Amsterdam, De Boelelaan 1111, Amsterdam, 1081 HV, Netherlands*
[g] *Utrecht University, Heidelberglaan 8, Utrecht, 3584 CS, Netherlands*
[h] *Radboud University, Houtlaan 4, Nijmegen, 6525 XZ, Netherlands*
[i] *University of Southampton, University Road, Southampton, SO17 1BJ, United Kingdom*
[j] *Unaffiliated, Utrecht, 3572 XH, Netherlands*

**Abstract**

Parallel functional array languages are an emerging class of programming languages that promise to combine low-effort parallel programming with good performance and performance portability. We systematically compare the designs and implementations of five different functional array languages: Accelerate, APL, DaCe, Futhark, and Single Assignment C (SaC).

We demonstrate the expressiveness of functional array programming by means of four challenging benchmarks, namely $N$-body simulation, MultiGrid, Quickhull, and Custom/Flash Attention. These benchmarks represent a range of application domains and parallel computational models. We argue that *the functional array code is much shorter and more comprehensible than the hand-optimized baseline implementations* because it omits architecture-specific aspects. Instead, the language implementations generate both multicore and GPU executables from a single source code base. Hence, we further argue that *functional array code could more easily be ported to, and optimized for, new parallel*

*Corresponding author

*Email addresses:* d.p.vanbalen@uu.nl (David van Balen), t.de.matteis@vu.nl (Tiziano De Matteis), clemens.grelck@uni-jena.de (Clemens Grelck), athas@sigkill.dk (Troels Henriksen), aaron@dyalog.com (Aaron W. Hsu), g.k.keller@uu.nl (Gabriele K. Keller), thomas.koopman@ru.nl (Thomas Koopman), trevor.mcdonell@gmail.com (Trevor L. McDonell), cosmin.oancea@di.ku.dk (Cosmin Oancea), sven.scholz@ru.nl (Sven-Bodo Scholz), a.sinkarovs@soton.ac.uk (Artjoms Sinkarovs), t.j.smeding@uu.nl (Tom Smeding), Phil.Trinder@Glasgow.ac.uk (Phil Trinder), i.g.dewolff@uu.nl (Ivo Gabe de Wolff), alexandros.ziogas@iis.ee.ethz.ch (Alexandros Nikolaos Ziogas)

*architectures than conventional implementations of numerical kernels such as the benchmarks above.*

We demonstrate these benefits by reporting the performance of the five parallel functional array languages on a total of 39 instances of the four benchmarks on both a 32-core AMD EPYC 7313 multicore system and on an NVIDIA A30 GPU. We analyze why each language performs well or poorly in each benchmark and architecture. *We argue that the results demonstrate that mature functional array languages have the potential to deliver performance competitive with hand-optimized C and CUDA code.*

---

## 1. Introduction

Parallel programming is hard, not only must the developer solve the challenges of sequential software, i.e., specify a correct and efficient algorithm operating over appropriate data structures, but they must also specify parallel coordination, e.g., how the computation is broken into sub-computations, how the sub-computations communicate and synchronize, how they are mapped onto cores, etc. Some parallel hardware is notoriously hard to exploit. For example, obtaining good performance on a GPU requires careful mapping of computations to the hardware, and careful management of the memory hierarchy, e.g., transferring data between GPU and CPU memories or effectively utilizing scratchpad (fast) memory as a cache.

Over time, parallel programming support in mainstream languages has evolved into directive-based interfaces, such as OpenACC [1] and OpenMP [2], or with low-level programming APIs, such as OpenCL [3] and CUDA [4]. These aim to simplify parallel programming across multiple popular architectures, such as multicore CPUs and GPUs. Such solutions may offer high-performance implementations for a given hardware. However, they (1) require significant programming expertise that may render it inaccessible to application-domain experts, (2) are error-prone, in the sense that the correctness of parallel execution is entirely the programmer's responsibility, e.g., data races are possible, and (3) do not necessarily offer performance portability, i.e., when switching from one hardware to another, it might be necessary to refactor a program to obtain good utilization of the new hardware.

In the light of the increasing variety of parallel architectures, and their rapid evolution, performance portability takes a more central role among the challenges of parallel computing. The key question is how to avoid the need for refactoring programs without giving up the performance gains of novel hardware.

An alternative to directive-based interfaces and low-level programming APIs has evolved from specialized fields: Domain-Specific Languages (DSLs) have been designed to capture domain-specific high-level operations for which sophisticated compilers can apply domain-specific optimizations enabling code generation for highly efficient parallel executions. These DSLs, often using a functional, array-based or dataflow paradigm, are accessible to domain experts, guarantee

correct parallel execution, and achieve high and portable performance in their narrow domain. Examples include Halide [5] for image-processing pipelines, DISTAL [6] and SpDistal [7] for dense and sparse tensor algebra leveraging einsum notation, TensorFlow [8] and PyTorch [9] for deep learning, etc.

This paper aims to investigate *to what extent functional array languages can transfer the benefits offered by DSLs, understood as high-level programming accessible to domain experts, to more general-purpose problems.* Such functional array languages are designed to express computations on arrays concisely and to implement them efficiently in parallel. For example, they commonly generalize operations on scalars to apply transparently to vectors, matrices, and higher-dimensional arrays/temsors. They have sophisticated implementations that exploit memory hierarchies, and often take advantage of hardware capabilities such as vector units to obtain greater computational throughput.

Examples of functional array languages include APL, whose design has stood the test of time, and others that have emerged more recently, such as Accelerate [10], DaCe [11], Futhark [12], and SaC [13]. These languages offer the attractive prospect of low-effort parallel programming, good performance, and performance portability between architectures.

Although functional array languages adopt a range of design, implementation, and optimization decisions, they have some core commonalities. They use a data-parallel model where parallel operations are expressed as computations over entire arrays, e.g., mapping a function over each element of an array or computing the sum or product of its elements. This model of parallelism is often called *bulk* parallelism, and it greatly simplifies parallelization as it avoids explicit synchronization and communication. Parallel functional array languages also avoid issues such as race conditions, deadlocks, and lifelocks by design.

### *1.1. Research Contributions*

Despite their apparent suitability for parallel array programming key questions remain. Are such high-level parallel programming models adequately expressive? What are the implications of functional array language design choices? Can language implementations deliver performance competitive with state-of-the-art conventional languages? We investigate these questions by comparing the designs, the implementations, and the performance of five parallel functional array languages: Accelerate, APL, DaCe, Futhark, and SaC. We start by introducing their design concepts using the rather simple $N$-body benchmark (Section 2), and then make the following research contributions:

- We systematically compare the designs of five very different functional array languages: Key language design aspects include the typing discipline, the forms of arrays supported, the role of higher-order functions (i.e., functions that take other functions as arguments), and the parallel paradigm(s) supported (Section 3).
- We systematically compare the implementations of the five functional array languages. Key implementation aspects are the implementation model (e.g., stand-alone compiler vs embedded DSL), the target architectures, and the supported optimizations (Section 4).

- We demonstrate the expressiveness of functional array programming using four challenging benchmarks from a variety of application domains, namely $N$-body simulation from the astrophysics domain, MultiGrid from the numerical aerodynamics domain, Quickhull from the computational geometry domain, and Custom/Flash Attention from the deep learning domain (Sections 6 to 9). These benchmarks represent a diverse choice of parallel computational models and set different challenges for compilation toolchains. In particular, they are sufficiently diverse that no single DSL can effectively express them all [1]. Rather than choosing simple benchmarks that we can easily express and effectively optimize in all five languages, these benchmarks challenge the expressivity of each language. Crucially, we argue that *the functional array code is more comprehensible than the hand-optimized baseline implementations* (Section 10.1).

- We compare the code base sizes, and hence approximate the programming effort, of the benchmarks in the functional array languages and in the OpenMP and CUDA baselines. We use Source Lines Of Code (SLOC) as a crude but widely accepted measure of programming effort [14]. A major difference is that the functional array programmer writes a single program that is compiled for multiple targets, where there are separate CPU and GPU baselines. Hence, *the total baseline code base is much larger, at least 17× than any of the functional array code bases. The functional code bases are at least 5× smaller than the CPU baseline, and at least 12× smaller than the GPU baseline code bases* (Section 10.2).

- We investigate to what extent performant implementations can be generated from a single high-level source for multiple architectures. Can performant code be generated even though the functional array code omits important architecture-specific information, e.g., aspects of the memory hierarchy? While several of the languages have previously demonstrated performant execution of the same code across different platforms, e.g. [15, 11], our multi-language study finds only partial evidence since only DaCe and Futhark consistently achieve good performance on both the multicore and GPU architecturess (Section 12).

- We report benchmark results for the five languages on both a 32-core AMD EPYC 7313 multicore system and on an NVIDIA A30 GPU (Sections 6 to 9). We analyze the multicore and the GPU performance of the languages on 39 instances of the four selected benchmarks to explore why each language performs well, or poorly, on each benchmark and architecture. We find that in 64 % of the instances, at least one language matches or outperforms the baseline; that in 78 % of the instances, at least one language achieves more than 80 % of the baseline performance. In only 6 % of the cases no language achieves more than 50 % of the baseline per-

[1] For example, Quickhull and MultiGrid are not in the application domain of TensorFlow and PyTorch, while Quickhull and Custom/Flash Attention are not supported by Halide.

formance. *We argue that the results show that mature functional array languages have the potential to deliver performance competitive with the best available conventional techniques* (Section 10).

### *1.2. Paper Structure and Reading Guide*

The remainder of the paper is organised as follows. Section 2 introduces the five parallel functional array languages and illustrates them by means of naive $N$-body simulation. Section 3 compares and contrasts the five language designs. Section 4 outlines and compares key aspects of the five language implementations. Section 5 provides the rationale for the four benchmarks studied, specifies the evaluation platforms, and outlines our development and measurement methodologies. Sections 6 – 9 report the GPU and multicore performance of the five languages on the four benchmarks. Section 10 evaluates expressivity and programming effort, summarizes overall performance, and analyzes the performance of each language. In Section 11 we discuss related work, and in Section 12 we draw our conclusions.

Not every reader will wish to read this substantial paper linearly from introduction to conclusion. Readers familiar with, or perhaps less interested in, one or more of the five languages may skip those language-specific sections. Readers interested in programming language design can focus on Sections 2 and 3. Readers interested in programming language implementation can focus on Section 4. The benchmark Sections 6 – 9 can be read in any order, or skipped if the benchmark or application domain is not of interest. These sections have a uniform structure: the first subsection introduces the algorithm, the second discusses the baseline and datasets, followed by presenting the implementation in each language. The descriptions for some languages are larger than for others. Each benchmark is first described in Futhark, introducing concepts that subsequent subsections rely on, making them relatively large. For some benchmarks in some languages there is simply more or less to say, and specifically some subsections discuss code fragments of particular interest. Each benchmark concludes with a summary of expression and performance results across all languages. The impatient reader can skim directly to the subsection of interest.

## 2. Parallel Functional Array Languages

*Array languages* express programs as operations on entire arrays, rather than on individual elements. The very first array language was APL [16], already conceived in the early 1960s. Array programming has proven particularly effective for scientific computing: it allowed efficient computations on matrices even with limited programming expertise. MATLAB is an example of a popular array language designed to provide a user-friendly interface to underlying linear algebra routines [17].

*Functional array languages* encourage the use of pure functions, i.e., functions that do not have side-effects, such as mutating variables [18]. Many such languages resemble well-known functional languages like Standard ML [19] or Haskell [20] extended with specialized array facilities. APL has been extended with functional programming capabilities in the form of direct functions [21].

*Parallel functional array languages* exploit the combination of array operations and the absence of side-effects to automatically parallelize programs. They are in the focus of our study.

To illustrate the five parallel functional array languages compared here, we introduce a simple benchmark, namely a (naive) solution to the well-known $N$-body problem, that simulates the gravitational interaction of particles. Outlining how $N$-body simulation is implemented introduces the languages and some of their design and optimization choices. More importantly, comparing the implementation in each language with the mathematical specification in the following section, as well as with the CUDA and OpenMP baseline implementations[2], gives a taste of functional array programming.

### *2.1. Running Example: the N-body Problem*

The $N$-body problem is a numerical simulation of the gravitational interaction of $N$ particles in space and time, by repeatedly computing the gravitational forces affecting each particle and moving them accordingly. The naive algorithm is as follows. We are given $N$ particles, each identified by its position $\vec{p_i}$, velocity $\vec{v_i}$ (both 3D vectors), and mass $m_i$ (a scalar). We write

$$\|\vec{p_i} - \vec{p_j}\| = \sqrt{(p_i.x - p_j.x)^2 + (p_i.y - p_j.y)^2 + (p_i.z - p_j.z)^2} \tag{1}$$

for the Euclidean distance between two points $\vec{p_i}, \vec{p_j}$, where $p_i.x$ is the $x$ component of the point $p_i$. We compute the *acceleration* $\vec{a_i}$ of particle $i$ as affected by all other particles using the formula

$$\vec{a_i} = \sum_{j \neq i}^{N-1} \text{force}(\vec{p_i}, \vec{p_j}, \vec{m_j}) \tag{2}$$

where

$$\text{force}(\vec{p_i}, \vec{p_j}, \vec{m_j}) = \frac{m_j(\vec{p_j} - \vec{p_i})}{(\|\vec{p_j} - \vec{p_i}\|^2 + \epsilon^2)^{3/2}} \tag{3}$$

and $\epsilon$ is a softening constant used to avoid excessive interactions between particles that are extremely close to each other. After computing the acceleration, the velocities $v_i$ are updated with

$$\vec{v_i} \leftarrow \vec{v_i} + \vec{a_i} \tag{4}$$

and once all velocities have been computed, the positions are updated with

$$\vec{p_i} \leftarrow \vec{p_i} + \vec{v_i} \ . \tag{5}$$

The computation is repeated for some number of steps to simulate the progress of time. As computing the acceleration for a single particle involves looking at all $N$ particles, the total number of interactions computed is $O(N^2)$.

[2] https://github.com/diku-dk/CFAL-bench/

```
-- Expressions
(a,b,c)                       -- tuples
{a=0, b=1, c=2}               -- records
[a,b,c]                       -- arrays
#foo 0                        -- sum type constructor
loop x = 0 for i < n do x + i -- sequential loop summing 1+2+...n-1

-- Types
(i32, f32, u8)                   -- tuple type
{a: i32, b: f32, c: u8}          -- record type
[n][m]i32                        -- array type, of shape n by m
#foo i32 | #bar bool             -- sum type with two constructors
[n]i32 -> [n]i32 -> [n](i32,i32) -- type of zip
```

Figure 1: Some Futhark syntax examples.

### *2.2. The Futhark Language*

Futhark is a statically typed functional array language with support for data parallelism [12]. It is a standalone language with an optimizing compiler that generates code for a variety of parallel architectures, although its GPU backends are the most developed.

Futhark's syntax and semantics are based on the $\lambda$-calculus. Futhark arrays are modeled as "lists of lists", and the programming model is similar to that of a conventional functional language such as Standard ML. Futhark supports standard parametric polymorphism and higher-order functions, although with some restrictions: functions are not entirely first class: they cannot be elements of arrays. This is to ensure that higher-order functions can be defunctionalized without requiring control flow or heap allocation of closures [22].

Futhark supports multidimensional arrays of any ***rank***, understood as the number of dimensions or axes. However, in contrast to languages such as SaC and APL, Futhark does not support ***rank polymorphism***, where functions can be applied to arrays of any rank. This means that to apply a function ot every element of an array, the programmer must explicitly use the `map` function, which may result in verbose code. For example, to scale a matrix `A` by 0.5 in Futhark, we write `map (map (0.5*)) A` — the outer `map` goes across the rows, and the inner across the elements of each row. In contrast, this operation is written as `0.5×A` in APL or as `0.5*A` in SaC. In practice, Futhark programmers typically define utility functions for such operations.

In addition to arrays, the supported compound types are records, tuples, and sum types (data types with multiple constructors, called *algebraic data types* or *enum types* in some languages). Fig. 1 summarizes the syntax by example. An unusual feature is that array shapes are tracked through a ***size-dependent type system***. This detects errors such as multiplying matrices of incompatible size, which is an example of a common source of runtime errors in other array languages, e.g., the dreaded `RANK ERROR` of APL. For example, the function `zip` takes two arrays and pairs up their elements, producing an array of pairs. The type of `zip` is shown at the bottom of Fig. 1 and states that the two arrays must have the same size `n` (for any `n`).

Futhark has some restrictions to enable efficient execution on parallel architectures with limited capabilities, such as GPUs. In particular, Futhark does not support general recursion, but merely provides tail recursion through a special syntactic construct, named `loop`. The `loop` construct binds the loop parameter (induction variable) `x` to `0`, then evaluates the body `n` times; binding the result of each evaluation to `x`, and returning the final value of `x`, as illustrated in Fig. 1.

Another restriction is that arrays of arrays must be ***regular*** (sometimes called ***rectangular***), meaning all elements of a multidimensional array must have the same shape. This is enforced through the size-dependent type system, with an escape hatch through dynamically checked coercions. For example, the type of the `zip` function, shown in Fig. 1, requires two arrays of the same length.

Parallelism is primarily expressed through ***Second Order Array Combinators*** (SOACs), functions that manipulate an array based on a function argument. Most SOACs are equivalent to common higher-order functions: `map`, `reduce`, `scan`, `scatter` (parallel write), etc. Parallel code is only generated by the SOACs and for functions using them. Futhark also allows the programmer to write sequential loops using a `loop` construct Futhark permits a language-based work/span cost model in the style of Blelloch [23], where the *work* of a function is the count of the total number of operations, and the *span* is the longest chain of dependencies, or how long it would take to run the program on an infinitely parallel machine.

The `reduce` and `scan` SOACs require the user to provide an associative operator and a neutral element (i.e., a ***monoid***). These properties are not verified by the compiler, i.e., providing an invalid operator, like subtraction, may produce unpredictable results.[3] Outside of a few such cases, Futhark is a deterministic language free of any kind of observable side-effects.

A more thorough description of Futhark is available in [12].

*Naive N-body simulation using Futhark*

The Futhark implementation in Fig. 2 illustrates a number of key language design aspects.

*Vectors.* The three-dimensional vectors `vec` in $N$-body are defined as records with `x`,`y`, and `z` fields, rather than as three-element arrays. This is different from how many array languages would model vectors, but similar to ML-style languages. Although partially a style choice, there is also an efficiency impact as the Futhark compiler generally assumes that arrays are "large" and sometimes does not manage small arrays efficiently. Hence, it is best practice to model small collections as records. The downside is that utility functions like `vecadd`

[3] Addition of floating point numbers is strictly speaking not associative either, yet it is common in Futhark programs. This is often not a problem because passing a non-associative operator to a reduction does not result in a completely arbitrary *result*, but merely in an arbitrary order of application. The result will still be some sum, only the rounding error will be scheduling-dependent. This is common practice in parallel programming.

```
type vec = {x: f64, y: f64, z: f64}

def vecadd (a: vec) (b: vec) =
  {x = a.x+b.x, y = a.y+b.y, z = a.z+b.z}
def vecsub (a: vec) (b: vec) =
  {x = a.x-b.x, y = a.y-b.y, z = a.z-b.z}
def vecscale (s: f64) (a: vec) =
  {x = s * a.x, y = s * a.y, z = s * a.z}
def dot (a: vec) (b: vec) =
  a.x*b.x + a.y*b.y + a.z*b.z

type body = {pos: vec, vel: vec, mass: f64}

def EPSILON : f64 = 1e-9

def accel (x: body) (y: body): vec =
  let r = y.pos `vecsub` x.pos
  let rsqr = dot r r + EPSILON
  let inv_dist = 1 / f64.sqrt rsqr
  let inv_dist3 = inv_dist * inv_dist * inv_dist
  let s = y.mass * inv_dist3
  in vecscale s r

def advance_body (dt: f64) (body: body) (acc: vec): body =
  body with pos = vecadd body.pos (vecscale dt body.vel)
       with vel = vecadd body.vel (vecscale dt acc)

def calc_accels [n] (bodies: [n]body): [n]vec =
  let move (body: body) =
    let accels = map (accel body) bodies
    in reduce_comm vecadd {x=0, y=0, z=0} accels
  in map move bodies

def step [n] (dt: f64) (bodies: [n]body): [n]body =
  map2 (advance_body dt) bodies (calc_accels bodies)

def nbody [n] (k: i32) (dt: f64) (bodies: [n]body): [n]body =
  loop bodies' = bodies for i < k do step dt bodies'
```

Figure 2: Futhark $N$-body simulation, representing vectors as records. In a larger application, we would likely define a reusable module for vectors, and perhaps parameterize over the number type. Only the functions `calc_accels`, `step`, and `nbody` involve parallelism, and the latter only by virtue of invoking `step`.

on line 3 are relatively verbose: the analogous function on array vectors would simply be `map2 (+)`.

*Compound structures.* The `body` type defined on Line 12 is a record with three fields, where the `pos` and `vec` fields are records themselves. Other parts of the program use arrays of `body` values. In most languages, such ***Array-of-Structures*** (AoS) are represented by storing the array elements consecutively in memory, which can lead to poor locality. Futhark guarantees that such arrays are instead stored as ***Structures-of-Arrays*** (SoA), i.e., multiple arrays that each contain only primitive elements. In this case, an array of type `[n]body`, like `bodies` on Line 28, will be represented as seven arrays of type `[n]f64`. This transformation is invisible to the programmer and improves access locality.

*Custom reduction.* Line 31 performs a reduction with the `vecadd` function using the zero vector as neutral element. The `reduce_comm` function indicates to the compiler that `vecadd` is commutative as well as associative. This information permits additional optimization and the generation of faster code.

### *2.3. The Accelerate Language*

Accelerate is a parallel array language deeply embedded in Haskell [10]. It offers the most common Haskell list operations as corresponding parallel operations on multidimensional, rank-polymorphic arrays, in addition to permutations, stencil operations, and others. Consider a simple dot-product calculation in Accelerate:

```
import Data.Array.Accelerate
dotp :: (Elt a, Num a)
     => Acc (Vector a)
     -> Acc (Vector a)
     -> Acc (Scalar a)
dotp xs ys = fold (+) 0 (zipWith (*) xs ys)
```

This function is almost identical to a sequential dot-product for two lists in Haskell, but works on one-dimensional Accelerate arrays (`Vector`). The `zipWith (*)` operation takes the two argument arrays and point-wise multiplies their elements in parallel; with `fold (+)` we then sum this point-wise product. The `fold` operation is a parallel reduction and, hence, requires the binary combination function, `(+)` in this case, to be associative.

It is worthwhile to take a closer look at the type of `dotp`. The type constraints `Elt a` and `Num a` require that the element type `a` is a member of the standard `Num` type class, so that numerical operations, such as addition and multiplication, are applicable. Further, `a` must be a member of Accelerate's `Elt` type class that contains all types that may be used as elements of an Accelerate array. This class includes the usual base types: integral types, floating point types, characters, tuples, and more. It can be extended with user-defined types. However, arrays cannot be used as elements: like Futhark, Accelerate supports only regular (rectangular) arrays, but instead of accomplishing this by reflecting array sizes in the type system, this constraint is expressed by allowing only rectangular, multi-dimensional arrays.

The type constructor `Vector` is a synonym for a one-dimensional array with elements of type `a`. Similarly, `Scalar` is short for a zero-dimensional array. In general, the `Array` type constructor is parameterized by the dimension type and the element type:

```
type Scalar a = Array Z a
type Vector a = Array (Z :. Int) a
type Matrix a = Array (Z :. Int :. Int) a
```

Dimension types are captured by the type class `Shape` and represent the *rank* of the array: `Z` for zero dimensions, `Z :. Int` for one dimension, etc. Values of these types are array shapes or indices. For example, `Z :. 3` is a value of type `Z :. Int`.

The arguments of `dotp` are not of type `Vector a`, as one might expect, but instead `Acc (Vector a)`. The reason is that Accelerate is an *embedded* language: the operations do not apply to values, but rather combine abstract syntax trees representing computations. So, the type `Acc (Vector a)` represents a *parallel* computation that, when evaluated, returns a value of type `Vector a`. There is a second calculation type, `Exp`, for sequential scalar computations.

As an example, the full types of the `fold` and `zipWith` operations are:

```
fold :: (Shape sh, Elt a)
     => (Exp a -> Exp a -> Exp a) -> Exp a
     -> Acc (Array (sh :. Int) a)
     -> Acc (Array sh a)
zipWith :: (Shape sh, Elt a, Elt b, Elt c)
        => (Exp a -> Exp b -> Exp c)
        -> Acc (Array sh a)
        -> Acc (Array sh b)
        -> Acc (Array sh c)
```

As we can see from the functions' types, their first arguments must be sequential (scalar-level) computations. The third argument for `fold`, the source array, is not restricted to be one-dimensional, but can be of any rank $n \geq 1$. The operation folds over the innermost dimension and returns an array of rank $n-1$. If $n \geq 2$, all inner vectors are reduced simultaneously, using *regular nested parallelism* as those inner arrays all have the same length.

As both functions operate on arrays of any dimensionality, they are said to be *rank-polymorphic*; this holds for most of the Accelerate language. As a result, `dotp` can be generalized to arrays of any rank $\geq 1$ without changing its body:

```
dotp :: (Elt a, Num a)
     => Acc (Array (sh :. Int) a)
     -> Acc (Array (sh :. Int) a)
     -> Acc (Array sh a)
```

Recall that the type of an Accelerate array mentions only its rank, not its shape. Hence, in contrast to Futhark, while the two array arguments to `zipWith` are guaranteed to have the same rank, at runtime they may have different shapes. The shape of the result is the coordinate-wise minimum of the shapes of `zipWith`'s inputs, i.e., their "overlapping" part.

Values of `Shape` type can be used to specify the size of an array or an index into it. For example, the function

```
generate :: (Shape sh, Elt a)
         => Exp sh -> (Exp sh -> Exp a) -> Acc (Array sh a)
```

```
data Vec = Vec_ Double Double Double
  deriving (Generic, Elt)

type Mass         = Double
type Position     = Vec
type Acceleration = Vec
type Velocity     = Vec
data PointMass    = PointMass_ Position Mass
  deriving (Generic, Elt)
data Body         = Body_ Position Mass Velocity
  deriving (Generic, Elt)

mkPatterns [''Vec, ''PointMass, ''Body]

instance Prelude.Num (Exp Vec) where
  Vec a b c + Vec x y z = Vec (a+x) (b+y) (c+z)
  Vec a b c - Vec x y z = Vec (a-x) (b-y) (c-z)
  Vec a b c * Vec x y z = Vec (a*x) (b*y) (c*z)
  negate (Vec a b c) = Vec (negate a) (negate b) (negate c)
  abs    (Vec a b c) = Vec (abs a)\ \ \ \ (abs b)\ \ \ \ (abs c)
  signum (Vec a b c) = Vec (signum a) (signum b) (signum c)
  fromInteger i = Vec (fromInteger i) (fromInteger i) (fromInteger i)

dot :: Exp Vec -> Exp Vec -> Exp Double
dot (Vec a b c) (Vec x y z) = a*x + b*y + c*z

scale :: Exp Double -> Exp Vec -> Exp Vec
scale s (Vec a b c) = Vec (s*a) (s*b) (s*c)

epsilon :: Exp Double
epsilon = 1e-9

pointmass :: Exp Body -> Exp PointMass
pointmass (Body p m _) = PointMass p m
```

Figure 3: Accelerate $N$-body simulation: data type definitions and auxiliary functions.

returns a new array with the shape given by its first argument, containing values as produced by its second argument (a function from index to value).

*Naive $N$-body simulation using Accelerate*

The Accelerate implementation of $N$-body simulation in Figs. 3 and 4 demonstrates other characteristics of the language. Fig. 3 defines the data types, and as Accelerate is deeply embedded, user-defined data types are lifted so they can be represented in the abstract syntax tree. This is done automatically for many types by adding `deriving (Generics, Elt)`.

Figure 4 shows our specification of $N$-body simulation in Accelerate. The `calc_accels` function takes the `bodies` vector containing all `n` bodies. To calculate the acceleration of all the combinations of bodies, it creates two temporary matrices by replicating each value `n` times (`bodies'`) and by replicating the vector `n` times (`bodies''`). The `accel` function is then applied to each point-wise pair, and the result is folded into a vector again. The generation of many intermediate arrays may seem inefficient, but the implementation fuses (eliminates) these arrays, and the generated code is a single parallel loop.

To better structure the code, we can define our own data types, like `Vec`. To pattern-match on this within an embedded language, we need a work-around

```
accel :: Exp PointMass -> Exp PointMass -> Exp Velocity
accel (PointMass xpos _) (PointMass ypos ymass) =
  let r = ypos - xpos
      rsqr = dot r r + epsilon
      inv_dist = 1 / sqrt rsqr
      inv_dist3 = inv_dist * inv_dist * inv_dist
      s = ymass * inv_dist3
  in scale s r

advance_body :: Exp Double -> Exp Body -> Exp Acceleration -> Exp Body
advance_body time_step (Body pos mass vel) acc =
  let position = pos + scale time_step vel
      velocity = vel + scale time_step acc
  in Body position mass velocity

calc_accels :: Acc (Vector PointMass) -> Acc (Vector Acceleration)
calc_accels bodies = fold (+) (Vec 0 0 0) (zipWith accel bodies' bodies'')
  where Z_ ::. n = shape bodies
        bodies'  = replicate (Z_ ::. n ::. All_) bodies
        bodies'' = replicate (Z_ ::. All_ ::. n) bodies

step :: Exp Double -> Acc (Vector Body) -> Acc (Vector Body)
step dt bodies =
  zipWith (advance_body dt) bodies (calc_accels (map pointmass bodies))

nbody :: Acc (Scalar Double) -> Acc (Scalar Int)
      -> Acc (Scalar Int) -> Acc (Vector Body)
nbody dt n k = afst $ awhile (\(T2 _ i) -> map (< the k) i)
   (\(T2 x i) -> T2 (step (the dt) x) (map (+1) i))
   (T2 (gen_input n) (unit 0))
```

Figure 4: Accelerate $N$-body simulation: main functions.

as we are pattern matching on values of type `Exp Vec`, not `Vec` itself. Specifically, the `Vec` data type has a constructor that is really called `Vec_`, and the `mkPatterns` macro (provided by Accelerate) defines a Haskell pattern synonym called `Vec` that works in the embedded language, like one would expect such a data constructor to work in normal Haskell. This pattern synonym is used in many of the functions in the figures, such as `dot` in Fig. 3. For `PointMass_` and `Body_` similar pattern synonyms are generated and used. These features allow the code in Figs. 3 and 4 to read like idiomatic Haskell: the code is very similar to a simple sequential implementation in Haskell.

For sum types (data types with multiple constructors) a more sophisticated technique is necessary. This includes a helper function called `match`; see [24] for details.

### *2.4. The Single Assignment C (SAC) Language*

SaC is a functional array language that aims to provide a platform for easy-to-understand yet generic programs that can be compiled into high-performance parallel codes for a variety of different platforms, including multicore systems, clusters, and GPUs [13].

As the name suggests, SaC can be seen as a side-effect-free variant of C: it supports a carefully chosen subset of C which can be mapped into a purely functional semantics. Interestingly, most language constructs from C are present

in SaC and, despite the purely functional semantics of SaC, they behave in exactly the same way as expected in C. C-style assignments are interpreted as nested let-expressions, if-then-else statements are mapped into conditional expressions, loops are transformed into tail-recursive functions, etc.

The key difference between C and SaC that enables this duality between the imperative and the functional world lies in abandoning the notion of assignable memory locations. All data in SaC is conceptually immutable and viewed as multidimensional arrays. Even *scalar values* are considered arrays of rank 0 in SaC. Arrays are rectangular, and their shape is described by a ***shape vector*** that contains the upper bounds for indices along each dimension (or axis). The length of the shape vector matches the rank of the array. For example, an array of shape `[10,20]` describes a matrix whose first index can range from `0` to `9` and whose second index ranges between `0` and `19`.

Arrays can be nested, but all sub-arrays must have the same shape (rectilinear arrays). This choice simplifies the semantic of SaC since nested arrays are indistinguishable from higher-dimensional arrays. For example, `[[1,2,3],[4,5,6]]` can be regarded as a 2-element vector of 3-element vectors or as a rank-2 array of shape `[2,3]`. Heterogeneously nested arrays like `[[1,2,3],[4,5]]` are not allowed.

SaC's type system adopts all element types from C. Type checking on element types is purely static. On top of that, all types in SaC do carry some shape information, which is denoted in square brackets behind the element type. To provide flexibility for programmers, different levels of shape information are expressible in the type system: the most generic SaC types are represented by the element type followed by `[*]`. For example, `int[*]` refers to arbitrarily shaped arrays of `int` elements. Arrays can be restricted to a particular rank; e.g., `int[.]` denotes integer vectors (rank-1 arrays). Finally, we can denote arrays of a specific shape; an array of `int` elements of shape `[10, 20]` has type `int[10, 20]`. All these shape specifications induce a natural sub-type relation, which can be used to overload functions. The type system aims to infer as many shapes as possible statically, but injects runtime checks where necessary. Type declarations for variables can be omitted, but they are mandatory within function signatures.

In addition, SaC supports stateful types for IO and external types for interfacing with foreign libraries; details can be found in [25]. Furthermore, SaC supports non-recursive records adopting C's `struct` syntax. Arrays of such structures are being transformed into collections of separate arrays each of which contains the values of one particular field of the given record. Details on how this transformation is implemented and what the performance impact is can be found in [26].

The key to the expressive power of SaC is the ability to define rank-polymorphic functions in combination with function overloading and rank-polymorphic array comprehensions named ***tensor-comprehensions***. Hence, functions such as `+`, `*`, `==`, etc., are all overloaded by rank-polymorphic versions in the SaC standard library. For example, the subtraction of two arrays of doubles of arbitrary rank and shape is defined as:

```
double[d:shp] - (double[d:shp] arr_a, double[d:shp] arr_b)
{
   return { iv -> arr_a[iv] - arr_b[iv] };
}
```

Here, we overload the subtraction function `-` for two array arguments `arr_a` and `arr_b`. The types of both array arguments are `double[*]`. However, we denote these types by `double[d:shp]`. This notation, referred to as ***type pattern***, enable programmers to express relations between the shapes of argument and return values. More details on type pattern can be found in [27]. Here, the type pattern requires both arguments to have the same rank (`d`) and the same shape (`shp`).

The result is defined by a *tensor comprehension*: for each index vector `iv` within the given shape, the result is the difference of the corresponding elements of array `arr_a` and array `arr_b`.

The equivalence between nested and higher-dimensional arrays naturally leads to generalizations of many standard functions, including selections and operations such as `take` and `drop`. For example, selections with fewer indices than the rank of an array return the corresponding hyperplanes as if the array was nested. For an array of shape `[2,3]`, such as `[[1,2,3],[4,5,6]]`, selecting at the index vector `[0]` results in the first row (`[1,2,3]`) while a selection with the index vector `[0,1]` yields the value `2`.

In addition to tensor comprehensions SaC supports rank-polymorphic reduction on multi-dimensional arrays, such as sum or product of all elements of a numerical array or conjunction and disjunction of the elements of a Boolean array. Reduction is the only second-order construct in SaC. It expects an associative and commutative binary function or operator as an argument. As usual, these properties cannot be verified by the compiler for arbitrary user-defined operations. Instead, we consider the use of a function or operator as a reduction operation as an implicit assertion of these properties.

Both tensor comprehensions and reductions are syntactic sugar for a more general multi-dimensional data-parallel construct, named *with-loop*, which mainly serves as an internal representation of array operations. Code optimization, code generation, and parallelization all benefit from this uniform intermediate representation of array code. A more detailed description of the design principles of SaC in general and the key language constructs of SaC can be found in [13].

*Naive $N$-body simulation using Single Assignment C (SaC)*

Naive $N$-body simulation is an excellent example to showcase the programming style of SaC. We first define a structure `Body` to capture the position, velocity, and mass for any given body:

```
struct Body {
    struct Vec3 pos;
    struct Vec3 vel;
    double mass;
};
```

The position and velocity vectors are `struct Vec3` records that are defined in the standard library as:

```
struct Vec3 {
    double x;
    double y;
    double z;
};
```

The `acc` function computes the acceleration between two bodies as:

```
struct Vec3 acc (struct Body b, struct Body b2)
{
   dir = b2.pos - b.pos;
   return dir * ( b2.mass / pow3 (EPSILON2 + l2norm (dir)) );
}
```

The definition uses generalized versions of C's built-in functions: subtraction, multiplication, and division are all applied to either arrays of identical shapes, or to a scalar (array with rank 0) argument and a non-scalar array (array with rank greater than 0). Furthermore, we have overloaded these versions not only for all of C's scalar types, but likewise for our record types, here `struct Vec3`.

Finally, we define the time step of the $N$-body algorithm:

```
struct Body[n] timeStep (struct Body[n] bs, double dt)
{
   acc = { [i] -> sum ({ [j] -> acc (bs[i], bs[j]) })}

   bs.pos += bs.vel * dt;
   bs.vel += acc * dt;

   return bs;
}
```

The function takes a vector of `n` bodies `bs` and a time delta `dt`. In the first line of code in the function body we define an array `acc` of accelerations for the bodies as follows: For each body `bs[i]`, we compute the sum of the accelerations between that body and all other bodies `bs[j]`. As each acceleration value is a record of type `struct Vec3d`, the type of `acc` is `struct Vec3 [n]`. The remaining two lines update the positions and velocities of all bodies, capitalizing on rank-polymorphism to multiply, add, and assign vectors of type `struct Vec3 [n]`.

### *2.5. The APL Language*

APL was conceived by Kenneth Iverson in the 1960s [21]. It can be viewed as a dynamically-typed functional language with primitives operating over $N$-dimensional, rectilinear arrays of arbitrary depth (nested arrays) containing character and numeric elements. Arrays may be heterogeneous in both element type and depth, and APL distinguishes the concept of depth, meaning the nesting level of an array, and rank, meaning the number of dimensions in an array.

APL implementations typically support a range of large and small numeric types, automatically converted as needed. This can save space when the representations are small, and usually improves performance, but does come at the cost of checking arrays for their element ranges.

APL's syntax and semantics are an unusual combination. Functions are limited to second-order and do not have closures (i.e., they are not first-class). Function application is written using infix notation, and functions take either no arguments (niladic functions), one argument (monadic), or two arguments, one on the left and one on the right (dyadic). Second-order functions associate to the left, whereas first-order functions associate to the right. Monadic functions take their single argument on the left (for second-order functions) or on the right (for first-order functions). The term "operator" designates second-order functions, and the term "function" designates first-order functions. Operator applications have higher precedence than function applications. All functions, including primitives, obey the same precedence rules.

APL's main claim to fame is its conciseness, largely achieved by denoting primitive functions and operators as a single symbol. Primitives are defined for all shapes, depths, and sizes of arrays. Exploiting the implicit traversal and iteration patterns in the primitives makes it possible to implement many computations without syntactic control flow.

The following code gives examples of APL's syntax using the Pythagorean theorem, first with parentheses and then using the commute operator to remove the need for them, which is a common programming technique in APL code.

```
h←(+⌿d*2)*0.5 ⍝ Pythagorean theorem for distance
h←.5*⍨+⌿×⍨d   ⍝ Same, but using commute and multiply
```

APL's core primitives may be scalar, like the arithmetic functions: these are lifted to apply element-wise for any depth or shape of array. Core functions that are not defined solely over single scalar elements are called mixed functions. The resulting shape or values of these functions depend on a combination of the shapes of the arguments and their values. Among these functions are those to alter or return the shape of an array, reorder its elements, and those to select elements from an array. The following two examples show the function ⊃ for taking the first element of an array, and the function ⍴ for reshaping an array.

```
      ⊃1 2 3  ⍝ Taking the first element of an array
1

      3 4⍴⍳12  ⍝ 3 by 4 Matrix of values [0, 12)
0 1  2  3
4 5  6  7
8 9 10 11
```

In addition to the core functions, APL defines a set of core operators, which define most of the commonly used iteration patterns over arrays. Unlike similar operations in other languages, these operators are defined over all array shapes, allowing for more direct control over the traversal of an array without the need to use more than a single operator. For example, the inner product operator (.) applies over arrays of any rank, including vectors or scalars, as well as matrices, cubes, and arrays with more than 3 dimensions. The inner product is also agnostic to its operands; so, `+.×` implements vector product, matrix

multiplication, and batched matrix multiplication in a single function. The expression `∧.=` is a common way to check for equal rows or columns within 2 matrices, but it also tests if 2 simple vectors have equal scalar elements. Likewise, operators like Each (¨) or Reduce (`/`, `⌿`) can apply over any rank or dimension, such as reducing the first or last dimension of an array.

APL permits binding values to alphanumeric names (as well as `⍙`) using the symbol `←` called "gets". For example, `x←1 2 3` binds `x` to the array `1 2 3`.

Square brackets may be used to select individual elements or sub-arrays from an array. For example, if array `A` is a rank 3 array (that is, having three dimensions), then the expression `A[1;3;0]` selects the single element from `A` at the index `(1, 3, 0)`. Arrays are indexed in row-major order. Selections are not limited to single elements, and sub-arrays or various selections (with duplication) may be obtained from an array using bracket notation. For example, if `M` is a matrix of shape at least `3×4`, then one could select the individual rows as `M[0;]`, `M[1;]`, and `M[2;]`, respectively; likewise, the columns may be selected as `M[;0]`, `M[;1]`, etc. A `2×2` matrix consisting of the second and third rows and first and fourth columns of `M` could be obtained by the expression `M[1 2;0 3]`.

Brackets may also be used to indicate the dimension or axis along which a function should operate. For example, the expression `+⌿A` will sum the elements along the first axis of `A`, that is, it will reduce the first axis using addition. Thus, if `A` is a matrix, the expression `+⌿A` will sum the columns of `A`. If `A` is a matrix of shape `3×4`, then `+⌿A` will result in a vector of shape `4`, that is, it contains four elements, where each element is the sum of the elements in the corresponding column of `A`. If we instead make `A` a rank-3 cuboid with shape `2×3×4`, we can reduce along the second dimension instead of the first by writing `+⌿[1]A` instead of `+⌿A`. In this case, `+⌿[1]A` will give an array of shape `2×4`, where index `(x, y)` of the result will be the sum of the elements `A[x;;y]`, that is, all the elements at sub-matrix $x$ and column $y$. Another way of saying this is that `+⌿[1]A` will sum the rows of a three-dimensional array of sub-matrices.

Most APL primitives are defined as operating independently on their elements (scalar primitives) or correspond to algorithms with known existing data-parallel implementations. This makes APL a natural fit for SIMD architectures. There are, however, two challenges: APL does not guarantee the associativity and referential transparency required for the safe parallel execution of operations like scan (`\` or `⍀`); and, the high-level expression of parallel array operations may not necessarily map effectively to a specific SIMD architecture.

*2.5.1. Naive N-body simulation using APL*

The following code for the $N$-body simulation demonstrates many features of APL as well as some common pitfalls. For a set of `N` bodies in a 3-dimensional space, we have a vector `m` of their masses, a vector `v` of their velocities, and a matrix `p` of their positions. The shape of `p` is `3 N`, which matches the lengths of `m` and `v`. Given two points represented as vectors, the difference between their coordinates can be expressed as `p1-p2`, using the implicit lifting of the `-` function. We can compute the difference of coordinates for all points against all

other points with an outer product `∘.-`. For a single dimension `x`, we can compute all differences by writing `p[x;]∘.-p[x;]`, which will give an `N N` matrix of the differences. We can extend this to compute the differences for each dimension in `p` by writing `p∘.-⍤1⊢p`, which will apply `∘.-` over each vector sub-array of `p`, of length `N`, which corresponds to our coordinate dimensions. The result `d` is a `3 N N` cube of the difference of each position against each other position:

```
d←p∘.-⍤1⊢p
```

We compute the distances `h` between each point using the Pythagorean theorem, plus the softening constant `e`:

```
h←(e++⌿d*2)*0.5
```

Here, the sum `+⌿` computes over the first axis, which has size `3`; `*` (exponent) is a scalar function that applies to each element in `d`. However, experienced APLers often prefer to avoid needless nesting. We can do so by using `×` instead of `*` to compute squares by using the commute operator `⍨`, which has the following semantics:

```
    X f⍨ Y ←→ Y f X ⍝ Commute arguments
      f⍨ Y ←→ Y f Y ⍝ Replicate right argument in monadic case
```

This allows us to simplify the original expression as follows:

```
h←.5*⍨e++⌿×⍨d
```

We compute the acceleration of each body by multiplying the mass of the body by the reciprocal of the cube of the distances:

```
a←m×[1]÷3*⍨h
```

Since `h` is an `N N` matrix, the expression `÷3*⍨h` will also be an `N N` matrix, and so we use the axis operator to multiply `m` over the second axis with `×[1]`.

Given accelerations `a`, we can compute new velocities by multiplying the differences in `d` of shape `3 N N` by the accelerations and by summing the results for each point, then multiplying by our time step `t`, which yields the velocity vector `v`:

```
v+←t×+/d×⍤2⊢a
```

We use `+/` rather than `+⌿` to multiply over the last axis and not the first, and `×⍤2` to multiply each matrix (rank 2) subarray in `d` by the accelerations in `a`. Using the rank operator `⍤` allows us to map over sub-arrays and compute over each dimension in our 3-D space without repeating the expression or indexing into the array.

We then compute the new positions for each of our points:

```
p+←t×v
```

```
nbody←{k t m←⍺
  accel←{+⌿d×[0]m×3*⍨÷.5*⍨e++/×⍨d←⍺-[1]⍵} ⍝ Acceleration
  step←{p v⊣p+←t×v←v+t×accel⍤1 2⍨⊃p v←⍵}  ⍝ Time step
  step⍣k⊢⍵                                ⍝ k time steps
}
```

Figure 5: APL $N$-body simulation. See text for description of each element.

This single step in the $N$-body computation can then be repeated for `k` steps by writing `step⍣k⊢p v` where `step` is a function that computes the updated values for `p` and `v`.

There is one major flaw in this computation. While the naive $N$-body benchmark assumes the use of the quadratic algorithm, the formulation above may also require quadratic space as it specifies the computation over all points at the same time. So, while the formulation is concise and easily verified, it does not scale to large inputs. A solution is to adapt the code to compute over a single point at a time as follows.

If we alter the shape of the position matrix `p` to be `N 3` rather than `3 N`, we can use `⍤` (Rank) to compute over each point. If we have a function `accel` that computes the accelerations of point `⍺` using all points `⍵`, we can compute the new accelerations for all points with `p accel⍤1 2⊢p`, which applies `accel` for each position in `p` as the left argument `⍺` against all points `p` as the right argument `⍵`.

The definition of `accel` is almost identical to our previous acceleration calculation except that it accounts for the transposed shape of `p` and we no longer need to use the `∘.-` outer product to compute the differences between points:

```
accel←{+⌿d×[0]m×÷3*⍨.5*⍨e++/×⍨d←⍺-[1]⍵}
```

Each time step is computed as above using the following function:

```
step←{p v⊣p+←t×v←v+t×accel⍤1 2⍨⊃p v←⍵}
```

In Figure 5 we combine all these elements to define the $N$-body simulation over `k` steps.

### *2.6. The DaCe Framework and SDFG IR*

In contrast to the languages presented so far, DaCe is a framework for mapping code written in high-level programming languages to CPU, GPU, and FPGA programs. DaCe mainly targets high-performance computing (HPC) applications, where two main development roles coexist: the *domain scientist* who provides the mathematical formulations, and the *performance engineer* who optimizes the code for the target hardware. One of DaCe's core principles is the *separation of concerns* of these two roles. The domain scientist provides only a

high-level description of the application's models, unencumbered by hardware-specific code and performance considerations in general. The performance engineer does not improve the high-level description directly, but works on an automatically generated representation amenable to data movement optimizations. Decoupling the application's mathematical description from its optimized implementation streamlines development, making it easy to add new scientific models and to specialize for different hardware architectures.

The workflow of DaCe starts with a high-level program written in C [28], Python [11], or another supported programming language. DaCe iterates over the program's abstract syntax tree (AST) to convert the application's high-level description into a graph-based intermediate representation (IR) called *Stateful Dataflow multiGraphs (SDFG)* [29]. A program's SDFG representation can also be created directly with a graph-based API. In this IR, the program is optimized with DaCe's tools and workflows, either guided by the performance engineer or automatically. Finally, DaCe's backends map the optimized SDFG representation to high-performance C/C++, CUDA/HIP, or HLS codes for CPU, GPU, and FPGA architectures, respectively.

Here we focus on the SDFG IR as it is a parallel functional array language. SDFG is dataflow-based and its programming model follows three principles:

1. Data containers and computations are separate.
2. Data movement is explicit from data to computations and other data.
3. Control flow is only used to define execution order that is not defined by dataflow.

A detailed description of the SDFG language can be found in DaCe's documentation [30]. Here, we describe the core elements of the IR, as shown in Fig. 6.

SDFG's basic data container is the multidimensional *Array*, a homogeneous random-access data structure. The element types supported include numbers (integers, floats, and complex floats) with different bit sizes, boolean values, and strings. An Array's dimensionality (or rank) is constant, but the length of each dimension can be parametric. However, *jagged arrays* are currently not supported. The SDFG does not directly impose constraints on array sizes, but the generated code does, as discussed in Section 4.5. In addition to arrays, the SDFG IR also defines the following data containers:

1. *Scalar*: Memory allocated for a single value; a "0-dimensional array".
2. *Stream*: Array of First-In-First-Out (FIFO) queues.
3. *Structure*: Data containing nested data containers.
4. *View*: Reinterpretation of data, e.g., a slice of an array.
5. *Reference*: A pointer to data containers of the same description (shape, data type, etc.), which may be set to another container dynamically.

Data containers do not appear in the SDFG; rather accesses to data containers are represented by ovals called *access nodes*. Global data, i.e., inputs and outputs of an SDFG program, are visualized with a thicker border to distinguish

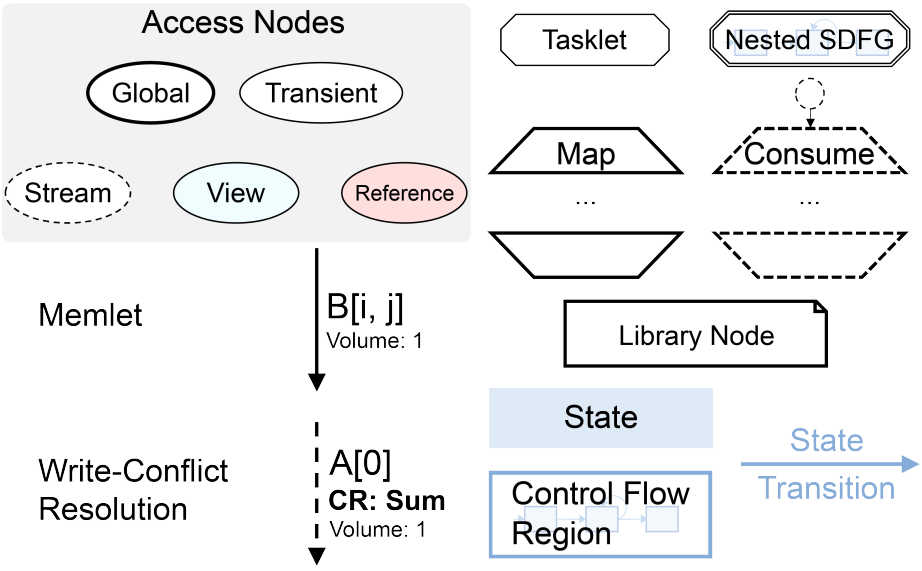



Figure 6: Elements of the SDFG IR, as shown in DaCe's documentation [30].

them from transient data defined only in the program's scope. Furthermore, streams have dotted borders, while views and references have cyan and red backgrounds, respectively.

Computations are represented by octagons called *Tasklets*, which are typically stateless and operate on scalar values. Tasklets may contain coarser computations, such as C++ code for calling an optimized BLAS library for matrix and vector operations. They can also entirely abstract away another nested SDFG. However, coarse computations are typically expressed through a *Map* or a *Consume*. The SDFG Map represents parametric parallelism as in the other functional array languages. Visually, a Map is described by two trapezoidal nodes, the *MapEntry* and *MapExit*, which pre- and post-dominate its body (an arbitrary subgraph) and are annotated with the Map's iteration space. Consume is the Map's equivalent for streaming computations and is visualized with dotted borders. Another class of computation nodes are *Library Nodes*, which represent high-level operations, such as matrix multiplication or reduction, but do not define their implementation. The latter is freely chosen during DaCe's optimization workflows, allowing Library Nodes to abstract away the use of specialized routines for different target architectures.

The edges connecting data and computations represent data moving from one memory location to another. The SDFG IR annotates its dataflow edges with *Memlets*, which present the data container and the exact subset accessed. For example, in Fig. 6 one of the edges shown reads or writes from data container B the value at index $[i, j]$. Whenever multiple computational units, e.g., threads, concurrently write to the same memory location, a *Write-Conflict Resolution* (WCR) edge must be used. Such cases typically occur inside Maps

implementing a parallel fold. WCR edges are dotted to distinguish them from normal dataflow edges, and their Memlet defines how to update the target memory location using the current and the new values. For example, the second edge in Fig. 6 writes to index 0 of data container A using the *Sum* method, which atomically adds the new value to the current one.

To represent program execution that does not follow data movement, the SDFG IR encapsulates dataflow in *States*, shown as blue rectangles. States are connected with each other with blue edges annotated with *State Transitions*. These transitions consist of conditions that must be met for the edge to become active and assignments to symbolic variables, such as iteration indices.

DaCe, although significantly different from the other languages in this study, has strong functional constructs (for example, the Map) and the array as the main data container. The SDFG representation allows mutable state, but this is mainly because DaCe requires certain optimizations, which are typically realized in a language's backend, to be explicit on the SDFG IR through subsequent lowering passes. An example would be a parallel fold implemented in low-level code using atomic operations. In DaCe, this optimization must be realized on the SDFG, after certain transformations/passes as a WCR edge while the actual low-level code generator will simply "blindly" generate the appropriate atomic construct for the target architecture.

*Naive $N$-body simulation using DaCe*

The naive $N$-body simulation in Fig. 7 illustrates many of these aspects. The code is "data-centric" Python [11], a subset of Python supported by DaCe and enhanced with elements tailored for developing HPC applications. The equivalent SDFG IR representation is shown on the right. The IR has been partially optimized (Section 4.5) to simplify the correspondence between the Python syntax and the SDFG elements, shown with double-headed arrows.

The program takes three inputs: the bodies' positions `pos` and velocities `vel`, and the timestep's duration `dt`. We include the bodies' masses in the positions data container, i.e., `pos` has four elements for every body. Since the positions and velocities of the bodies are four- and three-element vectors of the same datatype (float64), instead of using structures, we represent them as 2D matrices of size $4 \times N$ and $3 \times N$, respectively. The positions and velocities are overwritten at the end of a timestep; they are the program's outputs.

The computation for a single timestep takes place in a single dataflow SDFGState. Map scopes represent operations among arrays. For example, the `dist * dist` product is implemented with a Map scope iterating over the integer space $(d, i, j) \in \{0..2\} \times \{0..N-1\} \times \{0..N-1\}$, and the result is stored in the transient variable `tmp1`. "Reduce" library nodes represent reductions, such as those computing the intermediate data `dist_sq` and `accel`.

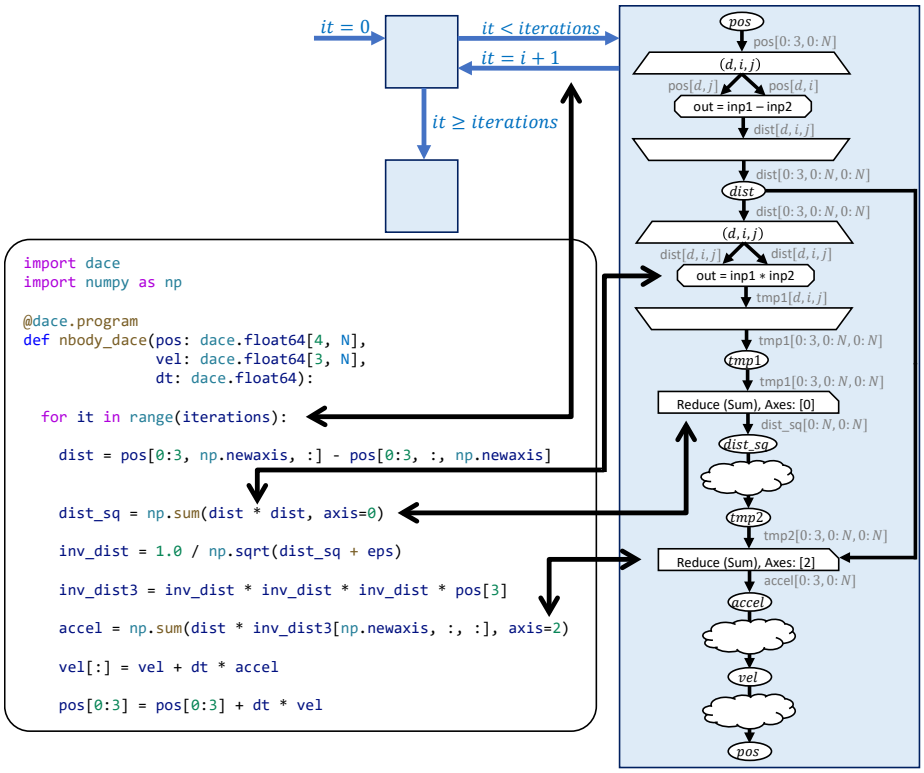



Figure 7: DaCe $N$-body simulation in data-centric Python and its SDFG IR representation. The double-headed arrows match selected computations' Python code with their corresponding SDFG representation. Cloud-like shapes abstract away several SDFG subgraphs containing Map scopes for brevity.

## 3. Language Summaries and Comparison

Having outlined the five functional array languages, this section provides a systematic comparison of their characteristics, properties, and features.

### *3.1. Type Systems*

The type systems are one of the most fundamental properties of the languages: Table 1 compares the systems supported by each language. APL is the only dynamically typed language in our comparison. Accelerate and Futhark provide Hindley-Milner-style parametric polymorphism, whereas SaC is monomorphic in array element types but polymorphic in array shapes and ranks. SaC supports ad-hoc polymorphism through function and operator overloading in combination with a shape-based subtype hierarchy. In DaCe any SDFG is monomorphic with polymorphic frontend languages supported via those languages instantiating any polymorphic constructs at runtime before generating and compiling the SDFG.

Table 1: Type Systems in the Functional Array Languages

| | Accelerate | APL | DaCe | Futhark | SaC |
|---|---|---|---|---|---|
| Discipline | Static | Dynamic | Static | Static | Hybrid |
| Parametric polymorphism | ✓ | ✗ | Inherited | ✓ | ✗ |
| Size-dependent | ✗ | ✗ | ✗ | ✓ | ✓ |

Table 2: Array Representation in the Functional Array Languages

| | Accelerate | APL | DaCe | Futhark | SaC |
|---|---|---|---|---|---|
| Shape | Rect. | Rect. | Rect. | Rect. | Rect. |
| Rank Polymorphism | ✓ | ✓ | ✗ | ✗ | ✓ |
| Jagged Arrays | ✗ | ✓ | ✗ | ✗ | ✗ |
| Heterogeneous | ✗ | ✓ | ✗ | ✗ | ✗ |
| User-defined types | ✓ | — | ✓ | ✓ | ✓ |

Both Futhark and SaC support a limited form of dependent typing [31], where program variables can occur within types to denote structural array properties. In Futhark, these features are used exclusively to rule out shape errors, e.g., that the two matrices passed to a matrix multiplication are incompatible. SaC goes beyond the detection of shape errors and supports so-called *type patterns* [27] that enable program variables to not only denote array sizes, but also their ranks and shapes. Type patterns further allow the extraction of shape properties from function arguments and the formulation of pre- and post-conditions of functions. All such type checks are performed in a hybrid fashion: statically when decidable at compile time, dynamically otherwise.

In Accelerate, the programmer can reason about shapes and ranks at the type level, but shapes are cleanly distinguished from terms. Unlike SaC, both Futhark and Accelerate check types statically and reject programs where rank-, shape-, or size-correctness cannot be determined at compile time.

We discuss additional implications of the type systems in the next subsection.

### *3.2. Array Representation*

Array representation is at the core of any array language, and Table 2 compares the representations supported by the languages. APL is the only language in our comparison that supports jagged arrays, i.e., multidimensional arrays

Table 3: Parallel Computation Paradigms in the Functional Array Languages

| Accelerate | APL | DaCe | Futhark | SaC |
|---|---|---|---|---|
| **Data parallelism** | | | | |
| Explicit | Implicit | Explicit / Implicit | Explicit | Implicit |
| **Higher-order functions** | | | | |
| Restricted | ✓ | Frontend-dependent | Restricted | ✗ |
| **Recursion** | | | | |
| ✗ | ✓ | ✗ | ✗ | ✓ |
| **Nested Parallelism** | | | | |
| ✗ | ✓ | ✓ | ✓ | ✓ |
| **Task parallelism** | | | | |
| ✗ | Fork-Join | Explicit | ✗ | ✗ |
| **Determinism** | | | | |
| ✓ | ✓ | Code dependent | ✓ | ✓ |

where inner arrays can have different shapes. All other languages support rectilinear arrays only. While the flexibility of APL is widely used by programmers, we shall see later that high-performance APL programs tend to use a subset of features similar to those supported by the other, statically-typed, languages in our comparison. In particular, jagged arrays are used sparingly.

Rank polymorphism, where functions can be applied to arguments of any rank, is a cornerstone of classic array programming, and is supported in Accelerate, APL, and SaC. It arises naturally from how these languages view the shape of an array as distinct from its elements. Specifically, APL, SaC, and Accelerate view an array as a flat *value vector* of scalars along with a *shape vector* of integers, where the product of the shape vector equals the length of the value vector. Data parallel operations, e.g. addition, operate directly on the elements of the value vector and simply propagate the shape vector. This enables an efficient value representation and ease of implementation, no matter the rank of the arrays.

Accelerate, APL and SaC even model scalars as 0-dimensional arrays, containing a single-element value vector, meaning that operations on arrays are not a special case. Accelerate distinguishes arrays and scalars in the type system, mainly to eliminate arrays-of-arrays. SaC has different runtime representations of arrays depending on static rank and shape knowledge. Therefore, SaC efficiently stores 0-dimensional arrays on the stack, instead of allocating and managing heap space.

Futhark is an outlier: its array model is more inductive and views multi-dimensional arrays as arrays-of-arrays. Futhark then uses a size-dependent type system to avoid jagged arrays. At runtime, the elements of a Futhark array will still be a single flat vector of values, similar to the rank-polymorphic languages.

*3.3. Parallel Computation Paradigms*

Table 3 compares the parallel computation paradigms supported by our five languages. All languages provide bulk *data parallelism*, i.e., parallel operations on entire arrays, but they do so in different ways. Accelerate and Futhark rely on *parallel combinators*, or higher-order functions [4], which must be explicitly invoked by the programmer. In APL, much parallelism is *implicit* in rank-polymorphic array operations. In SaC, parallelism ultimately arises from the use of tensor comprehensions and reductions, which in many cases are located within rank-polymorphic library functions. Compiler and runtime system jointly decide which concurrent operations are actually run in parallel and how) on the target architecture. In DaCe, programmers can *explicitly* specify parallelism by creating an SDFG containing parallel operations like Map. However, if the SDFG is generated by a frontend for a language with array operations, such as Python with NumPy arrays, parallelism is introduced by the frontend.

*Higher-order functions.* All of our five languages restrict the use of *higher-order functions* or entirely disallow them. APL only supports second-order functions, meaning that higher-order functions must take only first-order functions or arrays as arguments and can only return first order functions. APL's notation, outside of implementation-specific extensions, does not support the creation of closures.

Futhark supports higher-order functions, but they are not fully first-class. In particular, they may not be returned from conditional expressions, be array elements, or be carried across sequential loops. These restrictions ensure that higher-order constructs can be completely defunctionalized without any runtime overhead [22].

In Accelerate, higher-order functions can only be parameterized with sequential functions. Effectively, this results in restrictions similar to Futhark.

SaC does not support higher-order functions in general, but features a language construct that implements reductions using an associative and commutative reduction operation. The programmer must ensure these properties for user-defined operations.

DaCe supports a limited set of higher-order functions. In particular, depending on the frontend used, higher-order functions may be defunctionalized or otherwise evaluated prior to the generation of the SDFG, very similar to Accelerate. Moreover the Python frontend also includes custom reductions where the operation is defined via a lambda function, method parameters as callbacks to the CPython interpreter, and advanced indexing of NumPy arrays, which filters the underlying data based on another array.

With the exception of APL, these limitations are all rooted in implementation concerns: in high-performance code, function values are awkward, as they are applied using function pointers. Moreover arrays of functions can be difficult to represent efficiently as, even when their type is the same, different

[4] Functions that take other functions as arguments, e.g., `map`.

function and closure environments may vary greatly. Higher-order functions are restricted in Futhark, Accelerate, and DaCe to ensure that they can be compiled away (*defunctionalized*) early in the compilation process to avoid these representation challenges.

*Nested parallelism.* A key property is whether parallel constructs can be *nested*. Nesting constructs increases expressive power and aids performance, e.g., to specify how computations are mapped to GPU thread blocks, warps/wavefronts within a block, and threads within a warp. Futhark and DaCe allow nested parallelism, although in the case of Futhark's GPU backend, the nesting must be *regular*: the sizes of inner parallel loops must be invariant to outer loops. This implementation restriction does not apply when using the multicore CPU backend. Although Accelerate does not allow parallel functions as arguments to higher-order parallel functions, rank polymorphism allows implicit "mapping" of parallel operations. For example, a fold operation applied to a multidimensional array will fold, in parallel, over all arrays in the innermost dimension. This permits a restricted form of regular nested data parallelism. APL has implicit data parallelism. The runtime may parallelize the code differently depending on the shape of the data and the particular parallel operations involved. This includes potentially using nested parallelism and kernel-level task parallelism. SaC allows nested parallelism when the underlying nestings execute over rectilinear data.

Of the languages studied, only APL and DaCe provide support for explicit *task parallelism*. However, this paper focuses on data parallelism, and we do not explore combining data and task parallelism any further.

*Determinism.* Some parallel functional languages provide *deterministic parallelism* where the value computed by the program is always the same regardless of how it is executed, and in particular, is the same as the value computed by a sequential execution. Parallelism in imperative languages is often nondeterministic due to factors like random task scheduling. Non-determinism is recognized as one of the greatest challenges for parallel programming as it makes understanding, debugging, testing, and securing a program extremely difficult [32]. Determinism avoids this entire class of problems and, in particular, guarantees the absence of race conditions.

Accelerate, Futhark, SaC and the data parallel fragment of APL that we consider here, guarantee determinism in principle, although with caveats. Specifically, all languages provide parallel reductions that are only deterministic if the operator provided is associative, a property that usually cannot be automatically verified for user-defined operations. Moreover, all five languages unanimously consider floating point addition and multiplication to be associative, whereas they are strictly speaking not. This is a common feature across parallel computing in general because reductions with floating point addition and multiplication are common in numerical codes and floating point arithmetic is only an approximation of real arithmetic. However, the precise binary representation of a reduction may depend on runtime parameters beyond the control

of programmers. This source of non-determinism is widely accepted in parallel computing.

A DaCe program may be deterministic: the generated code for a given SDFG is always exactly the same, but the program may inherit non-determinism from the backend languages, e.g., from CUDA or C/C++ with OpenMP.

## 4. Language Implementations and Comparison

This section outlines the implementations of the five parallel functional array languages before making a systematic comparison that covers the implementation model, execution targets, and key optimizations supported.

### *4.1. Implementing the Futhark Language*

The Futhark compiler is a heavily optimizing, but architecturally conventional ahead-of-time compiler. It targets both GPUs (via OpenCL, CUDA, or HIP) or CPUs (via POSIX Threads) by generating C code that calls appropriate APIs. No code generation or optimization takes place at runtime, although some hardware characteristics (such as cache sizes and number of cores) are queried by a runtime system and are used to make scheduling decisions.

Each compilation target has a distinct optimization pipeline, although with significant overlap in passes. Compilation begins by compiling away module-level constructs, monomorphizing polymorphic functions, defunctionalizing higher-order functions, and flattening compound types, such as tuples. Arrays of compound types are represented as *multiple* arrays, each storing only primitive scalar types. For example, a source array of type `[n](i32,bool)` is represented in the compiler as two arrays of type `[n]i32` and `[n]bool`, i.e., as a structure-of-arrays (SoA) representation. Hence, the remainder of the compilation operates on a first-order, monomorphic program with only primitive types, arrays of primitives, and a handful of built-in second-order array combinators (SOACs), such as maps, scans (prefix sum), reductions, and generalized histograms [33].

The compiler performs standard optimizations, such as copy propagation, constant folding, inlining, common subexpression elimination, loop hoisting, etc. The most important early optimization is ***fusion*** (or *deforestation*) [34], where adjacent array traversals are combined to avoid materializing intermediate arrays. Many fusion rules are used; a simple example is map fusion, which replaces traversals for `f` and for `g` with a single traversal for the composition `f o g`:

$$\texttt{map f} \circ \texttt{map g} \Rightarrow \texttt{map (f} \circ \texttt{g)}$$

Futhark supports nested parallelism, but some compilation targets, like GPUs, only efficiently support a fixed number of parallel levels, and there are significant performance implications in how they are used. Therefore, one of the most important transformations is *flattening* [35], where multiple levels of application-level parallelism are collapsed and assigned to different hardware levels. This is achieved by ***incremental flattening*** [36], which uses map fission

and loop interchange to create semantically equivalent code versions that utilize more and more levels of application parallelism. Essentially, using a top-down program traversal, whenever a new `map f` operation is discovered, the analysis:

(1) creates a first version corresponding to a CUDA kernel in which each thread (independently) executes an application of $f$;

(2) creates a second version, dubbed *intra-block kernel*, in which the `map` parallelism discovered so far is mapped on the CUDA grid, and the parallelism inside $f$ is flattened and mapped to the CUDA block level; this has the benefit that intermediate arrays are created and reused from fast memory;

(3) continues flattening recursively — by means of map fission and loop interchange — in a parallel context that is extended with the current map.

The resulting code versions are combined in a program that branches depending on some dynamic program measure with a threshold. The best combination of code versions is derived by autotuning the threshold values [37].

Subsequent optimization passes (i) rewrite code using accumulators in terms of more specialized constructs such as reduce(-by-index) [38], (ii) optimize temporal locality, e.g., tiling in registers and shared memory is performed when an array that is invariant to one of the parallel dimensions is "streamed" through an operator, and (iii) minimize the footprint of scratchpad memory and eliminating unnecessary copy operations [39].

Finally, SOACs benefit from specialized GPU code generation, e.g., the implementation of scan [40, 41] uses Merrill and Garland's single-pass algorithm [42], and that of reduce-by-index [33] uses techniques aimed at (1) ameliorating cache thrashing and (2) reducing atomic conflicts by having subgroups of threads compute partial histograms, which are merged in a later step.

### *4.2. Implementing the Accelerate Language*

As Accelerate is deeply embedded (Section 2.3), rather than constructs directly operating on arrays or scalar values, they operate on abstract syntax trees (ASTs) that represent computations of arrays and scalars. Programmers are mostly shielded from this aspect, and write programs that look like regular Haskell. An embedded implementation brings a number of advantages: much of the mature host language infrastructure, such as the GHC compiler front-end, runtime system, and some of the memory management can be used. However, embedding also means that generating the code incurs some runtime overhead. The implementation aims to minimize the overhead by optimizing compilation time and caching compiled code so that if the same code is run multiple times, it is only compiled once. Moreover, it is possible to enforce code generation and compilation at Haskell compile time.

Accelerate currently has three fully-implemented back-ends: one that targets NVidia GPUs generating PTX code via LLVM, a multicore CPU back-end (also via LLVM), and a sequential interpreter for debugging. However, most of the implementation is platform-independent.

As for Futhark and many other languages where array operations are expressed as higher-order functions, fusion is a core optimization [10]. Regular nested operations are flattened, and monomorphic instances of polymorphic operations are generated. Scans and folds in inner loops are only executed in parallel if the range of the outer loop is too small to generate sufficient parallelism. The mapping of the surface types used by the programmer, such as arrays of compound types, to machine-friendly SoAs of primitive types is also similar to Futhark and other array languages. In Accelerate, the transformation to SoA form is implemented as part of the translation from surface to internal types, and it takes advantage of GHC's type families and associated types [43, 44].

In addition to array language optimizations, Accelerate additional embedded language optimizations. One such optimization is *sharing recovery* that maps user-friendly higher-order embedding with lambda abstractions and let-bindings into a first-order embedding [10]. Runtime compilation also allows for additional specializations of the code.

*4.3. Implementing the Single Assignment C (SaC) Language*

Although the name and syntax of Single Assignment C (SaC) may suggest a light-weight implementation on top of a C compiler, the reality is very different: sac2c is a many-pass compiler for a language that syntactically is very similar (and often identical) to C but semantically quite different; Fig. 8 provides an overview.

The first block of code transformations is ***functionalization*** where the intermediate code is transformed from the imperative-flavored source SaC into a representation that makes the underlying functional semantics more explicit. For example, loops are transformed into tail-recursive functions, if-then-else statements into conditional expressions, etc.

The next block of compiler passes performs type inference and checking. In contrast to C, variables in SaC can be introduced on the fly and without declaring their types.

The high-level optimization pass is crucial, and most optimizations are architecture-independent. Figure 8 also enumerates the most important optimizations. The optimizations include both conventional compiler optimizations, like copy propagation, and specific optimizations for functional array processing, like the various with-loop optimizations mentioned in Fig. 8. As pointed out in Section 2.4, with-loops are the crucial construct to represent array operations in SaC; all front-end tensor comprehensions are internally represented by with-loops. The compiler aims to fuse with-loops in three fundamentally different ways: First, the compiler fuses with-loops where the result of one becomes the input to another; we call this ***with-loop-folding***. With-loop-folding exploits the same equivalence as loop fusion (or deforestation) in other languages. Second, the compiler fuses with-loops that compute results based on the same or at least an overlapping set of argument arrays; we call this ***with-loop-fusion***. With-loop-fusion is similar to loop tupling in conventional languages. Finally, the compiler fuses nested with-loops where one computes the element-wise value of another; we call this ***with-loop-scalarization***.

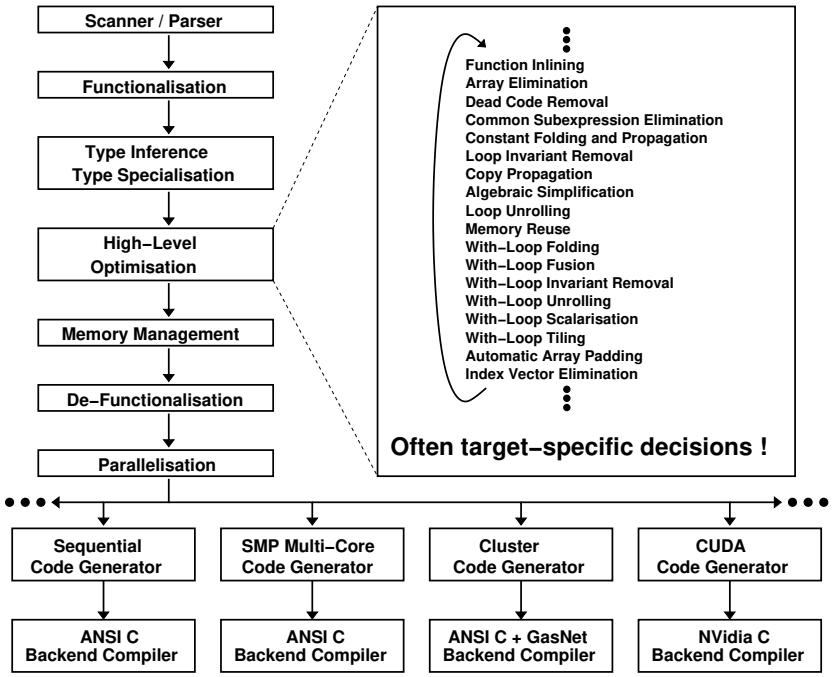



Figure 8: Structural organization of the SaC compiler sac2c

With-loops are SaC's one-stop-shop solution to both specify and internally represent data-parallel array operations; for details see [45]. Any language-level array operation is converted into some with-loop, including the tensor comprehensions introduced in Section 2.4. All optimizations benefit from the underlying purely functional semantics of SaC, and can usually be applied more aggressively than with an imperative interpretation of syntactically identical or very similar C code.

The memory management pass associates abstract arrays with concrete memory. This is where we control memory reuse and go to great lengths to counteract the aggregate update problem, both key aspects to achieving high performance with functional arrays. Details can be found in [46].

The next pass *defunctionalizes* the IR for the imperative target architectures. For example, tail-recursive functions are converted back into loops, etc.

Parallelization is a surprisingly small pass, largely because the functional array code is typically highly parallel in nature and because all array operations are internally represented by a single IR construct: the with-loop. With-loops typically have more concurrency than most architectures have execution units. For this reason, the compiler focuses on how much work to map to each processor (be it a core, a node, or a thread).

In a final pass, the SaC compiler selects and parameterizes the backend

compiler to produce a linked executable. A more detailed description of the SaC compilation pipeline is available in [47].

### *4.4. Implementing the APL Language*

APL has multiple implementations, and in the benchmark sections we use the Dyalog interpreter on the multicores and the Co-dfns compiler on the GPUS. Dyalog is a bytecode interpreter for APL source token streams implemented in C/C++. The interpreter calls a specialized function for each APL primitive. Most primitives have multiple implementations designed to run more efficiently on different input shapes, data ranges, etc. For some primitives, a minor degree of multi-threading is used if the data ranges are large enough, but this appears to have little impact on the benchmarks used here.

The design and architecture of the Co-dfns compiler is novel [48] and based on an extension of the NanoPass architecture [49]. It consists entirely of small, incremental, data-parallel passes written in APL to construct the entire compiler pipeline. It uses almost zero abstractions over pure APL arrays, where the AST is represented using an SoA design and is exceptionally efficient in its data representation. Outside of the major pipeline functions encapsulating the parser, compilation, and code generation phases, it contains almost no function abstraction nor many internal temporary variables, but only primitive APL function expressions over a few AST fields and temporary variables. The code, thus, lacks explicit looping constructs, branching, conditionals, or other explicit control flow outside of the use of the power operator to create iteration to fixed points for certain passes. As a result, the compiler is exceptionally compact while being data-parallel by construction and thus GPU compatible.

The Co-dfns compiler optimizes very little at compile time. Efficient data representations, fusion of APL primitive calls, GPU acceleration, inlining of functions, etc., all utilize runtime implementations or the backend compiler, such as a C compiler.

Fusing the application of multiple primitives into a smaller number of GPU kernels is a critical optimization for array programs. Co-dfns utilizes existing JIT and GPU libraries to fuse calls to primitive functions [50], simplifying the compiler while enabling features such as fusion across user-defined function call boundaries. However, the computations being fused must be combinations of pre-existing APL primitives. For example, reductions using max, min, or plus can all be fused with scalar primitives in a single pipeline, whereas reduction with a user-defined function cannot. The runtime can detect the use of some known functions, and if idiomatic combinations are used, such as $+.\times$ for matrix multiplication, the runtime dispatches to an optimized library function specialized based on a combination of the datatypes involved and the functions being called. However, these specializations are limited to what the runtime can see. Thus, optimizations like algebraic-style expression simplification are not applied.

The latest version of the Co-dfns compiler uses a runtime implemented in APL, with a small subset of functionality provided by the underlying host platform, such as C or JavaScript, in the form of a small kernel library.

*4.5. Implementing the DaCe Framework*

DaCe expresses program optimizations as graph transformations on the SDFG IR [29]. Transformations are applied in optimization passes and may alter any aspect of the graph representation. They may be applied to the whole program or only to specified subgraphs. Multiple passes can be combined into *pipelines*, ensuring that pass dependencies are met. DaCe provides a large library of transformations, passes, and pipelines. The framework includes user-level APIs to allow the development of new transformations and pipelines [51, 52]. Although many of the built-in passes and pipelines can be applied automatically, the user may choose to optimize a DaCe program manually. Manual optimization can use a set of APIs or visual tools that guide minimizing data movement [53]. DaCe also provides APIs for instrumenting, optimizing, and auto-tuning SDFG programs through performance modeling with built-in or external tools [54, 55].

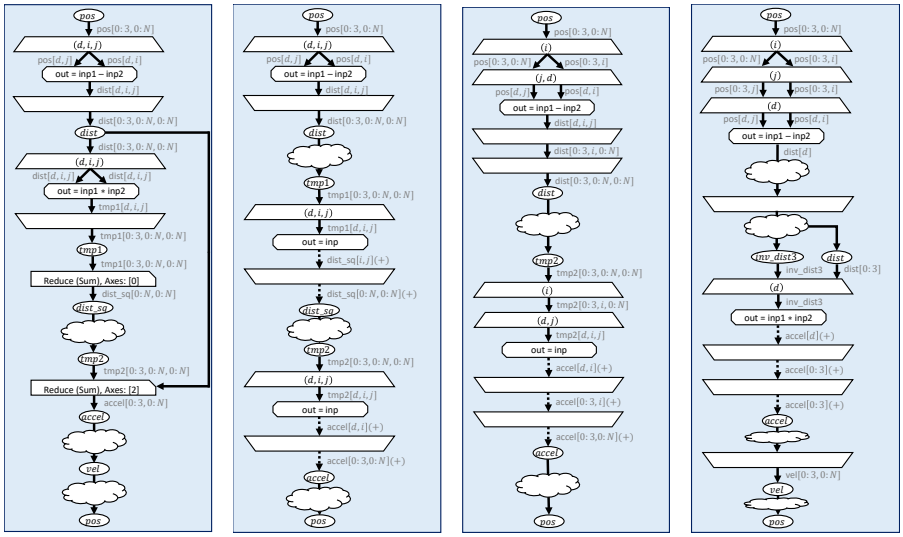



Figure 9: Optimization workflow for the acceleration computation of the $N$-body program in DaCe: (a) unoptimized SDFG IR, (b) Library Nodes' expansion, (c) Map scope dimensions' permutation and *MapExpansion*, (d) *SubgraphFusion*.

We illustrate some built-in data-centric transformations for the $N$-body program, focusing on the single iteration of the algorithm, shown in Fig. 9a. The first optimization expands the Reduce Library Nodes to Map scopes, and must introduce Write Conflict Resolution (WCR) on the output edges, as multiple Map iterations write to the same memory location (Fig. 9b). Next, we permute the dimensions of selected Map scopes and apply the *MapExpansion* transformation so that all subgraphs have an outer iteration over the bodies using index $i$ (Fig. 9c). Finally, we apply the *SubgraphFusion* transformation to fuse all subgraphs up to (and including) the computation of the updated velocities (`vel`)

under a common outer Map scope (Fig. 9d). The transformation does not apply to the Map scope that updates the bodies' position, as that would produce read/write conflicts on `pos`. Fusing the Map scopes renders all intermediate results thread-local and reduces their dimensionality. For example, `dist` and `accel` become three-element vectors and `inv_dist3` reduces to a scalar value.

Specialization for a specific architecture like a GPU also occurs in the SDFG IR. Data containers have storage types and Maps have schedule types (Section 2.6). For example, to "convert" the $N$-body program for execution on a GPU we simply switch the program's inputs and outputs to *GPU-global* memory. Similarly, the Maps' schedules must be changed to *GPU-device* and to *GPU-thread-block*. Storage and schedule changes can be applied manually (programmatically or using visual tools) or automatically by applying device-specific transformations, such as *GPUTransform*, which iterates over the SDFG and updates the schedule of Map scopes deemed suitable for GPU execution. Data movement between the host and device is explicit in the SDFG and can also be automated with the same device-specific transformations.

Since a program's SDFG representation includes scheduling and storage specification, DaCe's code generation is relatively simple. The first step is to traverse the SDFG to determine the lifetime of all transient variables, marking the appropriate states for allocation and de-allocation. Subsequently, DaCe partitions the SDFG into subgraphs defining control-flow scopes. For loops and structured control, DaCe generates the corresponding C/C++ syntax, e.g., `for`, `while`, `if`, etc. Unstructured control flow is generated as a state machine with `goto`s. The body of the control flow scopes is generated state-by-state in a topological order. Each state is code-generated in three steps: (1) transient allocation, (2) dataflow code generation, and (3) transient de-allocation. To generate code for dataflow, DaCe traverses the graph in topological order, generating appropriate code based on the schedule and storage types, e.g., a map with a *CPU-multicore* schedule generates an OpenMP for-loop, whereas a GPU-related schedule generates a CUDA/HIP GPU kernel.

### *4.6. Language Implementation Comparison*

The five languages studied use a variety of implementation techniques: the Futhark and SaC compilers, the embedded language Accelerate, the DaCe framework, and the APL interpreter/compiler. The key differences are summarized in Table 4.

The SaC and Futhark implementations are similar in being conventional ahead-of-time compilers that perform a series of compiler passes and optimizations. The passes are largely black-box optimization passes, although SaC provides a variety of command line options to control the process, similar to conventional optimizing compilers. Both Futhark and SaC generate optimized C executables. The Futhark compiler is written in Haskell, while the SaC compiler is written in C. The main difference is that while SaC has an IO facility based on uniqueness types [25], and thus allows the construction of entirely free-standing SaC programs, Futhark does not have any IO facilities, and therefore no way to read input or write output. The output of the Futhark compiler is

Table 4: Table summarizing key differences between language implementations.

| | Accelerate | APL | Futhark | DaCe | SaC |
|---|---|---|---|---|---|
| **Implementation model** | | | | | |
| Standalone | ✗ | ✓ | ✓ | ✗ | ✓ |
| Implementation language | Haskell | APL | Haskell | C/Python | C |
| **Automatic Optimizations** | | | | | |
| Fusion | ✓ | ✓ | ✓ | ✓ | ✓ |
| Data layout | ✗ | ✗ | ✓ | ✗ | ✓ |
| Tiling | ✗ | ✗ | ✓ | ✓ | ✓ |
| SoA (otherwise AoS) | ✓ | ✗ | ✓ | ✗ | ✓ |
| **Targets** | | | | | |
| CPU | ✓ | ✓ | ✓ | ✓ | ✓ |
| GPU | ✓ | ✓ | ✓ | ✓ | ✓ |
| FPGA | ✗ | ✗ | ✗ | ✓ | ✗ |
| Clusters | ✗ | ✓ | ✗ | ✓ | ✓ |

essentially a *library* that must be invoked by a program written in a general purpose language, through a C-based API. For convenience, the Futhark compiler provides a facility for automatically generating small wrapper C programs for benchmarking and testing purposes.

Accelerate is deeply embedded in Haskell as a Haskell library, and the compiler executes at runtime. Indeed, the Accelerate program is only usually constructed during the execution of the host Haskell program, where it is then compiled using a runtime compiler. Usually, the overhead is neglibile compared to the runtime of the programm. Otherwise, the Accelerate programmer can also choose to compilation of the Accelerate code at Haskell compile time. The Accelerate compiler automatically performs many optimizations and rewrites of the program, notably fusion. It is also possible for the user to perform Haskell-level computation to specialize the Accelerate program for a given workload, which can be seen as a form of staged programming. This capability is exploited for some of the benchmarks we use, such as MG (Section 7.4).

DaCe is similar to Accelerate in being embedded in a frontend language (commonly Python), although it differs from the other language implementations in exposing far more control over the optimization process, e.g., with the programmatic or visual tools outlined in Section 4.5.

APL is traditionally interpreted on CPUs, and the Dyalog interpreter (written in C) provides efficient operations on arrays of different sizes. The Co-dfns APL compiler is written using APL in a data-parallel style to facilitate

self-hosting on a GPU. It makes heavy use of runtime libraries to achieve performance, often calling specialized routines that implement APL primitives. Where most of the other compilers use sophisticated optimizations that may, however, produce unpredictable performance, the Co-dfns targets predictable performance. It provides tight integration with the Dyalog interpreter, allowing compiled modules to be used "in situ" from within the interpreter, while also producing standalone executables and shared objects that can be linked from other programming languages.

*Hardware targets.* Finally, the language implementations all target multiple architectures, and all target multicores and CUDA/NVIDIA GPUs. The languages may also target other architectures, e.g., APL, DaCe and SaC also target clusters, and DaCe also targets FPGAs.

## 5. Benchmark Rationale, Evaluation Platforms, Methodologies

This section provides the rationale for selecting the four benchmarks we study, specifies the GPU and multicore evaluation platforms, outlines the novel functional array program development methodology used, and discusses the measurement methodology.

### *5.1. Benchmark Rationale*

We illustrate functional array programming using four challenging benchmarks that represent a range of application domains and parallel computational models. Two benchmarks are from public suites, namely MultiGrid (numerical aerodynamic simulation) from NAS [56, 57] (NAS benchmark MG) and Quickhull (computational geometry) from PBBS [58]. The other two are well-known algorithms, namely brute-force $N$-body simulation (astrophysics simulation) and Flash or Custom Attention [59] (deep learning).

The benchmarks cover a *variety of code patterns, parallel constructs/structures, and optimization techniques*: Quickhull exercises flattening of irregular nested parallelism, where manual application results in a composition of (all) flat-parallel skeletons — map, scan (prefix sum), reduce-by-index, scatter. The other benchmarks exhibit regular nested parallelism. We have already described $N$-body in detail in Section 2. MultiGrid consists of various 3-dimensional 27-point stencil operations with different resolutions. Flash Attention has deeply nested parallel and sequential loops with matrix multiplication solvers at the innermost level.

The benchmarks warrant different optimizations. Quickhull's performance is determined by how well the flat-parallel skeletons are fused and mapped to the hardware. All other benchmarks require careful mapping of computation to different hardware levels, e.g., for achieving temporal reuse from fast memory and for enabling CPU vectorization. Flash Attention requires tiling multiple dimensions into each level of the memory hierarchy. Finally, $N$-body exhibits an interesting case of dataset sensitivity on GPUs: the largest dataset benefits

from sequentializing the inner function/loop, while the smaller datasets benefit from parallelizing all functions/loops.

Rather than choosing simple benchmarks that can readily be expressed and effectively optimized in all five languages of our comparison, we have selected a set of challenging problems that often take a language outside its "comfort zone". For example, Quickhull utilizes scans (prefix sums), where neither SaC nor DaCe provide a scan primitive. Several benchmarks require nested parallelism, and Accelerate does not normally provide it. MultiGrid uses stencils, and Futhark does not provide specific stencil support nor deep learning support for Flash Attention. The hypothesis is that *collectively* these five research languages offer state-of-the-art solutions to both the expression and the performance challenges imposed by these four benchmarks.

### *5.2. Evaluation Platform: the Radboud Server*

The evaluation is conducted on an experimental server at Radboud University Nijmegen that provides both a multicore CPU and a GPU. The server is booked for exclusive use during the experiments. The CPUs are two 16-core AMD EPYC 7313 CPUs running at 3.7 GHz. The peak performance of this machine is 1.895 Tflop/s (double precision). The RAM has a theoretical peak bandwidth of 204.8 GB/s, though experiments with the standard STREAM benchmark rather yield 140 GB/s.

The server also provides an NVIDIA A30 GPU with a peak performance of 10.3 Tflop/s and 5.2 Tflop/s for single and double precision, respectively. It has 933 GB/s main memory bandwidth. We use NVCC version 12.1.66 and GCC 11.4.0.

For multicores we use `numactl --interleave all` to set the page-mapping policy, and we configure the thread-binding policy of OpenMP to use `OMP_PLACES="cores" OMP_PROC_BIND=true`.

### *5.3. Development Methodology*

Baselines were selected or improved to serve as a "reasonable metric'' for state-of-the-art performance, regardless of programming effort. We targeted implementations in low-level mainstream languages, such as OpenMP+C/Fortran and CUDA. We used different implementations, specialized for CPU and GPU execution. As inclusion criterion, we take implementations that have been described in peer-reviewed publications, e.g., MultiGrid uses the NAS [56] and NPB-GPU [57] for CPU and GPU execution, respectively. Quickhull uses the CPU implementation of PBBS [58]. If it is possible to improve an implementation without introducing novel techniques, we have done so, e.g., as discussed in Section 6.2 for the GPU version of $N$-body simulation. Since we were unable to find a low-level CUDA implementation of Flash Attention, we have developed one, and checked that it matches the performance reported in the original paper [59] when run on the same hardware. The notable exception is Quickhull, for which we did not find a reputable GPU baseline; we highlight that some of the paper's discussion ***unfairly*** compares the languages' GPU performance with the CPU baseline of Quickhull.

The benchmarks are implemented in each of the languages following an iterative procedure: first, we develop and validate a correct implementation, and then repeatedly refactor it to optimize for the target hardware architectures. The optimizing iterations are not described in detail, although we discuss the key optimizations for each language in Sections 6 to 9. All benchmarks are implemented by programmers proficient with the language, and often by language developers. Exceptions include the Futhark Quickhull implementation which was developed by an MSc student.

In contrast to the separate CUDA and OpenMP baseline implementations, *there is a single benchmark implementation in each functional array language*. This implementation is compiled and optimized for both multicore and GPU execution and for all datasets. The only exception is DaCe, which uses separate CPU and GPU implementations of $N$-body and Flash Attention to improve performance. This is motivated by DaCe's high-level design rationale that aims to provide the user the tools that allow easy specialization of the optimization recipe to a given application and hardware. We discuss the development effort implications of having a single implementation in Section 10.2.

### *5.4. Measurement Methodology*

We measure total application runtime, but in the case of GPU execution, we exclude the time taken for the (1) CUDA-context initialization, (2) CUDA-kernel compilations, and (3) host-device transfers of program inputs and final result (only). For each benchmark and hardware platform, we perform up to five warm-up runs, and in the summary tables at the end of Sections 6 - 9 we report the average of at least ten runs, or as many as required to reduce the standard error to less than 3 %. All performance numbers relative to the baseline are computed based on the raw measurements and not those shown in the tables, which are rounded for the purpose of readability.

In addition to the summary tables, Appendix A reports more detailed performance measurements for each benchmark instance on the multicore and GPU platforms. This includes the number of runs, the median, mean and standard deviation, the 95% confidence interval, and maximum memory residency.

### *5.5. Data and Reproducibility*

The benchmark program source code is provided in a public repository[5] and archived on Zenodo [60], including all benchmark source code, instrumentation, input generation, and documentation. The raw measurement data is likewise archived on Zenodo [61], including information about the hardware and software configuration used to obtain the measurements.

[5] https://github.com/diku-dk/CFAL-bench/

## 6. Benchmark 1: N-body Simulation

The first benchmark is the well-known $N$-body simulation from the astrophysics domain, and has already been described in Section 2, together with implementations in the five functional array languages of this language comparison. This section introduces the structure used for each benchmark in Sections 6 to 9.We start by briefly introducing the benchmark problem followed by a description of the conventional performance baseline and benchmarking characteristics. Then we discuss the implementation in each of the five languages before summarizing and discussing the CPU and GPU performance results.

### *6.1. Benchmark Description*

The $N$-body simulation benchmark is specified in Section 2.1. One of the key design choices across programming languages pertains to the representation of bodies. Recall that each body is associated with multiple values, namely its position and velocity — each represented as a tuple of three double-precision floating point numbers — together with its acceleration and its mass (both once more double-precision floating point numbers). Hence, there are various ways to represent bodies in memory, such as using array-of-struct (AoS), struct-of-array (SoA), or some combination thereof.

### *6.2. Baselines, Performance Measure, Datasets*

The GPU baseline consists of two code versions, both using AoS layout, whose selection is optimally hardcoded for the target datasets. When the number of bodies $n$ is small ($n \leq 10^4$), we implemented a version that efficiently exploits both levels of parallelism by computing the acceleration for a body with a CUDA block of threads. For larger values of $n$ we use the CUDA-sample implementation[6] that utilizes only the outer level of parallelism. The key optimization is 1D tiling in shared memory. That is, computing the acceleration for a body accesses all other bodies in the same order. Hence, the computation is structured so that a chunk of bodies is collectively copied from global to shared memory by the threads in the same block and then reused. Without tiling, reuse would be served by the L2 cache, which has higher latency and smaller throughput than GPU shared memory.

We have written a C with OpenMP CPU baseline that (i) parallelizes the computation of the `accel` function across processors using OpenMP and static scheduling, and (ii) ensures that the underlying (GCC) compiler successfully vectorizes the code by exploiting a SoA representation consisting of eight arrays of doubles.

We compute the number of FLOPs for $n$ bodies and $t$ time steps as $(19n^2 + 12n) \cdot t$. Each interaction between two bodies is computed as

$$\frac{m_j(\vec{p_j} - \vec{p_i})}{(\|\vec{p_j} - \vec{p_i}\|^2 + \epsilon^2)^{3/2}} \tag{6}$$

[6] github.com/NVIDIA/cuda-samples/tree/master/Samples/5_Domain_Specific/nbody

requiring 16 FLOPs to compute the acceleration change and another 3 additions to update it. The evaluation uses three datasets corresponding to the following values of $(n, t)$: $(10^3, 10^5)$, $(10^4, 10^3)$ and $(10^5, 10^1)$. Note that these datasets all involve the same amount of work, but vary in how much parallelism is available.

### *6.3. N-body Simulation with Futhark*

As the running example used to introduce the functional array languages, the Futhark $N$-body simulation code is discussed in Section 2.2. In short, it is expressed as two nested loops — the outermost a `map` and the innermost a `reduce` (fused with a `map`). Futhark's CPU and GPU backends both generate two code versions: (i) where the parallelism of the `reduce` is exploited, resulting in a segmented reduction and (ii) where the innermost loop is sequentialized. A code version is dynamically selected at runtime, as explained in Section 4.1. For the $n = 10^5$ case, the outer loop has sufficient parallelism and version (ii) is used.

Futhark's GPU backend tiles the sequentialized inner loop, as in the baseline implementation. This optimization is not implemented for Futhark's CPU backend, which instead depends solely on the hardware caches. The source code uses an AoS layout, but the compiler always transforms it to the SoA layout.

### *6.4. N-body Simulation with Accelerate*

The Accelerate $N$-body simulation code is similarly discussed as the running example in Section 2.3. In the implementation the acceleration between each pair of bodies is calculated by expanding the vector of bodies once along the rows, once along the columns (using `replicate`), and zipping the two resulting matrices with the `accel` function. All rows of the resulting matrix are then summed in parallel using a (rank-polymorphic) `fold`, resulting in a vector. Fusion is essential here, as it would be inefficient, both in terms of runtime and memory allocation, to create the intermediate matrices.

### *6.5. N-body Simulation with Single Assignment C (SaC)*

The SaC $N$-body simulation code is likewise discussed in Section 2.4. To support vectorization in the CPU implementation, the code uses a struct for the $x, y, z$-coordinates instead of a `double[3]` array. The compiler eliminates the struct, and the generated code deals with three independent vectors of scalar coordinates. This ensures that the $x$-coordinates of consecutive bodies are contiguous in memory, and likewise for the $y, z$ coordinates and for the masses.

### *6.6. N-body Simulation with APL*

The APL $N$-body simulation code is also described in Section 2.5. In the implementation positions and velocities are stored following an SoA model of nested arrays of 3 elements corresponding to the $x$, $y$, and $z$ coordinates. If the conceptual computation is viewed as a summation of each row of an $N \times N$ matrix, the APL code tiles the matrix into a series of $N \times B$ columns to compute

a partial summation of each row a tile at a time. Each tiling pass first computes the differences $d$ for each dimension and then computes the weight matrix $w$ from these differences. To compute the velocity shifts based on these weights and each point's mass $m$, we utilize the expression `(d×w)+.×m[⍵]`, which leverages the `+.×` matrix multiplication in APL.

This particular ordering allows for each tile to first be computed as a fused kernel(s) of the differences and the weight computation before the fast matrix-multiplication libraries are used in the final computations. This allows the code to make use of the facilities in the APL runtime since fusion in Co-dfns only occurs on primitives and not on user-defined functions. It also enables portions of the problem to be mapped to fast matrix multiplication code in the runtime.

### *6.7. N-body Simulation with DaCe*

The DaCe $N$-body simulation code is likewise discussed in Section 2.6. The CPU implementation starts with the $N$-body Python program in Fig. 7 and we optimize the resulting SDFG with the custom workflow summarized in Fig. 9. The input data are matrices of size $3 \times N$ and $4 \times N$, corresponding to a SoA representation.

For GPU execution, we use a Python program with two main differences compared with the CPU implementation. The first is additional tiling of one of the inner loops and the use of GPU shared memory to reduce the number of loads from GPU global memory. The second is the transposition of the matrices to produce an AoS representation and allow the compiler to vectorize the load operations from GPU shared memory.

### *6.8. Benchmarking Results and Summary*

Table 5 summarizes the $N$-body compute rate for the five languages on both multicore CPU and GPU.

Table 5: $N$-body compute rate in Gflop/s (higher is better) for different numbers of bodies ($n$) and iterations ($t$). CPU performance is reported for execution on 1 and 32 cores.

| | $n = 10^3$, $t = 10^5$ | | | $n = 10^4$, $t = 10^3$ | | | $n = 10^5$, $t = 10^1$ | | |
|---|---|---|---|---|---|---|---|---|---|
| | CPU | | GPU | CPU | | GPU | CPU | | GPU |
| | 1C | 32C | | 1C | 32C | | 1C | 32C | |
| Baseline | 18.9 | 199 | 449 | 19.0 | 559 | 691 | 18.9 | 584 | 1407 |
| Accelerate | 18.6 | 7.0 | 12.8 | 19.5 | 264 | 237 | 19.5 | 558 | 246 |
| APL | 1.4 | — | 36.2 | 1.5 | — | 40.4 | 1.4 | — | 48.4 |
| DaCe | 19.1 | 199 | 117 | 19.1 | 546 | 972 | 19.0 | 595 | 1640 |
| Futhark | 19.2 | 134 | 574 | 19.3 | 535 | 1021 | 19.2 | 600 | 1663 |
| SaC | 19.3 | 127 | 37.6 | 19.4 | 559 | 222 | 19.4 | 597 | 264 |

*GPU performance.* Futhark and DaCe beat the GPU baseline on the second and third datasets with speedup factors between $1.17-1.48\times$. Futhark is competitive with the baseline on the first dataset, but DaCe lags behind because it utilizes only the parallelism of the outer loop, which is insufficient. Accelerate and SaC come next with SaC achieving 8 %, 32 %, and 19 % of the baseline performance on the three datasets and Accelerate achieving 3 %, 34 %, and 17 %; the first dataset similarly suffers a drop in performance due to insufficient utilization of parallelism. APL provides useful acceleration—in that its GPU compute rate is significantly higher than the sequential compute rate—but its performance is only as high as 8 %, 6 %, and 3 % of the GPU baselines. Further analysis of the performance of the $N$-body GPU implementations can be found in Appendix B.

*Multicore performance.* SaC, Futhark, and DaCe are competitive with the baseline on the second and third datasets (exceeding 95 %). DaCe also matches the baseline on the first dataset (100 %), while SaC and Futhark achieve 64 % and 67 %, respectively. Accelerate provides good performance on the third (large) dataset (95 %) but performs less well on the second (47 %) and first (4 %) datasets.

Futhark and DaCe present the best blend of performance across the two hardware platforms. SaC is best developed for multicore execution, and its suboptimal GPU performance is directly related to its GPU backend receiving much less attention, and conversely for Accelerate. These performance trends are reflected in the other benchmarks.

*Expression.* The baseline consists of three low-level C++ code versions — two aimed at GPU and one aimed at multicore execution — that use *different layouts* (SoA *vs.* AoS) and *manually-applied optimizations* (see Section 6.2). In comparison, the functional array languages use a *single* high-level code (see Section 2) that is *automatically optimized* for multicore and GPU execution. Subjectively, all $N$-body implementations seem significantly more elegant than the baseline, with a special mention of APL for its brevity; our code repository is available to the interested readers to make up their own minds.

## 7. Benchmark 2: MultiGrid (NAS MG)

Our second benchmark is MultGrid from the computational fluid dynamics domain. More precisely, we adopt the benchmark MG from the The NAS parallel benchmark suite [62].

### *7.1. Benchmark Description*

Multigrid methods are employed to numerically solve differential equations. The NAS MG benchmark is a simplified version of the V-cycle multigrid method to solve the discrete Poisson equation

$$\nabla^2 u = v \tag{7}$$

**Algorithm 1** The V-cycle function $M$

**procedure** M(r)
  **if** $n \neq 2$ **then**
    $rs = P(\text{f2c}(r))$
    $zs = M(rs)$
    $z = Q(\text{c2f}(zs))$
    $r = r - A(z)$
    $z = z + S(r)$
  **else**
    $z = S(r)$
  **return** z

**Algorithm 2** MG

**Input:** $v$ : an $n \times n \times n$ array.
**Input:** $t$ : a number of iterations.
**Output:** $u$ : an $n \times n \times n$ array approximately satisfying $\nabla^2 u = v$.
  $u = 0$
  **for** $i = 1$ **to** $t$ **do**
    $r = v - A(u)$
    $u = u + M(r)$

Figure 10: MultiGrid method for numerically solving $\nabla^2 u = v$.

on a $n \times n \times n$ grid with periodic boundary conditions. The Laplacian $\nabla^2$ is approximated with a trilinear finite element discretization $A$, which is defined as a function taking and returning an $n \times n \times n$ grid of real numbers:

$$\begin{aligned} A(x))[i_1, i_2, i_3] = \\ \sum_{j_1, j_2, j_3 = 0}^{2} w[j_1, j_2, j_3] \cdot x[&(i_1 + j_1 - 1) \bmod n, \\ &(i_2 + j_2 - 1) \bmod n, \\ &(i_3 + j_3 - 1) \bmod n] \end{aligned} \tag{8}$$

for some weights $w \in \mathbb{R}^{3\times3\times3}$. This type of computational pattern is called a 27-*point stencil with periodic boundary conditions*.

We define a function `f2c` for projecting a fine grid on a coarser grid. This maps $x$ to $x(1 : n : 2, 1 : n : 2, 1 : n : 2)$, where we use the Fortran triplet notation for the index range from 1 to $n$ with step 2, in each dimension. Similarly, we have a function `c2f`, which refines a coarse grid into a fine grid. This step turns some array $x$ of size $n \times n \times n$ into an array $y$ of size $2n \times 2n \times 2n$ as $y(1 : n : 2, 1 : n : 2, 1 : n : 2)$. The remainder of $y$ are zeros.

The computational core of MultiGrid is the V-cycle shown in Algorithm 1. This is a function composed of $A$ and three more 27-point stencils $P, Q, S$. The recursive call causes the stencils to operate on progressively coarser grids until the input has shape $2 \times 2 \times 2$, after which the grid being refined step by step. The MultiGrid benchmark, Algorithm 2, repeatedly uses the V-cycle function $M$ to update a solution to Eq. (7).

*7.2. Baselines, Performance Measure, and Datasets*

The CPU baseline selected is the Fortran+OpenMP implementation from NAS[7] [56]. The selected GPU baseline is the CUDA implementation of NPB-GPU[8] [57]. Both baselines perform an optimization that exploits a domain-specific property of the $3 \times 3 \times 3$ array of weights, namely that the weights situated at the same distance from the center (at index $[1,1,1]$) are equal, i.e., the 27 weights only have 4 distinct values. It follows that the number of FLOPs per element can be significantly reduced from the 26 additions and 27 multiplications of a naive implementation, e.g., by performing rewrites of the form $w_1a + w_1b + w_1c + w_1d = w_1(a + b + c + d)$. Further, the number of additions can also be reduced by observing that for a portion of a slice in the $xy$-plane:

$$\begin{pmatrix} * & - & * \\ - & \circ & - \\ * & - & * \end{pmatrix}$$

the sum of the $*$s and $-$s are used to update not only $\circ$, but also the points directly above and below $\circ$. This can be exploited by precomputing, for each $(i,j)$, two temporary buffers containing the sums $x[i{+}1,j,.]{+}x[i{-}1,j,.]{+}x[i,j{-}1,.]{+}x[i,j{+}1,.]$, and $x[i{+}1,j{+}1,.]{+}x[i{-}1,j{-}1,.]{+}x[i{+}1,j{-}1,.]{+}x[i{-}1,j{+}1,.]$. The temporary buffers are then used to update $x[i,j,.]$. This technique not only reduces the number of operations, but also enables temporal reuse, e.g., the CUDA baseline allocates the temporary buffers in shared memory, thus reducing the number of accesses to global memory [63].

We report performance in Gflop/s following the same methodology as in the NAS benchmark, where the number of FLOPs for an $n \times n \times n$ grid and $t$ iterations is considered to be $58tn^3$. The evaluation uses the *Class A*, *Class B*, and *Class C* NAS datasets.

*7.3. MultiGrid with Futhark*

The Futhark implementation supports both the naive and optimized versions of MultiGrid introduced in Sections 7.1 and 7.2. A simplified code excerpt from the latter is shown in Fig. 11. In contrast to the lengthy baseline implementation (4662 source lines of code in Table 9) that consists of a multitude of distinct (specialized) computational kernels, the Futhark implementation abstracts (most of) the core computation of the 27-point stencil into *one* function that is suitably parameterized, e.g., over (i) the size of the result-array dimension $n$, (ii) an index-function representation $elmAt$ of the source array and (iii) the array of weights $ws$. The instantiations required to compute $Q(\texttt{c2f}(zs))$, $r - A(z)$ and $z + S(r)$ in Algorithm 1 are shown on the right-hand side of Fig. 11, as functions named simply $Q$, $A$ and $S$, respectively. Finally, the function parameter $relaxKer$ may be instantiated with either the naive (not shown) or

[7] https://www.nas.nasa.gov/software/npb.html
[8] https://github.com/GMAP/NPB-GPU

```
1  def relaxNAS (n:i64)
2              (elmAt:i32->i32->i32->real)
3              (ws: [4]real) : *[n][n][n]real =
4   let computeRow (ii: i64)(jj: i64) : [n]real =
5      let (i, j) = (i32.i64 ii, i32.i64 jj)
6      let f (k: i32) =
7         ( elmAt i ((j-1) % n) k +
8           elmAt i ((j+1) % n) k +
9           elmAt ((i-1) % n) j k +
10          elmAt ((i+1) % n) j k
11        ,
12          elmAt ((i-1) % n) ((j-1) % n) k +
13          elmAt ((i-1) % n) ((j+1) % n) k +
14          elmAt ((i+1) % n) ((j-1) % n) k +
15          elmAt ((i+1) % n) ((j+1) % n) k
16        )
17     let tab n h = map (h<-<i32.i64) [0...n-1]
18     let (u1s, u2s) = tab n f
19     let g u1s u2s (k: i32) = #[unsafe]
20           ws[0] * elmAt i j k
21        + ws[1] * (u1s[k] +
22                   elmAt i j ((k-1) % n) +
23                   elmAt i j ((k+1) % n))
24        + ws[2] * (u2s[k] + u1s[(k-1) % n] +
25                            u1s[(k+1) % n])
26        + ws[3] * (u2s[(k-1)%n] + u2s[(k+1)%n])
27     in  tab n g
28  in  map (\i -> map (computeRow i) (0...n-1))
29          (0...n-1)
30
```

```
def getEachElm arr i j k =
  #[unsafe] arr[i, j, k]

def get8thElm arr i j k =
  if (i%2) + (j%2) + (k%2) == 3
  then #[unsafe]
       arr[i/2,j/2,k/2]
  else 0.0

def Q [n] relaxKer
          (zs: [n][n][n]real)
        : [2*n][2*n][2*n]real =
  relaxKer (2*n) (get8thElm zs)
           [1, 1/2, 1/4, 1/8]

def A [n] relaxKer
          (z: [n][n][n]real)
          (r: [n][n][n]real)
        : [n][n][n]real =
  relaxKer n (getEachElm z)
           [-8/3, 0, 1/6, 1/12]
  |> map2 (map2 (map2 (-))) r

def S [n] relaxKer
          (ws: [4]real)
          (r: [n][n][n]real)
          (z: [n][n][n]real)
        : [n][n][n]real =
  relaxKer n (getEachElm r) ws
  |> map2 (map2 (map2 (+))) z
```

Figure 11: The left-hand side shows a *simplified* implementation of the optimized computational kernel of MultiGrid, named `relaxNAS`, which is made generic by parameterizing it over an index function (`elmAt`) rather than a 3-dimensional array. The right-hand side shows the instantiations required to implement the computations $Q(\texttt{c2f}(zs))$, $r - A(z)$ and $z + S(r)$ in Algorithm 1. Of note, the actual implementation uses fast arithmetic for modulo operations. `#[unsafe]` prevents the insertion of dynamic bounds checks (which are expensive on GPUs).

optimized version (*relaxNAS*). On the downside, Futhark does not support arbitrary recursion. It follows that the recursive procedure $M$ (of Algorithm 1) is verbosely implemented with a while loop that explicitly maintains the stack.

The high-level implementation critically relies on the compiler to specialize the code. For example, Futhark's simplification engine aggressively inlines functions such as *relaxNAS*, *getEachElm*, *get8thElm*, $f$, $g$ at call sites, and performs a battery of transformations, such as copy propagation, constant folding, common subexpression elimination, dead variable elimination, and scalar specialization of array-literal indexing. For example, the specialization of *relaxNas* at the call site of $A$ (in Fig. 11) will eliminate the whole term starting with $ws[1] * (\ldots)$ appearing in the computation of $g$ (at lines 21-23) — because $ws[1]$ corresponds to constant 0 in this context, and multiplication with 0 results in 0.

Furthermore, as presented in Section 4.1, for the GPU case, the incremental flattening transformation will generate an "intra-block" code version that essentially maps the (inner) parallelism available within *computeRow* to the CUDA block level and the outer parallelism to the CUDA grid level, i.e., the two nested *map*s at line 28. Consequently, the arrays $u1s, u2s$ produced by *tab* $n$ $f$ (line 18) are placed in shared memory and reused from there in the

computation of *tab n g* (line 27). Finally, memory-related optimizations eliminate some of the array copying operations resulting from explicit modeling of the (recursion) stack.

*7.4. MultiGrid with Accelerate*

The Accelerate implementation uses recursion to implement procedure `M`, despite Accelerate, like Futhark, not supporting recursion. It does so by exploiting the recursion provided by the Haskell host language via meta-programming. This unfolds the recursion, similar to how loop unrolling converts a loop to a sequence of instructions.

Accelerate features stencil primitives that simplify the implementation of relaxations. Again, using meta-programming, the weights are inlined into the stencil kernel.

Accelerate's fusion algorithm fuses `f2c` and `c2f` into the following stencil operation. This improves performance while at the same time allowing the user to write the algorithm at a higher level.

*7.5. MultiGrid with Single Assignment C (SaC)*

SaC implements the MultiGrid benchmark recursively, faithful to the specification. We use the generic stencil implementation shown in Fig. 12. This function generalizes to arbitrary rank `d` and weights of arbitrary shape `wshp`. The modulo computations (`mod`) are optimized away by generating multiple partitions.

```
double[d:shp] stencil(double[d:shp] x, double[d:wshp] w)
{
  return {i -> sum({j -> w[j] * x[mod(i + j - wshp / 2, shp)]})
              | i < shp};
}
```

Figure 12: A rank-polymorphic stencil in SaC. For suitably chosen weights `w`, this computes the scaled (higher-order) derivative of a discretized function `x`.

SaC's constant folder currently reduces the number of multiplications automatically, but not the number of additions. The Fortran version has one temporary array per thread, which it reuses. As SaC does not have explicit parallelism, it is far from straightforward to mimick the Fortran behavior. An alternative would be to compute the temporary arrays for all $(i, j)$ in parallel, but that would likely suffer from poor cache performance.

For GPU code generation, SaC assumes by default that all data resides on the CPU. When a with-loop is executed on the GPU, the compiler generates transfers to and from the GPU for the arguments and the result. The compiler then tries to optimize these transfers away. However, the recursive nature of the MultiGrid benchmark prevents the compiler from deriving the calling context in most cases, and so it fails to eliminate many of the data transfers. Unnecessary data movement between CPU and GPU results in a slowdown with respect to single-threaded performance. Hence, we refrain entirely from reporting GPU results for MultiGrid.

*7.6. MultiGrid with APL*

The APL implementation is a fairly direct translation of the pseudo code:

```
M←{α≤1:S st ω ⋄ z←S st ω-A st⊢z←Q st up(α-1)∇dn P st ω}
L2←{((+⌿,ω×ω)÷×⌿ρω)*.5}
MG←{⍬L2 α-A st α{⍬ω+k M α-A st ω}⍣IT⊢ω}
```

Where $MG$ is run for $IT$ iterations and the functions $st$, $dn$, and $up$ correspond to the stencil computation, downsampling, and upsampling, respectively. Downsampling and upsampling are implemented as direct array indexing operations:

```
dn←{ω[i;i;i←1+2×⍳(≢ω)÷2]}                  ⍝ Downsample
up←{X⊣X[i;i;i←1+2×⍳≢ω]←ω⊣X←1e¯100⍴⍨2×ρω} ⍝ Upsample
```

APL does have a built-in stencil operator, but it is not suitable for high-performance optimizations such as those needed for MultiGrid. We instead implement a custom stencil function $st$ that minimizes the number of arithmetic operations as described in Section 7.2. Since the entire function can be expressed as a single composition of arithmetic primitives and rotations, the resulting code can be fused into a single conceptual kernel:

```
⍝ MG Stencil Operation
st←{k←α
    x←ω×k[0]
    x+←k[1]×(1⌽ω)+(¯1⌽ω)+r0←(1⌽ω)+(¯1⌽ω)+r01←(1⌽[1]ω)+¯1⌽[1]ω
    x+←k[2]×(1⌽r0)+(¯1⌽r0)+r1←(1⌽r01)+¯1⌽r01
    x+←k[3]×(1⌽r1)+¯1⌽r1
    ⍬⊃x
}
```

*7.7. MultiGrid with DaCe*

Since the SDFG IR does not support recursion, the DaCe implementation of MultiGrid is based on a Python version [64]. We automatically optimize the CPU and GPU implementations, which fuses parallel computations (Map scopes) and tiles those that contain write-conflict resolutions to minimize atomic operations. However, as the auto-optimizer transforms the graph in a specific order, some optimization opportunities are missed. For example, Fig. 13 shows the MultiGrid `norm2u3` method and its DaCe-Python implementation, which includes two map-reduce patterns for computing `s` and `rnmu`. The auto-optimizer successfully fuses these computations into a single Map scope but cannot subsequently tile it because the relevant DaCe optimization does not support Map scopes with write (WCR) conflicts to multiple data containers. To work around this limitation, we design a custom workflow. First, we apply the `MapReduceFusion` transformation repeatedly until no subgraph matches the optimization pattern, resulting in two fused Map scopes, one for `s` and one for `rnmu`. Only then do we apply WCR tiling, which matches since each scope writes to a single data container.

```
n0i, n0j, n0k = \
  (dace.symbol(s, dtype=dace.int32) for s in ('n0i', 'n0j', 'n0k'))

@dace.program
def norm2u3_dace(r: dace.float64[n0i, n0j, n0k], dn: dace.int32):
    s = dace.reduce(lambda a, b: a + b,
                    r[1:-1, 1:-1, 1:-1] * r[1:-1, 1:-1, 1:-1],
                    identity=0.0)
    rnmu = dace.reduce(lambda a, b: max(a, b),
                       numpy.abs(r[1:-1, 1:-1, 1:-1]),
                       identity=0.0)
    rnm2 = numpy.sqrt(s / dn)
    return rnm2, rnmu

fast_sdfg = norm2u3_dace.to_sdfg()
auto_optimize(fast_sdfg)

faster_sdfg = norm2u3_dace.to_sdfg()
faster_sdfg.apply_transformations_repeated([MapReduceFusion])
tile_wcrs(faster_sdfg)
greedy_fuse(faster_sdfg)
```

Figure 13: DaCe Python implementation of `norm2u3` and its optimization.

### *7.8. Benchmarking Results and Summary*

The MultiGrid performance results for the five languages on both multicore CPU and GPU are summarized in Table 6. We follow the baseline performance measurement and report compute rate (Gflop/s).

Table 6: MultiGrid compute rate in Gflop/s (higher is better) for NAS input classes A, B, and C.

| | A | | | B | | | C | | |
|---|---|---|---|---|---|---|---|---|---|
| | CPU | | GPU | CPU | | GPU | CPU | | GPU |
| | 1C | 32C | | 1C | 32C | | 1C | 32C | |
| Baseline | 7.0 | 77.3 | 146 | 7.5 | 88.3 | 222 | 6.7 | 44.8 | 223 |
| Accelerate | 1.5 | 6.6 | 23.9 | 1.7 | 7.9 | 24.9 | 1.1 | 13.7 | 79.3 |
| APL | 0.5 | — | 11.2 | 0.5 | — | 11.2 | 0.5 | — | 8.8 |
| DaCe | 3.1 | 25.8 | 120 | 3.4 | 38.8 | 131 | 2.8 | 34.0 | 216 |
| Futhark | 1.2 | 14.7 | 224 | 1.4 | 24.8 | 232 | 1.4 | 23.6 | 233 |
| SaC | 5.3 | 46.5 | — | 6.4 | 56.3 | — | 5.3 | 40.4 | — |

*GPU performance.* Futhark offers consistent performance across all datasets ($224 - 233$ Gflop/s), and outperforms the baseline across all datasets (153 %, 105 %, and 104 %). DaCe also offers competitive performance on the largest `C` dataset (97 %), but is less competitive for the first two datasets (82 % and 59 %). Accelerate offers variable performance across datasets ($24 - 79$ Gflop/s) reaching 16 %, 11 %, and 36 % of the baseline performance. APL reports useful acceleration in comparison with the sequential execution, but reaches only 4 % to 8 % of the baseline performance. For SaC, compiler shortcomings have prevented GPU execution.

*Multicore performance.* The baseline outperforms all of the functional array languages, likely due to suboptimal thread-binding policies. SaC performs best, reaching 60 %, 64 %, and 90 % of baseline performance on the `A,B,C` datasets, followed by DaCe with 33 %, 44 %, and 76 %. At a considerable distance come Futhark (19 %, 28 %, and 53 %) and Accelerate (9 %, 9 %, and 31 %), whose multicore backends are less mature than the GPU backends.

*Expression.* The Accelerate, APL, Futhark, and SaC teams found it infeasible to develop their code by following the long, low-level, and complex baseline code (4662 source lines of code, Table 9). The baseline specializes and hand-optimizes one generic stencil computation for a multitude of contexts. Instead, our implementation teams based their implementations either on the NAS specification or the SaC code, which is short, clean, and arguably very close to the algorithmic formulation (Sections 7.1 and 7.2). Moreover, Fig. 11 shows that the key baseline optimizations, providing temporal reuse and a reduced number of arithmetic operations, can also be expressed generically and concisely without compromising GPU performance, as evidenced by Futhark's performance. Although code comparisons are always subjective, we invite the reader to browse through our code repository [9].

## 8. Benchmark 3: Quickhull

Our third benchmark is Quickhull from the computational geometry domain.

### *8.1. Benchmark Description*

The Quickhull benchmark computes the convex hull of a finite set of points in 2-dimensional space. Algorithm 3 shows the pseudocode of the Quickhull algorithm: given a set $S$ containing at least 2 two-dimensional points, the convex hull of $S$ is computed by first (i) identifying the leftmost and rightmost points of $S$, named $A$ and $B$, respectively, then (ii) by partitioning $S$ into subsets $S_1$ and $S_2$ corresponding to the points below and above the line $AB$, and finally (iii) by combining the convex hulls of $S_1$ and $S_2$. The last step is facilitated by the $findHull$ helper function presented in Algorithm 4. The divide-and-conquer $findHull$ function takes two points $P$ and $Q$ and a set of points $S$ that are either all above or all below line $PQ$. The $findHull$ function computes the point $C$ of $S$ that is furthest away from $PQ$, and hence must belong to the convex hull. Then, it eliminates from $S$ the points inside the triangle $\Delta CPQ$ as they are guaranteed not to belong to the convex hull. The resulting set is partitioned into the points that are on the right- and left-hand side of lines $CP$ and $CQ$, named $S_l$ and $S_r$, respectively. The convex hull of $S$ consists of point $C$ together with the convex hulls of $S_l$ and $S_r$.

[9] https://github.com/diku-dk/CFAL-bench/

**Algorithm 3** Quickhull

**Input:** $S$: a set of $n \geq 2$ two-dimensional points
**Output:** $CH$: the convex-hull set of $S$
$(A, B)$ = the leftmost and rightmost points of $S$
$S_{1,2}$ = points of $S$ above and below line $AB$
$CH = \{A, B\} \cup$ findHull($S_1$, A, B) $\cup$ findHull($S_2$, A, B)

**Algorithm 4** Divide-And-Conquer Helper

```
1:  procedure findHull(S, P, Q)
2:      if S ≠ ∅ then
3:          C = furthest point of S from line PQ
4:          (S_l, S_r) = the points of S on the left-
5:                       and right-hand side of lines
6:                       CP and CQ, respectively
7:                       (and not inside ΔPCQ)
8:          Hull = {C} ∪
9:                 findHull(S_l, P, C) ∪
10:                findHull(S_r, C, Q)
11:     else
12:         Hull = ∅
13:     return Hull
```

Such divide-and-conquer recursion induces *irregular nested parallelism*: the recursive calls form a binary tree where each call performs parallel operations (e.g., filtering and partitioning) on arrays of different sizes. Importantly, all nodes at the same level in the tree can be processed in parallel.

Ideally, the Quickhull implementation should resemble the divide-and-conquer structure of Algorithm 4. This approach was pioneered in the NESL language [23]. It can be automatically mapped to hardware using flattening transformations [35] and may target multicore [65] or GPU architectures [66, 67]. However, such solutions have not yet demonstrated performance competitive with manually-flattened code. Since none of the studied languages support flattening of irregular parallelism, their corresponding implementations are derived by manually flattening the code corresponding to Algorithm 4.

### *8.2. Baselines, Performance Measure, and Datasets*

The CPU baseline selected is the convex-hull implementation of the PBBS benchmark[10] [58], that offers the best multicore performance out of implementations that are described in peer-reviewed papers. The implementation uses a fork-join (task-based) strategy that dynamically decomposes and exploits both levels of parallelism while tasks have good granularity. This strategy results in cache-friendly behavior that cannot be replicated by any of the studied languages, given their data-parallel paradigm. This paradigm effectively enforces barriers around the parallel processing of the nodes at each level in the tree, resulting in larger reuse distances.

We do not use a GPU baseline as, other than the studied languages, we could not find a publicly available implementation competitive with the PBBS

[10] https://github.com/cmuparlay/pbbsbench

implementation. This emphasizes the importance of high-level data-parallel models offering efficient hardware mappings, e.g., to GPUs.

We follow PBBS in reporting performance as runtime. The evaluation uses three PBBS datasets, each consisting of a set of one hundred million points that are uniformly distributed in the interior of a *Rectangle*, in the interior of a *Circle* (a disk), and on a *Quadratic* curve. Their convex hulls consist of 49, 1681, and 548136 points, respectively.[11]

### *8.3. Quickhull with Futhark*

The Futhark implementation of Quickhull has been manually flattened to be expressible with flat data parallelism. At a high level, it consists of an outer `while`-loop that tracks the points determined to be part of the hull, as well as a sequence of sets of points that have not yet been classified. This sequence is conceptually a multidimensional jagged array and is represented using a standard flag array approach. The implementation consists of a composition of map, reduce-by-index, filter, and partition operations, where the implementation of the latter two relies on scan (prefix sum) and scatter (parallel write). Efficient execution on GPU hardware is enabled by specialized code generation for scan and reduce-by-index (Section 4.1).

### *8.4. Quickhull with Accelerate*

As with Futhark above, the Accelerate implementation is likewise flattened manually, making it necessary to maintain some segment information. The main work during one step is a parallel filter operation, applied to all segments. This consists of a mapping operation to determine which points to discard, a segmented scan over the resulting flag vector to compute the target indices, and a subsequent permutation.

### *8.5. Quickhull with APL*

The APL version first reorders the points so that membership in the hull set can be tracked using a Boolean mask to partition the data vectors into segments. On each iteration, this mask is used to group each point and determine membership within the hull set. Points falling within the hull are removed, while the new point that divides each segment is computed and added to the set by flipping its bit in the bit mask.

APL currently lacks a method for computing scans or reductions over subsegments of an array indicated by a key. Instead, the expression `M[R≠⌿K[M←⍒D]]` computes the first indices of $D$ that have the maximum $D$ for each unique key in $K$. This expression is reasonably efficient, but involves sorting two vectors to find the maximum, which is significantly less efficient than a max reduction might be. Moreover, the above code does not split out the hull elements from elements still to be determined. As a result, if the number of points on the hull

[11] The datasets are named as in PBBS and are generated with the PBBS infrastructure.

is large, we may redundantly compute distances that do not matter over time, leading to reduced performance when the hull contains many points. Finally, reordering the points at the beginning instead of at the end requires sorting a much larger array.

### *8.6. Quickhull with Single Assignment C (SaC)*

We implement the Quickhull algorithm recursively. As SaC is syntactically very close to C, we can straightforwardly adapt the partition scheme from Edelkamp [68]. We also find the furthest points in the same loop that partitions $S$ into $S_l$ and $S_r$ to reduce the number of passes over the data. As PBBS performs a Hoare partitioning and does not merge these loops, the SaC implementation is at least twice as fast compared with PBBS in sequential execution. PBBS also works with indices instead of permuting the array, which gives bad cache behavior for deep recursion. Therefore, SaC is even five times faster than the baseline for the Quadratic data set.

Unfortunately, the SaC compiler is not able to parallelize recursive calls and does not support the parallel scatter operation. These shortcomings prevent us from parallelizing Quickhull or its flattened variant. Therefore, we cannot show any parallel performance results, neither on CPU nor on GPU.

### *8.7. Quickhull with DaCe*

DaCe does not support recursion and, therefore, cannot directly implement Algorithm 4. The alternative formulation using flattening transformations, described in Section 8.1, could be implemented in DaCe by composing control flow (for-loops) with map scopes and reductions to compensate for the lack of (segmented) scan primitives. However, such an implementation would be tedious and uncompetitive with languages that can express those parallel paradigms naturally. The Library Node mechanic could also be leveraged to map a high-level (segmented) scan representation directly to optimized calls, e.g., to the NVIDIA Thrust library for GPUs. Overall, we consider the additions required to facilitate a reasonable Quickhull implementation in DaCe beyond the scope of this work.

### *8.8. Benchmarking Results and Summary*

It is not feasible to develop parallel Quickhull implementations in DaCe or SaC, mainly because they lack support for a key operation like prefix-sum or scatter, or for fork-join parallelism. The Quickhull performance results of Accelerate, APL, and Futhark on multicore CPU and GPU are summarized in Table 7. We follow the baseline performance measurement and report runtime (in seconds).

*GPU performance.* Exceptionally, the performance of the Quickhull GPU implementations are compared with the multicore baseline as no GPU baseline is available. Futhark outperforms the multicore baseline, achieving 340 %, 259 %, and 541 % of its performance. Accelerate is competitive with the baseline on the

Table 7: Quickhull runtime in seconds (lower is better) for $10^8$ points sampled uniformly from three geometric shapes. Note that the baseline runs exclusively on CPU and all language runtimes are compared against the CPU baseline, see Section 5.3.

| | Circle | | | Rectangle | | | Quadratic | | |
|---|---|---|---|---|---|---|---|---|---|
| | CPU | | GPU | CPU | | GPU | CPU | | GPU |
| | 1C | 32C | | 1C | 32C | | 1C | 32C | |
| Baseline | 3.1 | 0.2 | — | 2.3 | 0.1 | — | 27.1 | 2.9 | — |
| Accelerate | 7.0 | 3.5 | 0.7 | 4.2 | 2.7 | 0.1 | 78.5 | 24.2 | 4.5 |
| APL | 20.9 | — | 1.2 | 13.8 | — | 0.7 | 114 | — | 7.6 |
| DaCe | — | — | — | — | — | — | — | — | — |
| Futhark | 6.6 | 1.4 | 0.1 | 4.8 | 1.3 | 0.0 | 36.3 | 4.1 | 0.5 |
| SaC | 2.2 | — | — | 1.9 | — | — | 7.2 | — | — |

Rectangle (81 %) dataset, but falls behind on the Circle (30 %) and Quadratic (66 %) datasets. While APL reduces runtimes relative to sequential performance, it reaches only 16 %, 17 %, and 39 % of the baseline performance on the Circle, Rectangle, and Quadratic datasets, respectively.

*Multicore performance.* The Accelerate and Futhark implementations are obtained by manually applying a flattening transformation [35] to the divide-and-conquer (irregular) parallelism exposed by Quickhull. The baseline outperforms all languages as it uses a fork-join style that dynamically spawns parallel tasks that have good granularity and temporal reuse (Section 8.2). Futhark achieves 14 %, 9 %, and 72 % and Accelerate achieves 6 %, 4 %, and 12 % of the baseline performance on the three datasets.

*Expression.* It seems that Quickhull is best expressed in a divide-and-conquer parallel style, similar to Algorithm 4, where the text on lines 3 and 5–8 is implemented with data-parallel constructs such as map-reduce and partition. This would offer a clean and algorithm-faithful high-level specification that preserves the opportunities for efficient mapping to different architectures, e.g., by specializing the flattening transformation for multicore [65] or GPU [66, 67] execution. In principle, this would produce (1) efficient multi-core code resembling the baseline implementation and (2) efficient GPU code resembling the Accelerate and Futhark implementations. We consider that our implementations are shorter, easier to understand, and more high-level than the baseline. For example, they contain no notion of memory or memory management and provide correct-by-construction parallelism enabled by implicitly-parallel constructs.

## 9. Benchmark 4: Custom and Flash Attention

Our fourth and final benchmark is Custom or Flash Attention from the deep learning domain.

**Algorithm 5** Standard Self-Attention

**Input:** $\mathbf{Q} : N \times d$ matrix of query weights.
**Input:** $\mathbf{K} : N \times d$ matrix of key weights.
**Input:** $\mathbf{V} : N \times d$ matrix of value weights.
**Output:** $\mathbf{O} : N \times d$ attention matrix.

1: $\mathbf{S} = \mathbf{Q} \times \mathbf{K}^T$
2: $\mathbf{P} = \mathit{softmax}(\mathbf{S})$
3: $\mathbf{O} = \mathbf{P} \times \mathbf{V}$

**Algorithm 6** Custom Attention

**Input:** same as Algorithm 5
**Output:** same as Algorithm 5

1: **for all** $i \in 0 \,..\, N-1$ **by** $d$ **do**
2: $\quad \mathbf{Qb} = \mathbf{Q}_{i:i+d,\,:}$
3: $\quad \mathbf{Sb} = \mathbf{Qb} \times \mathbf{K}^T$
4: $\quad \mathbf{Pb} = \mathit{softmax}(\mathbf{Sb})$
5: $\quad \mathbf{O}_{i:i+d,\,:} = \mathbf{Pb} \times \mathbf{V}$

Figure 14: In the code, $\mathbf{K}^T$ denotes the transpose of $\mathbf{K}$ and $\times$ denotes matrix multiplication. **forall** $i \in 0 \,..\, N-1$ **by** $d$ denotes a parallel (map-like) computation in which $i$ starts from 0 and advances with a step of $d$, i.e., $i = 0, d, 2 \cdot d, \ldots$. $\mathbf{Qb}$ iterates over the $d \times d$ slices of $\mathbf{Q}$ and the last line of Algorithm 6 computes the corresponding slice of the result $\mathbf{O}$.

### *9.1. Benchmark Description*

Self-attention is a machine-learning mechanism for finding dependencies between inputs in a sequence [69]. We consider a sequence of $N$ words or tokens, each with an embedding $\mathbf{v}_i, i \in 1..N$ of length $M$. We want to find another embedding $\mathbf{o}_i$ for each word that encodes its relationship to the other tokens. To that end, we select one word, the query, and compute its similarities $\mathbf{s}_{ij}, j \in 1..N$ to each other token as the dot product of their embeddings, $\mathbf{v}_i \cdot \mathbf{v}_j$. We normalize these similarities using Softmax [70], converting them to a probability distribution. Multiplying the result vector $\mathbf{p}_i$ with the initial word embedding produces the final result.

Machine learning architectures, like transformers, learn weights for the different embedding uses: $\mathbf{Q}$ for query words, $\mathbf{K}$ for the similarity computation, and $\mathbf{V}$ for the output embedding. In multi-head attention, multiple heads compute the attention corresponding to distinct parts of the word embeddings in parallel, each with size $d$. Hence, in Algorithm 5, the weight matrices have size $N \times d$. Standard Self-attention materializes all the $\mathbf{S}$ and $\mathbf{P}$ matrices, which have size $N \times N$. In applications of interest, the sequence is large enough to consider $N \gg d$, and $Q$, $K$, $V$, and $O$ to be tall and skinny matrices. Multiplying such matrices exhibits low arithmetic intensity (computation is $O\left(N^2 d\right)$, but memory operations are $O\left(N^2\right)$), which makes self-attention memory-bound.

Softmax, shown in Algorithm 7, normalizes the similarities matrix $\mathbf{S}$ by computing for every row $i$ two quantities: the maximum value $m_i$ and the normalization term $l_i$, defined as $l_i = \sum_{j=1}^{N} \exp(s_{i,j} - m_i)$. The output probabilities $\mathbf{P}$ are computed as $p_{i,j} = \exp(s_{i,j} - m_i)/l_i$ to avoid overflow. Overall, each input row must be read three times since the dependency on $m_i$ disallows loop fusion. This limits a tiled matrix-multiplication of $\mathit{softmax}(\mathbf{S}) \times \mathbf{V}$ to tiles of the form $d \times M$, for some $M$. As this is unacceptably large for keeping tiles in registers, an alternative formulation called *Online Softmax* [71], shown in Algorithm 8, removes this constraint by using exponent rules to compute $l_i$ recursively.

Flash Attention [59] utilizes Online Softmax to tile the matrix multiplications

**Algorithm 7** Stable Softmax

**Input:** **S**: $M \times M$ input matrix.

**for all** $i \in 0 \,..\, M-1$ **do**
  $m_i = \overline{max}(S_{i,:})$
  $\tilde{p} : \text{vect}[M]$
  **for all** $j \in 0 \,..\, M-1$ **do**
    $\tilde{p}_j = \exp(S_{i,j} - m_i)$
  $l_i = \overline{sum}(\tilde{p})$
  **for all** $j \in 0 \,..\, M-1$ **do**
    $P_{i,j} = \exp(S_{i,j} - m_i)/l_i$

**Algorithm 8** Online Softmax

**Output:** **P**: $M \times M$ output matrix.

**for all** $i \in 0 \,..\, M-1$ **do**
  $m_i = -\infty, l_i = 0$
  **for** $jj \in 0 \,..\, M-1$ **by** $T$ **do**
    $\tilde{s} : \text{vect}[T] = copy(S_{i,jj:jj+T})$
    $m_i'' = \overline{max}(\tilde{s})$
    $m_i' = \max(m_i,\ m_i'')$
    $\tilde{p} : \text{vect}[T]$
    **for all** $j \in 0 \,..\, T-1$ **do**
      $\tilde{p}_j = \exp(\tilde{s}_j - m_i')$
    $l_i' = \exp(m_i'' - m_i') \cdot \overline{sum}(\tilde{p})$
    $l_i = l_i' + l_i \cdot \exp(m_i - m_i')$
    $m_i = m_i'$
  **for all** $j \in 0 \,..\, M-1$ **do**
    $P_{i,j} = \exp(S_{i,j} - m_i)/l_i$

Figure 15: Algorithm 8 uses the same input and output as Algorithm 7. $\overline{max}$ and $\overline{sum}$ denote reductions of their vector argument. $\tilde{p}$ `: vect[`$X$`]` declares of a temporary vector $\tilde{p}$ of length $X$. **for all** denotes parallel (map-like) computations and **for** $jj \in 0 \,..\, M-1$ **by** $T$ denotes a sequential computation in which the loop index $jj$ advances with step $T$. Online Softmax is an algorithmic refinement that reorganizes the computation as a sequence of parallel computations of length $T$, where $T$ is a configurable tile size. This significantly reduces the number of accesses to global memory because vectors $\tilde{s}$ and $\tilde{p}$ now fit in scratchpad memory.

and the normalization operation, fuse them together, and perform the overall computation in $T_i \times T_j$ blocks. In this reformulation, the intermediate results are small enough to fit in fast memory (e.g., registers and CUDA shared memory), dramatically reducing accesses to slower global memory. An implementation of the algorithm in DaCe data-centric Python is shown in Fig. 17.

Finally, Algorithm 6 presents a midpoint between Standard and Flash Attention, dubbed *Custom Attention*, that blocks the computation over $d \times d$ slices of $Q$ and computes corresponding slices of the result **O**. Custom Attention does not serialize computation, but also does not guarantee that the intermediate matrices (**Sb** and **Pb** of size $d \times N$) can be maintained in fast memory. As this algorithm is not tiled for registers, it is unnecessary to use *softmaxOnline*.

### *9.2. Baselines, Performance Measure, Datasets*

The reference Flash Attention implementation [72] supports half-precision arithmetic (fp16 and bf16) and utilizes the specialized tensor (matrix) core units on NVIDIA (AMD) GPUs. Since not all languages support half-precision arithmetic or execution on tensor cores, we write our own GPU baseline using single-precision arithmetic and regular FPUs. Following the high-level algorithm in Fig. 17, each $T_i \times d$ block of the output **O** is assigned to a single thread block. The computation is done in a single kernel in $T_j$ steps and utilizes GPU shared

```
1  def mmmT [m][n][k] (as: [m][k]f32)(bs: [n][k]f32) : [m][n]f32 =
2    map (\a -> map (\b -> map2 (*) a b |> f32.sum) bs) as
3  def mmm [m][n][k] (as: [m][k]f32) (bs: [k][n]f32) : [m][n]f32 =
4    mmmT as (transpose bs)
5
6  def FlashAttention [d][m] (Q: [m][d][d]f32) (K: [m*d][d]f32)
7                            (V: [m*d][d]f32) : [m][d][d]f32 =
8    let mkOneBlock Qi =
9            let P = mmmT Qi K
10                |> onlineSoftmax
11           in  mmm P V
12   in  map mkOneBlock Q
```

Figure 16: Custom Attention in Futhark. The implementation of onlineSoftmax (not shown) corresponds to Algorithm 8

memory and registers to reduce loads from GPU global memory. Each thread block reads its assigned **O** and **Q** blocks and all **K** and **V** matrices. Therefore, **O** and **Q** are read only once, whereas **K** and **V** are loaded as many times as the number of thread blocks. All intermediate results, for example, the maximum values $m$ and normalization factors $l$, are written to shared memory and registers. The implementation further 2D tiles the computation of each thread block and utilizes memory coalescing to achieve high GPU global memory throughput.

The CPU baseline tiles the computation in the same way. Each $T_i \times d$ block of the output **O** is assigned to a single CPU thread. The matrix multiplications are executed with a specialized single-threaded implementation optimized for the AMD Zen architecture.

We report the performance in Gflop/s and consider only the operations present in the standard self-attention algorithm when measuring the workload, i.e., $N^2\,(4d+5)$ floating point operations. Our evaluation uses four randomly generated datasets with the following parameters: (1) $d = 64, N = 16,384$, (2) $d = 64, N = 32,768$, (3) $d = 128, N = 8,192$, and (4) $d = 128, N = 16,384$.

### *9.3. Custom Attention with Futhark*

Futhark's implementation in Fig. 16 closely resembles Custom Attention, Algorithm 6. During GPU compilation the incremental flattening transformation distributes the outer map processing each $d \times d$ block of $Q$ (line 12) across the computations of (i) the transposed matrix multiplication `mmmT` at line 9, (ii) the Online Softmax of Algorithm 8 at line 10, and (iii) the second matrix multiplication `mmm` at line 11. The resulting batch matrix multiplication kernels (i & iii) are optimized for spatial and temporal locality using block and register tiling. Online Softmax (ii) is similarly decomposed into an intragroup kernel that performs the computation of $m_i$ and $l_i$ in shared memory, and another kernel that updates the elements of $P$.

The CPU compilation diverges early from the GPU compilation and does not perform locality optimizations, resulting in very poor performance. Notably, the compilation of Custom Attention manifests the intermediate $P$ matrix in global memory and, hence, cannot match the performance of the baseline that implements the more efficient Flash Attention algorithm.

*9.4. Custom Attention with Accelerate*

Accelerate implements Custom Attention, Algorithm 6. Matrix multiplication is implemented naively in the benchmarks. A faster alternative would be to use the matrix multiplication available through Accelerate's BLAS binding.

*9.5. Custom Attention with Single Assignment C (SaC)*

As the code generated by SaC does not produce registered-tiled assembly, we achieve lower runtimes with the same asymptotic memory usage by implementing custom attention, i.e. Algorithm 6. This results in a factor of two slowdown compared to peak performance as we can only keep half of the execution units busy from L1 cache. Moreover, the generated code does not fully vectorize, accounting for the remaining slowdown.

The GPU backend is not able to generate CUDA kernels for all computations because reduction over non-scalars is not yet supported. This prevents SaC from computing Flash or Custom Attention on the GPU.

*9.6. Custom Attention with APL*

The APL code is a fairly direct translation of the Flash Attention algorithm, but cannot control memory allocation and so cannot exploit the inner loops working on shared memory objects. Hence, the implementation is closer to Algorithm 6. Input values are converted to 3-dimensional arrays to facilitate blocking before entering the main loop. While the other languages can express single precision floating point computation, existing APL implementations only support double precision floating points or greater.

*9.7. Custom and Flash Attention with DaCe*

For the CPU, we use the Python code shown in Fig. 17 with auto-optimization heuristics. DaCe generates OpenBLAS calls for the matrix multiplications in Lines 16 and 25. We explicitly set the number of OpenBLAS threads to one, resulting in an implementation very similar to the CPU baseline code. For the GPU, we use the simpler implementation shown in Algorithm 6.

*9.8. Benchmarking Results and Summary*

The Flash Attention performance results for the five languages on both multicore CPU and GPU are summarized in Table 8. We follow the baseline performance measurement and report compute rate (Gflop/s).

*GPU performance.* All languages implement the less efficient Custom Attention (Algorithm 6), and hence cannot match the baseline performance. Futhark and DaCe are the closest, reaching (60 %, 55 %, 71 %, and 83 %) and (70 %, 72 %, 86 %, and 86 %) of the baseline performance on the four datasets, respectively. APL and Accelerate lack locality optimizations, which restricts the percentage of baseline performance to (5 %, 5 %, 10 %, and 10 %) and (0.2 %, 0.2 %, 0.3 %, and 0.3 %), respectively. Compiler shortcomings prevent a SaC implementation (Section 9.5). Lacking support for 32-bit floats, the APL implementation

```python
@dace.program
def flash_attention_dace(Q: dace.float32[N, d], K: dace.float32[N, d],
                         V: dace.float32[N, d], O: dace.float32[N, d]):

    for ti in dace.map[0:N:Ti]:

        m = np.full([Ti], -np.inf, Q.dtype)
        l = np.zeros([Ti], Q.dtype)
        S = np.empty([Ti, Tj], Q.dtype)
        Oi = np.zeros([Ti, d], Q.dtype)

        for tj in range(0, N, Tj):

            S[:] = Q[ti:ti+Ti, :] @ K[tj:tj+Tj, :].T

            max_row = np.max(S, axis=1)
            m_new = np.maximum(m, max_row)
            p_tilde = np.exp(S - m_new[:, np.newaxis])
            sum_row = np.sum(p_tilde, axis=1)
            l_tmp = l * np.exp(m - m_new)
            l_new = l_tmp + sum_row
            Oi[:] = (Oi * l_tmp[:, np.newaxis] + p_tilde @ V[tj:tj+Tj, :])
                    / l_new[:, np.newaxis]
            m[:] = m_new
            l[:] = l_new

        O[ti:ti+Ti, :] = Oi
```

Figure 17: DaCe data-centric Python implementation of Flash Attention.

uses 64-bit doubles, whereas all other languages use 32-bit floats. This has significant performance impacts: at least halving the compute rate and doubling the memory requirements.

*Multicore performance.* DaCe outperforms the baseline with 102 %, 106 %, 117 %, and 116 %, followed by SaC achieving 25 %, 26 %, 27 %, and 25 %. Accelerate and Futhark suffer from a lack of locality optimizations, reaching only (7 %, 8 %, 13 %, and 10 %) and (4 %, 4 %, 3 %, and 3 %) of the baseline performance. The implicit multi-threading of the APL interpreter does not significantly reduce 32-core runtimes, so we report only single core results. As with the GPU implementations, APL's use of 64-bit, rather than 32-bit, floats significantly decreases performance.

*Expression.* While Custom Attention is elegantly implemented by all languages, with code that closely matches Algorithm 6, we cannot meaningfully compare most languages with the baseline because it implements the more efficient Flash Attention. The exception is the DaCe multicore implementation of Flash Attention (Fig. 17), which is very close to the algorithmic specification.

## 10. Benchmarking Summary and Analysis

Measuring the performance of the five languages on such a challenging set of benchmarks on both a multicore and a GPU exposes their design and implementation strengths and weaknesses. Broadly speaking, a language delivers good

Table 8: FlashAttention compute rate in Gflop/s (higher is better) for different $N$ and $d$.

| | $d = 64, N = 16384$ | | | $d = 64, N = 32768$ | | |
|---|---|---|---|---|---|---|
| | CPU | | GPU | CPU | | GPU |
| | 1C | 32C | | 1C | 32C | |
| Baseline* | 75.1 | 2274 | 5917 | 75.4 | 2159 | 6580 |
| Accelerate† | 21.6 | 150 | 11.5 | 19.5 | 182 | 9.6 |
| APL†§ | 15.1 | — | 293 | 14.5 | — | 333 |
| DaCe‡ | 79.6 | 2312 | 4147 | 80.1 | 2325 | 4741 |
| Futhark† | 3.1 | 83.6 | 3574 | 3.4 | 82.3 | 3606 |
| SaC† | 24.4 | 569 | — | 23.6 | 573 | — |

| | $d = 128, N = 16384$ | | | $d = 128, N = 8192$ | | |
|---|---|---|---|---|---|---|
| | CPU | | GPU | CPU | | GPU |
| | 1C | 32C | | 1C | 32C | |
| Baseline* | 68.0 | 2041 | 6116 | 73.5 | 2193 | 5437 |
| Accelerate† | 24.4 | 266 | 15.4 | 25.8 | 210 | 17.5 |
| APL†§ | 23.8 | — | 605 | 25.8 | — | 560 |
| DaCe‡ | 83.5 | 2395 | 5276 | 88.7 | 2536 | 4657 |
| Futhark† | 2.5 | 68.1 | 4315 | 2.7 | 74.2 | 4498 |
| SaC† | 23.3 | 546 | — | 25.9 | 542 | — |

(*) Flash Attention, (†) Custom Attention, (‡) DaCe implements Flash Attention on CPU and Custom Attention on GPU, (§) APL uses 64-bit floats.

performance on a target architecture if it has the right constructs for specifying the benchmark, paired with the right optimizations to generate efficient code for the target architecture. Conversely, performance is lacking if the language lacks the required constructs, optimizations, or a performant implementation for the target architecture.

The remainder of the section evaluates expressivity and programming effort, summarizes overall performance, before briefly summarizing and analyzing the performance of each language on the set of benchmarks and target architectures.

### *10.1. Expressivity*

We find that *functional array languages are expressive: they can represent a variety of array applications both concisely, and preserving a close relationship to the high-level specification.* Detailed evidence for this claim is provided in the summary of each benchmark, i.e., in Sections 6.8, 7.8, 8.8 and 9.8, and some specific examples follow. SaC's MG implementation closely follows the mathematical formulation presented in Section 7.1 and Algorithm 1. Notably, it defines one two-line generic stencil kernel (Fig. 12) that is instantiated in different contexts (i.e., to perform the computations denoted by $A$, $P$, $Q$, $S$). In contrast, the CUDA/Fortran baselines specialize each stencil kernel into different code

| | Accelerate | APL | DaCe (CPU) | DaCe (GPU) | Futhark | SaC | Baseline (CPU) | Baseline (GPU) |
|---|---|---|---|---|---|---|---|---|
| $N$-body | 66 | 25 | 76* | 53* | 46 | 61 | 164 | 411 |
| MG | 95 | 27 | 255 | | 151 | 136 | 893 | 4662 |
| Quickhull | 209 | 26 | — | — | 161 | 203 | 208 | — |
| FlashAttention | 29 | 21 | 31† | 26† | 90 | 79 | 1280 | 908 |
| Total Codebase | 399 | 99 | — | — | 448 | 479 | 2545 | 5981 |
| | | | | | | | 8526 | |

Table 9: Benchmark sizes in Source Lines of Code (SLOC). (*) DaCe uses separate $N$-body programs for CPU and GPU. (†) DaCe implements Flash Attention on CPU but Custom Attention on GPU.

ultimately resulting in a code base that our benchmark implementation teams found extremely hard to understand. Futhark builds on SaC's implementation and demonstrates that the baseline's optimizations can be expressed at a high level (i.e., applied to the generic stencil kernel), as evidenced by the competitive GPU performance. DaCe's base $N$-body implementation consists of about ten lines of Python code, similar to the five equations describing the problem. In contrast, the baseline consists of three code versions (one for CPU and two for GPUs) that are optimized in very different ways, e.g., they utilize different levels of parallelism and different array layouts (SoA *vs.* AoS). For Quickhull, Accelerate and Futhark present reasonably straightforward manually-flattened implementations that build on their rich set of second-order array operators and guarantee deterministic-by-construction parallelism. Ultimately, all arguments related to "elegant expression" are subjective, and we encourage readers to inspect our code repository and draw their own conclusions.

### *10.2. Programming Effort*

As an approximate measure of code size and development effort we use Source Lines Of Code (SLOC), a crude but widely accepted measure [14]. SLOC counts the lines of code omitting blank and comment lines. Table 9 compares the benchmark sizes. The precise numbers must be taken with a grain of salt, as they do not account for differences in personal style and tooling—for example, some languages use external tooling for timing and input processing, while others implement it for each benchmark. The major trends, however, are not obscured by such issues. A major difference is that the functional array programmer writes a single program that is compiled for multiple targets, where there are separate CPU and GPU baselines. Hence the bottom two rows of the table show that *at 8526 SLOC the total baseline codebase is much larger, at least 17× than any of the functional array codebases (479 SLOC).* The functional code requires fewer lines of code than the baselines in every case except for Accelerate

| | $N$-body | | MG | | Quickhull | | FlashAttention | |
|---|---|---|---|---|---|---|---|---|
| | CPU | GPU | CPU | GPU | CPU | GPU | CPU | GPU |
| Baseline | 0.15 | 1.1 | 1.1* | 3.0* | 3.3 | — | 0.54 | 0.44 |
| Accelerate | 0.45 | 0.83 | 5.6* | 3.3* | 1.3 | 2.7 | 0.44 | 0.91 |
| APL | 0 | 9.0 | 0 | 11.1 | 0 | 11.4 | 0 | 9.8 |
| DaCe | 7.8 | 3.5 | 22 | 79 | — | — | 7.9 | 14.7 |
| Futhark | 1.4 | 1.3 | 61 | 62 | 8.2 | 5.2 | 4.1 | 4.2 |
| SaC | 10 | 12 | 170* | 225* | 12 | — | 7.2 | 7.4 |

Table 10: Compilation times in seconds. In those cases where the implementation uses dataset-specific compilation, marked with an asterisk, we report only the slowest time.

Quickhull on CPU. *The functional codebases are at least* $5\times$ *smaller than the CPU baseline codebase (479 vs 2545 SLOC) and much smaller, at least* $12\times$*, than the GPU baseline codebase (479 vs 5981 SLOC).* Of the functional languages APL is the most concise (99 SLOC in total), while the other languages have similar code sizes.

Important development aspects not captured by source code measurements are tooling and debugging support, ease of deployment or use, access to libraries etc. While a detailed account of these aspects is outside the scope of this paper, we briefly compare them here. Compilation times for each language and benchmark are presented in Table 10. All the languages provide stack tracing and offline error reporting. APL provides by far the richest tooling, with a sophisticated online editing and debugging environment native to APL. Accelerate is embedded in Haskell, a mature host language, and inherits its associated development environments. DaCe comes with an environment for analyzing, editing, and optimizing the SDFG IR. This environment is implemented as a VSCode plugin, enabling integration with other supported development tools for debugging the generated code. Futhark and SaC are standalone compilers, and while they do not provide language-native development tooling, their compiled object code can integrate with other programming environments, such as C or Python.

### *10.3. Overall Performance Summary*

Regarding GPU and multicore performance, there are a total of 39 benchmark instances, and we provide a baseline for 36 of them (all except Quickhull on GPU). In 64 % of the instances (23 instances), at least one language matches or outperforms the baseline. In 14 % of the instances (5 instances), at least one language achieves more than 80 % of the baseline performance (but less than 100 %). In a further 17 % of the instances (6 instances), at least one language achieves more than 50 % of the baseline performance (but less than 80 %). In only 6 % of instances (2 instances), no language achieves more than 50 % of the baseline performance. These instances are Quickhull solving the Circle and Rectangle datasets on a 32-core CPU, and the reasons are elaborated in Sections 8.2 and 8.8. *The key conclusion from these results is that they demonstrate that mature functional array languages have the potential to deliver performance competitive with the best available conventional techniques.*

*10.4. Futhark Summary and Analysis*

Futhark's GPU performance outperforms the baselines for all benchmarks other than Attention. A key factor for achieving good performance is the compiler's ability to map levels of application's parallelism to different levels of the hardware hierarchy. This is achieved by *incremental flattening* ($\mathcal{IF}$)—the multi-version compilation technique described in Section 4.1. For $N$-body, $\mathcal{IF}$ generates two versions of the code, one that utilizes only the outermost level of parallelism for small numbers of bodies, and one that parallelizes both outer and inner levels of parallelism for large numbers of bodies. For MultiGrid and Custom Attention, $\mathcal{IF}$ generates code versions that map the innermost level of parallelism to the CUDA thread block level, enabling reuse from fast memory.[12]

Finally, Quickhull benefits from high-performance GPU mappings of scan [42], and reduce-by-index [33] constructs, and Custom Attention from tiling-related optimizations that allow array slices to be mapped and reused from fast (register+shared) memory [38]. All benchmarks benefit from fusion [73], which reduces the number of accesses to global memory.

Futhark's CPU backend uses a different compilation pipeline that has received less attention than the GPU. Notably, it lacks locality optimizations, resulting in abysmal performance for Custom Attention, and it employs a runtime system that aims to dynamically adjust the levels of parallelism that are mapped to the hardware, but which introduces significant overhead for $N$-body and MG.

In terms of programming effort, we consider the implementations of $N$-body and Custom Attention straightforward. Quickhull has been developed by a student as part of MSc-course work. MG is the only benchmark that requires high expertise to unlock the performance.

*10.5. Accelerate Summary and Analysis*

All the Accelerate benchmark programs are straightforward implementations of the algorithms, close to what a programmer would write in regular Haskell. The results reveal that Accelerate has a relatively high performance overhead, although for large computations, the overhead is amortized. The Accelerate compiler pipeline is currently being completely rewritten to overcome these problems, and allow for more aggressive fusion. At the time of the writing, only the multi-core CPU backend has been fully implemented in the new compiler architecture, therefore we used the old version for the benchmarks. The problems of poor scalability and high memory usage, as seen in the benchmarks, are addressed in the CPU backend of the new version [74], by using a new scheduling algorithm and reference counting, whereas the old version uses Haskell's tracing garbage collector.

[12]Such code versions require multi-versioned compilation as a safety net, because they may run out of resources (e.g., fast memory) for datasets with large innermost parallel dimensions.

### *10.6. SaC Summary and Analysis*

One of the main design goals of SaC is to provide programmer productivity for computational scientists. This can be observed in the applications studied here. In particular for Multigrid and Flash Attention the source code of SaC is much closer to the mathematical specification than the baseline. Despite this clarity in specification SaC's CPU performance is competitive with the baseline in $N$-body and gets within factors of 1.2 – 3.9 for the other benchmarks.

An outstanding feature of SaC is that this level of performance is obtained despite full support for recursion and rank polymorphism. This is made possible by aggressive code specialization and optimization. None of the examples makes use of hand-tuned libraries such as BLAS although the FFI of SaC would enable this. The key optimizations are *with-loop folding* and *with-loop scalarization*, which improve temporal locality by composing data parallel operations into element-wise operations. Many more optimization tackle overheads that result from the generic nature of the language design. Prominent examples here are *index vector elimination* and *array elimination* which try to avoid the creation of small arrays at runtime.

The SaC GPU backend is not as well developed and the benchmark performance does not come close to the baselines. The code generation for GPUs is currently being re-developed to address the observed limitations.

### *10.7. APL Summary and Analysis*

APL is by far the most concise language, and the computational core of each benchmark is implemented using between 10–14 lines of code. The amount of native data type, functional, and structural flexibility in the APL language exceeds that of the other languages in this comparison. It also has the richest native set of primitive operations over its core data structure, the array.

However, both the Dyalog CPU interpreter and Co-dfns GPU compiler lack the full set of optimizations to extract the maximum performance from these benchmarks. In both implementations, the primary factor limiting performance is the inability to eliminate memory access costs associated with intermediate and temporary variables. For instance, the current runtime requires the use of double-precision floating points, which harms performance on the Flash Attention benchmark. Likewise, because the GPU runtime depends on runtime level JIT compilation provided by external libraries [50], there is some variability and lack of predictability in the level of fusion and the kind of laziness that will manifest. This causes some benchmarks to perform worse than strictly necessary for the given APL code. For the Quickhull benchmark, runtime JIT does not optimize the implementation of key-based reduction, which must be implemented using other primitives, since APL does not presently support key-wise reduction. This greatly increases both the number of kernels and the main memory throughput required. APL's dynamic typing makes type inference more difficult, limiting the ability to statically specialize the output to known data sizes. This requires runtime verification of some data types, array shapes, and data ranges to implement the primitives, further increasing overheads.

While it is possible to manually tile or tweak the APL code to encourage the runtime JIT to produce more optimal code, doing so is unidiomatic. The benchmarks are expressed as idiomatic APL without such manual optimization.

The Co-dfns GPU compiler is relatively new, and is the least optimizing compiler among those evaluated here. There is very little optimization of locality between kernels or between various computations. So when a computation maps to a specific library function it is performant, but combining unfused primitives typically requires writing to GPU main memory, limiting performance. Extending Co-dfns with additional optimizations would permit more efficient kernel generation and resolve many of these issues.

While the lack of optimizations limits APL performance, the Co-dfns compiler consistently delivers more than a $10\times$ performance improvement over the Dyalog CPU interpreter. This shows that APL is eminently suitable for GPU execution, and the ease with which it can be translated for the GPU, even without optimizations and specialized compilation pipelines. This is rare, since most languages rely on specific compiler optimizations to achieve good GPU performance.

### *10.8. DaCe Summary and Analysis*

DaCe's greatest asset is undoubtedly its extensive optimization arsenal. The automatic tools frequently lead to acceptable performance, while experimenting with different parallel schedules and data layouts is often relatively easy. At the same time, experienced users can further improve performance by manually applying transformation workflows, by editing the graph representation with visual tools, or even by embedding scheduling decisions in the high-level code. These capabilities can deliver high performance, as shown in $N$-body, where DaCe matches the baseline on CPU and surpasses it on GPU by 41 % and 17 % on the medium and large datasets, respectively.

However, taking advantage of such tools requires some level of expertise and time investment. For example, transformations may have to be applied in a specific order to achieve the desired result, or others may fail apply to a particular subgraph, not due to semantic violations, but rather due to issues related to the graph's structure and pattern matching. Such cases may lead to optimization "dead ends" that can be overcome by extending the framework's tools, which is one of DaCe's main use cases for the performance engineer. At the same time, the time cost may not be easy to amortize for the naive user. We encountered such issues while optimizing $N$-body, especially for the GPU, where we found it much simpler to rewrite the initial (Python) program using DaCe-provided features. Doing so, however, introduced multiple source codes.

Another of DaCe's strengths is its Library Node system, which simplifies integration with high performance libraries like BLAS or LAPACK. This leads to high performance in benchmarks such as Flash Attention, where DaCe achieves 102 % to 123 % of the baseline's performance on CPU. However, DaCe does not always provide the key primitives for all algorithms and target hardware. For example, to achieve high performance in Flash Attention on a GPU, a thread-block-level matrix multiplication primitive is required, but DaCe provides only

a device-level one. DaCe can represent the algorithm, but performance is reduced as it lacks some key optimizations: multi-level tiling, local storage, etc. Hence a naive programmer may find it simpler to implement Custom Attention, which is amenable to device-level matrix-matrix products but sacrifices overall performance.

Another significant drawback for some algorithms is the lack of support for recursion. For example, the lack of recursion constrains DaCe to an imperative implementation of MG, and makes the generation of a graph representation a difficult task in Quickhull.

## 11. Related Work

There are a multitude of parallel programming paradigms and a range of target architectures. We do not replicate the detailed discussions [10, 21, 29, 12, 13] of how each of the functional array languages covered here is situated in the parallelism space. Rather, we focus on describing the subspace in the parallel paradigm design space occupied by functional array languages.

*Low-level* vs *High-level, Task* vs *Data Parallelism.* Some parallel paradigms are *low-level* and others are *high-level*. In low-level paradigms, the parallel coordination is specified in great detail, giving the programmer a high degree of control, but also a significant coding challenge. For example, many MPI [75] programs use individual message `send`s that must be exactly matched with message `receive`s. In contrast, high-level paradigms have powerful coordination constructs, and this is the approach adopted by functional array languages. The coordination constructs are typically a set of parallel combinators, that is, polymorphic higher-order functions with implicit parallel semantics. A simple example is a parallel `map` that applies a function to every element of an array in parallel.

Some parallel paradigms use *task* parallelism, where tasks are spawned and collaborate to achieve the required computation. Examples include MPI and some parallel functional languages like Glasgow Parallel Haskell [76]. Other paradigms use *data* parallelism, where the parallelism is determined by the data structures being manipulated. Functional array languages take the latter approach. More precisely, they provide *bulk* data parallelism where the combinators operate over an entire multidimensional array.

Some parallel paradigms are implemented as libraries, like NumPy [77], or MPI [75], whereas others are provided as programming languages, or language extensions, e.g., Accelerate extends Haskell. Functional array languages are typically languages or DSLs, and rely on sophisticated compilers to generate high-performing code. The following discussion focuses mainly on parallel languages.

*Parallel Architectures.* Some parallel paradigms target shared-memory architectures, some distributed architectures, and others both. Parallel languages that can target distributed memory architectures, like Chapel [78], Charm++ [79], HPF [80], and X10 [81] scale across multiple hosts and may execute on a cluster

or in a cloud. In contrast, shared-memory paradigms like OpenMP [82] utilize the resources of a single host, possibly in conjunction with accelerators. Most of the languages discussed here are primarily shared-memory parallel languages, although APL, DaCe, and SaC have the capability to target clusters (Table 3).

Parallel architectures are increasingly heterogeneous, providing accelerators alongside multicore systems. Common accelerators are GPUs, FPGAs, or AI accelerators. All of the functional array languages discussed here target GPUs in addition to multicores, and DaCe can target FPGAs (Table 3). Some widely-used paradigms targeting accelerators are low-level, like CUDA [83] for GPUs, and OpenCL [3] for FPGAs and GPUs, while others offer replacements of mainstream libraries, e.g., cuNumeric [84] and CuPy [85] for NumPy. A range of other languages are similar to functional array languages in aiming to provide high-level accelerator programming, e.g., Chapel [78] and X10 [81].

*Imperative Approaches.* A large body of work is dedicated to libraries, runtime systems, and compilation techniques aimed at supporting heterogeneous/-portable (HPC) programming within mainstream imperative languages. Examples of library extensions of C++ include Kokkos [86] and RAJA [87], and runtime systems include HPVM [88] and Legion [89]. Similarly, OpenMP [2], OpenACC [1, 90] and SYCL/oneAPI [91, 92, 93] are industry-backed programming standards specifically designed to simplify heterogeneous programming across CPUs, GPUs, and other accelerators. Their performance portability has been compared and reported in several studies [94, 95]. These frameworks are lower level than the studied languages in that the programmer is entirely responsible (1) for the correctness of the parallelization strategy, (2) for managing and reusing memory, and (3) for mapping the arbitrary levels of application parallelism to the fixed ones of the hardware, e.g., using loop distribution and interchange to create perfect parallel nests that can be flattened and mapped to GPU.

Polyhedral analyses [96], integrated into optimizing compilers such as Pluto [97], PPCG [98], Polly [99], Tiramisu [100] have demonstrated that at least *affine* programs can be efficiently mapped to multicore and GPU hardware. The optimization spaces for polyhedral analysis and the rewriting in functional array languages both overlap and are complementarity. For example, map fusion/fission are analogous to loop fusion/distribution. On the other hand, low-level transformations requiring dependence-analysis on arrays, such as loop skewing, are not expressible as functional rewrites. Conversely, polyhedral analysis is limited to affine code[13]—consisting of loop nests in which all control structures and array indices are affine expressions in terms of the surrounding-loop indices—hence dynamic control flow and scatter/gather operations cannot be analyzed. Furthermore, polyhedral analysis lacks support for second-order parallel primitives such as (segmented) scan[14] (prefix

[13] A significant body of work was aimed at relaxing the affine constraints, by combining static with more dynamic instances of dependence analyses on arrays [101, 102, 103, 104, 105, 106].

[14] A sequential implementation of scan is affine code, but the transformation to a parallel scan requires an algorithmic change that is not expressible in terms of polyhedral transformations.

sum). Hence computations that are dominated by scans, like the Quickhull benchmark, fall outside the polyhedral scope.

*DSLs and Scheduling Languages.* Conventional compilers automatically generate low-level code using a set of optimization heuristics. For performance-critical applications, the heuristics often fall short of extracting optimal performance because (i) they cover a small part of a huge optimization space, and (ii) they typically generate one optimized code version, that may not be ideal for all datasets of interest. This has motivated the emergence of scheduling languages (surveyed in [107]) that specify optimization recipes as a combination of code transformations both as a way of allowing the user to take an active hand in the optimization process, and/or to enable systematic exploration of the optimization space. Early approaches, such as the X language [108], CHiLL [109], URUK [110], expose polyhedral transformation that are applied to scientific code written in mainstream languages such as C.

More recent scheduling languages are driven by the observation that specialization of the language and compiler repertoire paves the way to performance. They commonly employ a simple (often functional) specification language, that is accessible to domain experts, and a separate optimization recipe written by compiler experts. Classic examples include (i) image-processing DSLs like Halide [5], PolyMage [111], Darkroom [112], (ii) tensor algebra DSLs for dense computations, such as Baracuda [113], DISTAL [6], and for sparse computations, such as CHiLL-I/E [114, 115], TACO [116], Mosaic [117], and (iii) machine learning DSLs such as Triton [118], TVM [119] and SWIRL [120]. The functional languages studied in this paper are more general-purpose than these DSLs as they aim to parallelize any array-based computation. Among them, only DaCe has support for user-defined schedules, while Futhark uses a specialized compiler exploration strategy that generates multiple semantically equivalent code versions that are discriminated by autotuning, as described in Section 4.1.

*Other Functional Array Languages.* Dex [121] is similar to the languages we study, and resembles Futhark in expressivity. Dex does not support the parallel scans needed to implement a data parallel Quickhull, but would otherwise be able to express all of our benchmarks. The closely related Jax [122] is an embedded language widely used for machine learning, and generally offers very good performance by compiling via the XLA infrastructure. Jax also does not support parallel scans, and has limited support for nested parallelism.

Julia [123] is a popular array language for scientific and parallel computing, and while it is fundamentally an imperative language, the functional style is encouraged and well supported. Julia has a very diverse ecosystem for parallel programming, well beyond easy characterization. GPU programming is mainly through the CUDA.jl library [124], which supports both high and low level programming. CUDA.jl also supports higher-order programming and, in particular, provides a parallel scan, suggesting that it can express all of our benchmarks.

Other examples are the LIFT [125], RISE [126] and MDH [127] languages that are similar to, albeit more restricted than the languages discussed in this

paper, but may support user-guided exploration of the optimization space [128]. Unlike the languages in this paper, which all restrict higher-order functions, Erik Holk's Harlan language [129] supports first class procedures natively.

## 12. Discussion and Conclusion

Functional array languages offer the enticing prospect of the high-level specification of correct-by-construction parallel array programs that generate performant and portable code. Hence, they are attracting research interest, and they are emerging as a class of high-performance parallel languages. This paper compares the design, the implementation, and the achieved performance of five functional array languages, namely Futhark, Accelerate, SaC, DaCe, and APL.

Selected from the range of functional array languages outlined in the previous section, the languages selected for this study represent a diverse sample of approaches to both language design and implementation. The comparisons provide a broad view of the character of functional array programming and of the approaches to solving the challenges that arise, and specifically the trade-offs that come with making different language design and implementation choices.

We illustrate the programming paradigms of the five functional array languages using a simple $N$-body simulation benchmark (Section 2). The five $N$-body implementations are both more concise than the CUDA and OpenMP baseline implementations and, we argue, far closer to the mathematical specification of the problem.

We make a systematic comparison of the designs of the five functional array languages (Section 3). All the languages support rectilinear arrays, several support rank polymorphism, and user-defined element types, but only APL supports jagged arrays and heterogeneous arrays (Table 2). All languages except APL use static types, most support parametric polymorphism, with Futhark and SaC also providing size-dependent types (Table 1). Although most languages offer at least partial determinism, the languages otherwise offer a wide variety of parallel paradigms: parallelism may be implicit or explicit, may be nested or not, and task parallelism may be available alongside the data parallelism or not(Table 3).

We outline the implementation of the five functional array languages and make a systematic comparison of their feature sets (Section 4). There are a variety of implementation techniques: the Futhark and SaC ahead-of-time compilers, the Accelerate embedded language, the DaCe framework, and the APL interpreter and compiler. All implementations target multicore CPUs and GPUs, and several languages target further platforms. Although all implementations fuse computations where appropriate, implementations otherwise support different sets of optimizations (Table 4).

We demonstrate the expressiveness of functional array programming using four challenging benchmarks that represent a range of application domains, parallel computational models, and require different optimizations (Sections 6 to 9). The benchmark code is iteratively developed by applying a sequence of optimizations to an initial correct implementation.

We compare the code base sizes, and hence approximate the programming effort, of the benchmarks in the functional array languages and in the OpenMP and CUDA baseline implementations. We use Source Lines Of Code (SLOC) as a crude but widely accepted measure [14] (Table 9). A major difference is that the functional array programmer writes a single program that is compiled for multiple targets, whereas baseline implementations are specific to CPU and GPU architectures. Hence, *the total baseline code base (8526 SLOC) is much larger (at least 17×) than any of the functional array language code bases (479 SLOC).* The functional code requires fewer lines of code than the baselines in all but one case. *The functional code bases are at least 5× smaller than the CPU baseline code base and much smaller (at least 12×) than the GPU baseline code base.* Of the functional languages APL is the most concise while the other languages have similar code sizes.

One reason for their conciseness is that the functional array code omits architectural aspects like the memory hierarchy and how the algorithm is mapped to hardware. This enables the development of code without intimate knowledge of the parallel hardware. It also allows the implementation to generate both multicore and GPU codes from a single source. Thus, *functional array code should be more easily ported to, and optimized for, new parallel architectures.* The challenge of porting functional array programs to a new architecture reduces to writing a performant implementation, typically a compiler backend, for the new platform. Table 4 summarizes the range of platforms targeted by the five languages: multicores, GPUs, FPGAs, and clusters. Several of the languages have previously demonstrated performant execution of the same code across different platforms. For example [15] compares the performance of APL CNN implementations on an Intel 16-core and an NVIDIA GeForce RTX 3070. Likewise [11] reports DaCe performance results for a variety of benchmarks on an Intel 16-core, an NVIDIA V100, Intel and Xilinx FPGAs, and a 120-node cluster. The multi-language comparison reported in Sections 6–9 showcases portability and, in many cases, very good performance. However, it provides only partial evidence for performance portability, typically defined as the consistency of the achieved performance across different hardware targets relative to a theoretical or best-observed performance [130]. Only a few languages, notably DaCe and Futhark, consistently achieve good performance on both multicore and GPU for these challenging benchmarks (with the former, in some cases, using separate CPU/GPU source code to achieve the best results). However, they still come short of matching baseline's performance simultaneously on both platforms, see CPU Quickhull, the first two datasets of MG and the first dataset of N-Body.

Reasons for this include (1) language limitations, e.g., Futhark and DaCe do not support recursion; APL, SaC, and DaCe do not support prefix sum as a primitive, (2) unsupported code transformations, see Table 4, and (3) implementation maturity, e.g., the GPU backend issues of SaC and Accelerate's new compiler version not yet being ready for use in our benchmarks.

We summarize and analyze the multicore CPU and GPU performance of the five languages under consideration on 39 instances of the four selected bench-

marks[15] to explore why each language performs well, or poorly, on each benchmark and architecture (Section 10). We find that in 64 % of the instances, at least one language matches or outperforms the baseline; that in 78 % of the instances at least one language achieves more than 80 % of the baseline performance. In only 6 % of instances, no language achieves more than 50 % of the baseline performance. In some isolated instances a language fails comprehensively, for example SaC fails to generate MultiGrid code for GPU execution. These results are impressive considering how new the language implementations are, and *we argue that the results demonstrate that mature functional array languages have the potential to deliver performance competitive with the best available conventional techniques.* Measuring on a multicore system and on a GPU, this study uses shared-memory platforms. Future work could compare the performance of functional array languages on distributed memory platforms like clusters. We look forward to seeing the developments in the area over the coming years.

*CRediT authorship contribution statement*

**David van Balen**: Software (Accelerate), Writing - Review & Editing.

**Tiziano De Matteis**: Software (DaCe), Writing - Review & Editing

**Clemens Grelck**: Conceptualization, Software (SaC), Investigation, Writing - Review & Editing

**Troels Henriksen**: Conceptualization, Software (Futhark), Investigation, Writing - Review & Editing

**Aaron W. Hsu**: Conceptualization, Software (APL), Investigation, Writing - Review & Editing

**Gabriele K. Keller**: Conceptualization, Software (Accelerate), Investigation, Writing - Review & Editing

**Thomas Koopman**: Software (SaC), Investigation, Writing - Review & Editing.

**Trevor McDonell**: Software (Accelerate).

**Cosmin Oancea**: Conceptualization, Software (Futhark), Investigation, Writing - Review & Editing

**Sven-Bodo Scholz**: Conceptualization, Software (SaC), Investigation, Writing - Review & Editing.

**Artjoms Sinkarovs**: Software (SaC).

**Tom Smeding**: Software (Accelerate), Writing - Review & Editing.

**Phil Trinder**: Conceptualization, Investigation, Writing - Review & Editing

**Ivo Gabe de Wolff**: Software (Accelerate), Writing - Review & Editing.

**Alexandros Nikolaos Ziogas**: Software (DaCe), Investigation, Writing - Review & Editing.

[15] https://github.com/diku-dk/CFAL-bench/

*Software Teams*

**Accelerate:** David van Balen, Gabriele K. Keller, Trevor McDonell, Tom Smeding and Ivo Gabe de Wolff.

**APL:** Aaron W. Hsu.

**DaCe:** Alexandros Nikolaos Ziogas and Tiziano De Matteis.

**Futhark:** Troels Henriksen and Cosmin Oancea. The Quickhull implementations was developed by Mikkel K. Mathiesen, as part of his previous MSc-course work.

**SAC:** Thomas Koopman, Sven-Bodo Scholz, Clemens Grelck and Artjoms Sinkarovs.

**Baseline Development/Improvements:** N-body CPU: Troels Henriksen and Thomas Koopman; N-body GPU: Cosmin Oancea; Flash Attention CPU: Aaron W. Hsu, Thomas Koopman and Alexandros N. Ziogas; Flash Attention GPU: Alexandros N. Ziogas and Cosmin Oancea.

*Funding Sources*

This work was funded in part by the following research grants. Novo Nordisk Foundation (grant reference number NNF24OC0090447). Swiss National Science Foundation (SNSF), grant agreement "Quantum Transport Simulations at the Exascale and Beyond (QuaTrEx)" no. 209358. UK EPSRC STARDUST grant number EP/T014628. BMK, BMDW, and the State of Upper Austria in the frame of the SCCH competence center INTEGRATE (FFG grant no. 892418) part of the FFG COMET Competence Centers for Excellent Technologies Programme.

## References

[1] OpenACC-Standard.org, The OpenACC Application Programming Interface Version 3.3 (November 2022).
URL https://www.openacc.org/sites/default/files/inline-images/Specification/OpenACC-3.3-final.pdf

[2] L. Dagum, R. Menon, OpenMP: an industry standard api for shared-memory programming, Computational Science & Engineering, IEEE 5 (1) (1998) 46–55.

[3] A. Munshi, The OpenCL specification, in: 2009 IEEE Hot Chips 21 Symposium (HCS), 2009, pp. 1–314. doi:10.1109/HOTCHIPS.2009.7478342.

[4] J. Nickolls, I. Buck, M. Garland, K. Skadron, Scalable parallel programming with cuda, in: ACM SIGGRAPH 2008 Classes, SIGGRAPH '08, Association for Computing Machinery, New York, NY, USA, 2008. doi:10.1145/1401132.1401152.
URL https://doi.org/10.1145/1401132.1401152

[5] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, S. Amarasinghe, Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines, SIGPLAN Not. 48 (6) (2013) 519–530. doi:10.1145/2499370.2462176.

[6] R. Yadav, A. Aiken, F. Kjolstad, DISTAL: the distributed tensor algebra compiler, in: Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2022, Association for Computing Machinery, New York, NY, USA, 2022, p. 286–300. doi:10.1145/3519939.3523437.

[7] R. Yadav, A. Aiken, F. Kjolstad, Spdistal: compiling distributed sparse tensor computations, in: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis, SC '22, IEEE Press, 2022.

[8] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, X. Zheng, TensorFlow: Large-scale machine learning on heterogeneous systems, software available from tensorflow.org (2015).
URL https://www.tensorflow.org/

[9] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: an imperative style, high-performance deep learning library, Curran Associates Inc., Red Hook, NY, USA, 2019.

[10] T. L. McDonell, M. M. Chakravarty, G. Keller, B. Lippmeier, Optimising purely functional GPU programs, in: Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming, ICFP '13, Association for Computing Machinery, New York, NY, USA, 2013, p. 49–60. doi:10.1145/2500365.2500595.

[11] A. N. Ziogas, T. Schneider, T. Ben-Nun, A. Calotoiu, T. De Matteis, J. de Fine Licht, L. Lavarini, T. Hoefler, Productivity, portability, perfor-

mance: data-centric Python, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 1–13. doi:10.1145/3458817.3476176.

[12] T. Henriksen, N. G. W. Serup, M. Elsman, F. Henglein, C. E. Oancea, Futhark: Purely Functional GPU-programming with Nested Parallelism and In-place Array Updates, in: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, ACM, New York, NY, USA, 2017, pp. 556–571. doi:10.1145/3062341.3062354.

[13] C. Grelck, S. Scholz, SAC - A functional array language for efficient multithreaded execution, Int. J. Parallel Program. 34 (4) (2006) 383–427. doi:10.1007/S10766-006-0018-X.

[14] J. Rosenberg, Some misconceptions about lines of code, in: Proceedings fourth international software metrics symposium, IEEE, IEEE, Albuquerque, NM, USA, 1997, pp. 137–142. doi:10.1109/METRIC.1997.637174.

[15] A. W. Hsu, R. G. Serrão, U-net CNN in APL: exploring zero-framework, zero-library machine learning, in: Proc. ARRAY'23, Orlando, FL, USA, 18 June 2023, ACM, 2023, pp. 22–35. doi:10.1145/3589246.3595371.

[16] K. E. Iverson, A programming language, in: Proceedings of the May 1-3, 1962, spring joint computer conference, 1962, pp. 345–351.

[17] C. Moler, J. Little, A history of matlab, Proc. ACM Program. Lang. 4 (HOPL) (Jun. 2020). doi:10.1145/3386331.
URL https://doi.org/10.1145/3386331

[18] J. Hughes, Why functional programming matters, The computer journal 32 (2) (1989) 98–107.

[19] R. Milner, The definition of standard ML: revised, MIT press, 1997.

[20] S. Marlow, et al., Haskell 2010 language report, Tech. rep., The Haskell Team (Jul. 2010).
URL https://www.haskell.org/onlinereport/haskell2010/

[21] R. K. W. Hui, M. J. Kromberg, APL since 1978, Proc. ACM Program. Lang. 4 (HOPL) (Jun. 2020). doi:10.1145/3386319.

[22] A. K. Hovgaard, T. Henriksen, M. Elsman, High-performance defunctionalisation in futhark, in: M. Pałka, M. Myreen (Eds.), Trends in Functional Programming, Springer International Publishing, Cham, 2019, pp. 136–156. doi:10.1007/978-3-030-18506-0_7.

[23] G. E. Blelloch, Programming parallel algorithms, Commun. ACM 39 (3) (1996) 85–97. doi:10.1145/227234.227246.

[24] T. L. McDonell, J. D. Meredith, G. Keller, Embedded pattern matching, in: Proceedings of the 15th ACM SIGPLAN International Haskell Symposium, Haskell 2022, Association for Computing Machinery, New York, NY, USA, 2022, p. 123–136. doi:10.1145/3546189.3549917.

[25] C. Grelck, S.-B. Scholz, Classes and objects as basis for i/o in sac, in: T. Johnsson (Ed.), 7th International Workshop on Implementation of Functional Languages (IFL'95), Båstad, Sweden, Chalmers University of Technology, Gothenburg, Sweden, 1995, pp. 30–44.
URL sac-classes-objects-bastad-95.pdf

[26] R. Huijben, J. Aaldering, P. Achten, S.-B. Scholz, Flattening combinations of arrays and records, in: J. Hemann, S. Chang (Eds.), Trends in Functional Programming, Springer Nature Switzerland, Cham, 2025, pp. 220–240. doi:10.1007/978-3-031-74558-4_10.

[27] J. Aaldering, S.-B. Scholz, B. V. Gastel, Type patterns: Pattern matching on shape-carrying array types, in: Proceedings of the 35th Symposium on Implementation and Application of Functional Languages, IFL '23, Association for Computing Machinery, New York, NY, USA, 2024. doi:10.1145/3652561.3652572.
URL https://doi.org/10.1145/3652561.3652572

[28] A. Calotoiu, T. Ben-Nun, G. Kwasniewski, J. de Fine Licht, T. Schneider, P. Schaad, T. Hoefler, Lifting C semantics for dataflow optimization, in: Proceedings of the 36th ACM International Conference on Supercomputing, ICS '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 1–13. doi:10.1145/3524059.3532389.

[29] T. Ben-Nun, J. de Fine Licht, A. N. Ziogas, T. Schneider, T. Hoefler, Stateful dataflow multigraphs: a data-centric model for performance portability on heterogeneous architectures, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 1–14. doi:10.1145/3295500.3356173.

[30] Scalable Parallel Computing Laboratory, ETH Zurich, DaCe: Data-Centric Parallel Programming (2024).
URL https://spcldace.readthedocs.io

[31] H. Xi, F. Pfenning, Dependent types in practical programming, in: Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages, 1999, pp. 214–227.

[32] L. Gonnord, L. Henrio, L. Morel, G. Radanne, A Survey on Parallelism and Determinism, ACM Comput. Surv. 55 (10) (Feb. 2023). doi:10.1145/3564529.

[33] T. Henriksen, S. Hellfritzsch, P. Sadayappan, C. Oancea, Compiling generalized histograms for gpu, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '20, IEEE Press, 2020, pp. 1–14. doi:10.1109/SC41405.2020.00101.

[34] P. Wadler, Deforestation: transforming programs to eliminate trees, Theoretical Computer Science 73 (2) (1990) 231–248. doi:10.1016/0304-3975(90)90147-A.

[35] G. E. Blelloch, Vector models for data-parallel computing, Vol. 75, MIT press Cambridge, 1990.

[36] T. Henriksen, F. Thorøe, M. Elsman, C. Oancea, Incremental Flattening for Nested Data Parallelism, in: Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming, PPoPP '19, ACM, New York, NY, USA, 2019, pp. 53–67. doi:10.1145/3293883.3295707.

[37] P. Munksgaard, S. L. Breddam, T. Henriksen, F. C. Gieseke, C. Oancea, Dataset sensitive autotuning of multi-versioned code based on monotonic properties, in: V. Zsók, J. Hughes (Eds.), Trends in Functional Programming, Springer International Publishing, Cham, 2021, pp. 3–23. doi:10.1007/978-3-030-83978-9_1.

[38] R. Schenck, O. Rønning, T. Henriksen, C. E. Oancea, Ad for an array language with nested parallelism, in: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis, SC '22, IEEE Press, 2022, pp. 1–15. doi:10.1109/SC41404.2022.00063.

[39] P. Munksgaard, T. Henriksen, P. Sadayappan, C. Oancea, Memory optimizations in an array language, in: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis, SC '22, IEEE Press, 2022, pp. 1–15. doi:10.1109/SC41404.2022.00036.

[40] A. Nicolaisen, M. A. Persson, Implementing single-pass scan in the futhark compiler, Master's thesis, Department of Computer Science, Faculty of Science, University of Copenhagen, https://futhark-lang.org/student-projects/marco-andreas-scan.pdf (2020).
URL https://futhark-lang.org/student-projects/marco-andreas-scan.pdf

[41] M. T. Clausen, Regular segmented single-pass scan in futhark, Master's thesis, Department of Computer Science, Faculty of Science, University of Copenhagen, https://futhark-lang.org/student-projects/morten-msc-thesis.pdf (2021).
URL https://futhark-lang.org/student-projects/morten-msc-thesis.pdf

[42] D. Merrill, M. Garland, Single-pass parallel prefix scan with decoupled lookback, Nvidia technical report nvr-2016-002, march 2016, NVIDIA (2016).
URL https://research.nvidia.com/sites/default/files/pubs/2016-03_Single-pass-Parallel-Prefix/nvr-2016-002.pdf

[43] T. Schrijvers, S. Peyton Jones, M. Chakravarty, M. Sulzmann, Type checking with open type functions, in: Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming, ICFP '08, Association for Computing Machinery, New York, NY, USA, 2008, p. 51–62. doi:10.1145/1411204.1411215.

[44] M. M. T. Chakravarty, G. Keller, S. P. Jones, Associated type synonyms, in: Proceedings of the Tenth ACM SIGPLAN International Conference on Functional Programming, ICFP '05, Association for Computing Machinery, New York, NY, USA, 2005, p. 241–253. doi:10.1145/1086365.1086397.

[45] C. Grelck, S.-B. Scholz, Merging compositions of array skeletons in SAC, Journal of Parallel Computing 32 (7+8) (2006) 507–522. doi:10.1016/j.parco.2006.08.003.

[46] C. Grelck, K. Trojahner, Implicit Memory Management for SaC, in: C. Grelck, F. Huch (Eds.), Implementation and Application of Functional Languages, 16th International Workshop, IFL'04, University of Kiel, Institute of Computer Science and Applied Mathematics, 2004, pp. 335–348, technical Report 0408.

[47] C. Grelck, Single Assignment C (SAC): the compilation technology perspective, in: V. Zsók, Z. Porkoláb, Z. Horváth (Eds.), 6th Central European Functional Programming Summer School (CEFP'15), Budapest, Hungary, Vol. 10094 of Lecture Notes in Computer Science, Springer, 2019, pp. 207–282. doi:10.1007/978-3-030-28346-9_7.

[48] A. W.-y. Hsu, A data parallel compiler hosted on the GPU, Ph.D. thesis, Indiana University (2019).
URL https://hdl.handle.net/2022/24749

[49] A. W. Keep, A nanopass framework for commercial compiler development, Ph.D. thesis, Indiana University (2012).
URL https://andykeep.com/pubs/dissertation.pdf

[50] J. Malcolm, P. Yalamanchili, C. McClanahan, V. Venugopalakrishnan, K. Patel, J. Melonakos, ArrayFire: a GPU acceleration platform, in: E. J. Kelmelis (Ed.), Modeling and Simulation for Defense Systems and Applications VII, Vol. 8403, International Society for Optics and Photonics, SPIE, 2012, p. 84030A. doi:10.1117/12.921122.

[51] A. N. Ziogas, T. Ben-Nun, G. I. Fernández, T. Schneider, M. Luisier, T. Hoefler, A Data-Centric Approach to Extreme-Scale Ab initio Dissipative Quantum Transport Simulations, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC19), 2019, pp. 1–13. doi:10.1145/3295500.3357156.

[52] T. Ben-Nun, L. Groner, F. Deconinck, T. Wicky, E. Davis, J. Dahm, O. Elbert, R. George, J. McGibbon, L. Trümper, E. Wu, O. Fuhrer, T. Schulthess, T. Hoefler, Productive Performance Engineering for Weather and Climate Modeling with Python, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'22), 2022, pp. 1–14. doi:10.1109/SC41404.2022.00078.

[53] P. Schaad, T. Ben-Nun, T. Hoefler, Boosting Performance Optimization with Interactive Data Movement Visualization, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'22), 2022, pp. 1–16. doi:10.1109/SC41404.2022.00069.

[54] A. N. Ziogas, G. Kwasniewski, T. Ben-Nun, T. Schneider, T. Hoefler, Deinsum: Practically I/O Optimal Multilinear Algebra, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'22), 2022, pp. 1–15. doi:10.5555/3571885.3571918.

[55] L. Trümper, T. Ben-Nun, P. Schaad, A. Calotoiu, T. Hoefler, Performance Embeddings: A Similarity-Based Transfer Tuning Approach to Performance Optimization, in: Proceedings of the 37th International Conference on Supercomputing, ICS '23, Association for Computing Machinery, New York, NY, USA, 2023, p. 50–62. doi:10.1145/3577193.3593714.

[56] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, H. D. Simon, V. Venkatakrishnan, S. K. Weeratunga, The NAS parallel benchmarks—summary and preliminary results, in: Proceedings of the 1991 ACM/IEEE Conference on Supercomputing, Supercomputing '91, Association for Computing Machinery, New York, NY, USA, 1991, p. 158–165. doi:10.1145/125826.125925.

[57] G. Araujo, D. Griebler, D. A. Rockenbach, M. Danelutto, L. G. Fernandes, NAS Parallel Benchmarks with CUDA and beyond, Software: Practice and Experience 53 (1) (2023) 53–80. doi:10.1002/spe.3056.

[58] D. Anderson, G. E. Blelloch, L. Dhulipala, M. Dobson, Y. Sun, The problem-based benchmark suite (pbbs), v2, in: Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 445–447. doi:10.1145/3503221.3508422.

[59] T. Dao, D. Fu, S. Ermon, A. Rudra, C. Ré, FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), Advances in Neural Information Processing Systems, Vol. 35, Curran Associates, Inc., 2022, pp. 16344–16359.
URL https://proceedings.neurips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf

[60] C. Oancea, T. Henriksen, A. W. Hsu, alexnick83, D. van Balen, T. Smeding, I. G. de Wolff, T. D. Matteis, S.-B. Scholz, T. L. McDonell, diku-dk/cfal-bench: 2026-06-16 (Jun. 2026). doi:10.5281/zenodo.20715489.
URL https://doi.org/10.5281/zenodo.20715489

[61] D. van Balen, T. DE MATTEIS, C. Grelck, T. Henriksen, A. Hsu, G. Keller, T. Koopman, T. McDonell, C. Oancea, S.-B. Scholz, A. Sinkarovs, T. Smeding, P. Trinder, I. G. de Wolff, A. N. Ziogas, Raw measurements for the paper "Comparing Functional Array Languages" (Jun. 2026). doi:10.5281/zenodo.20715635.
URL https://doi.org/10.5281/zenodo.20715635

[62] D. Bailey, T. Harris, W. Saphir, R. Van Der Wijngaart, A. Woo, M. Yarrow, The nas parallel benchmarks 2.0, Tech. rep., Technical Report NAS-95-020, NASA Ames Research Center (1995).

[63] M. Krotkiewski, M. Dabrowski, Efficient 3d stencil computations using cuda, Parallel Computing 39 (10) (2013) 533–548. doi:10.1016/j.parco.2013.08.002.

[64] D. Di Domenico, G. G. H. Cavalheiro, J. V. F. Lima, NAS Parallel Benchmark Kernels with Python: A performance and programming effort analysis focusing on GPUs, in: 2022 30th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP), 2022, pp. 26–33. doi:10.1109/PDP55904.2022.00013.

[65] M. M. T. Chakravarty, R. Leshchinskiy, S. Peyton Jones, G. Keller, S. Marlow, Data parallel Haskell: a status report, in: Proceedings of the 2007 Workshop on Declarative Aspects of Multicore Programming, DAMP '07, Association for Computing Machinery, New York, NY, USA, 2007, p. 10–18. doi:10.1145/1248648.1248652.

[66] Y. Zhang, F. Mueller, CuNesl: Compiling nested data-parallel languages for SIMT architectures, in: Proceedings of the 2012 41st International Conference on Parallel Processing, ICPP'12, IEEE Computer Society, Washington, DC, USA, 2012, pp. 340–349. doi:10.1109/ICPP.2012.21.

[67] L. Bergstrom, J. Reppy, Nested data-parallelism on the gpu, in: Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming, ICFP '12, ACM, New York, NY, USA, 2012, pp. 247–258. doi:10.1145/2364527.2364563.

[68] S. Edelkamp, A. Weiß, Blockquicksort: Avoiding branch mispredictions in quicksort, ACM J. Exp. Algorithmics 24 (Jan. 2019). doi:10.1145/3274660.

[69] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, I. Polosukhin, Attention is all you need, in: I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett (Eds.), Advances in Neural Information Processing Systems, Vol. 30, Curran Associates, Inc., 2017, p. 6000–6010.
URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[70] J. S. Bridle, Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition, in: F. F. Soulié, J. Hérault (Eds.), Neurocomputing, Springer Berlin Heidelberg, Berlin, Heidelberg, 1990, pp. 227–236. doi:10.1007/978-3-642-76153-9_28.

[71] M. Milakov, N. Gimelshein, Online normalizer calculation for softmax (2018). arXiv:1805.02867.
URL https://arxiv.org/abs/1805.02867

[72] Dao AI Lab, FlashAttention, https://github.com/Dao-AILab/flash-attention (2024).

[73] T. Henriksen, C. E. Oancea, A T2 graph-reduction approach to fusion, in: Proceedings of the 2Nd ACM SIGPLAN Workshop on Functional High-performance Computing, FHPC '13, ACM, New York, NY, USA, 2013, pp. 47–58. doi:10.1145/2502323.2502328.

[74] I. G. de Wolff, D. van Balen, G. K. Keller, Scheduling task and data parallelism in array languages with work assisting, in: European Conference on Parallel Processing, Springer, 2025, pp. 38–52.

[75] D. W. Walker, J. J. Dongarra, MPI: a standard message passing interface, Supercomputer 12 (1996) 56–68. doi:10.1145/169627.169855.

[76] P. W. Trinder, K. Hammond, H.-W. Loidl, S. P. Jones, Algorithm+ strategy= parallelism, Journal of functional programming 8 (1) (1998) 23–60. doi:10.1017/S0956796897002967.

[77] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T. E. Oliphant, Array programming with NumPy, Nature 585 (7825) (2020) 357–362. doi:10.1038/s41586-020-2649-2.

[78] A. Sidelnik, S. Maleki, B. L. Chamberlain, M. J. Garzar'n, D. Padua, Performance portability with the chapel language, in: 2012 IEEE 26th international parallel and distributed processing symposium, IEEE, 2012, pp. 582–594. doi:10.1109/IPDPS.2012.60.

[79] L. V. Kale, S. Krishnan, CHARM++: a portable concurrent object oriented system based on C++, in: Proceedings of the Eighth Annual Conference on Object-Oriented Programming Systems, Languages, and Applications, OOPSLA '93, Association for Computing Machinery, New York, NY, USA, 1993, p. 91–108. doi:10.1145/165854.165874.

[80] C. Rice University, High performance fortran language specification, SIGPLAN Fortran Forum 12 (4) (1993) 1–86. doi:10.1145/174223.158909.

[81] P. Charles, C. Grothoff, V. Saraswat, C. Donawa, A. Kielstra, K. Ebcioglu, C. von Praun, V. Sarkar, X10: an object-oriented approach to non-uniform cluster computing, SIGPLAN Not. 40 (10) (2005) 519–538. doi:10.1145/1103845.1094852.

[82] R. Chandra, Parallel programming in OpenMP, Morgan Kaufmann, 2001.

[83] J. Sanders, E. Kandrot, CUDA by example: an introduction to general-purpose GPU programming, Addison-Wesley Professional, 2010.

[84] M. Bauer, M. Garland, Legate NumPy: Accelerated and Distributed Array Computing, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 1–23. doi:10.1145/3295500.3356175.

[85] R. Okuta, Y. Unno, D. Nishino, S. Hido, C. Loomis, CuPy: A NumPy-Compatible Library for NVIDIA GPU Calculations (2017).
URL http://learningsys.org/nips17/assets/papers/paper_16.pdf

[86] H. Carter Edwards, C. R. Trott, D. Sunderland, Kokkos: Enabling manycore performance portability through polymorphic memory access patterns, Journal of Parallel and Distributed Computing 74 (12) (2014) 3202–3216, domain-Specific Languages and High-Level Frameworks for High-Performance Computing. doi:https://doi.org/10.1016/j.jpdc.2014.07.003.

[87] LLNL, RAJA Performance Portability Layer (2019).
URL https://github.com/LLNL/RAJA

[88] M. Kotsifakou, P. Srivastava, M. D. Sinclair, R. Komuravelli, V. Adve, S. Adve, HPVM: heterogeneous parallel virtual machine, SIGPLAN Not. 53 (1) (2018) 68–80. doi:10.1145/3200691.3178493.

[89] M. Bauer, S. Treichler, E. Slaughter, A. Aiken, Legion: expressing locality and independence with logical regions, in: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis, SC '12, IEEE Computer Society Press, Washington, DC, USA, 2012, pp. 1–11.

[90] S. Lee, J. S. Vetter, Openarc: open accelerator research compiler for directive-based, efficient heterogeneous computing, in: Proceedings of the 23rd International Symposium on High-Performance Parallel and Distributed Computing, 2014, pp. 115–120. doi:10.1145/2600212.2600227.

[91] Khronos SYCL Working Group, SYCL 2020 Specification, The Khronos Group Inc., revision 8 (2020).
URL https://www.khronos.org/registry/SYCL/specs/sycl-2020-provisional.pdf

[92] A. Alpay, V. Heuveline, One pass to bind them: The first single-pass sycl compiler with unified code representation across backends, in: Proceedings of the International Workshop on OpenCL and SYCL (IWOCL), ACM, 2023. doi:10.1145/3585784.3585799.

[93] R. Gomes, N. Roma, P. Tomás, Straightforward heterogeneous computing with the oneapi programming model, Electronics 10 (19) (2021) 2386. doi:10.3390/electronics10192386.
URL https://www.mdpi.com/2079-9292/10/19/2386

[94] A. Marowka, On the performance portability of openacc, openmp, kokkos and raja, Journal of Systems Architecture (2022). doi:10.1145/3492805.3492806.
URL https://dl.acm.org/doi/fullHtml/10.1145/3492805.3492806

[95] K. Stec, P. Stpiczyński, Performance portability of sparse matrix–vector multiplication implemented using openmp, openacc and sycl, Future Generation Computer Systems 170 (2025) 107825. doi:https://doi.org/10.1016/j.future.2025.107825.
URL https://www.sciencedirect.com/science/article/pii/S0167739X25001207

[96] L.-N. Pouchet, U. Bondhugula, C. Bastoul, A. Cohen, J. Ramanujam, P. Sadayappan, N. Vasilache, Loop transformations: Convexity, pruning and optimization, in: Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '11, ACM, New York, NY, USA, 2011, pp. 549–562. doi:10.1145/1926385.1926449.

[97] U. Bondhugula, A. Hartono, J. Ramanujam, P. Sadayappan, A practical automatic polyhedral parallelizer and locality optimizer, in: Proceedings

of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '08, ACM, New York, NY, USA, 2008, pp. 101–113. doi:10.1145/1375581.1375595.

[98] S. Verdoolaege, J. Carlos Juega, A. Cohen, J. Ignacio Gómez, C. Tenllado, F. Catthoor, Polyhedral parallel code generation for cuda, ACM Transactions on Architecture and Code Optimization (TACO) 9 (4) (2013) 54:1–54:23. doi:10.1145/2400682.2400713.

[99] T. Grosser, A. Größlinger, C. Lengauer, Polly - Performing Polyhedral Optimizations on a Low-Level Intermediate Representation, Parallel Processing Letters 22 (04) (2012) 1250010. doi:10.1142/S0129626412500107.

[100] R. Baghdadi, J. Ray, M. B. Romdhane, E. Del Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, S. Amarasinghe, Tiramisu: a polyhedral compiler for expressing fast and portable code, in: Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2019, IEEE Press, 2019, p. 193–205.

[101] M. M. Strout, Performance transformations for irregular applications, Ph.D. thesis, University of California, aAI3094622 (2003).

[102] M. M. Strout, A. LaMielle, L. Carter, J. Ferrante, B. Kreaseck, C. Olschanowsky, An approach for code generation in the sparse polyhedral framework, Parallel Computing 53 (2016) 32–57. doi:10.1016/j.parco.2016.02.004.

[103] C. E. Oancea, L. Rauchwerger, Scalable conditional induction variables (civ) analysis, in: 2015 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 2015, pp. 213–224. doi:10.1109/CGO.2015.7054201.

[104] S. Moon, M. W. Hall, Evaluation of Predicated Array Data-Flow Analysis for Automatic Parallelization, in: Proceedings of the Seventh ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP '99, Association for Computing Machinery, New York, NY, USA, 1999, p. 84–95. doi:10.1145/301104.301112.

[105] C. E. Oancea, A. Mycroft, Set-congruence dynamic analysis for thread-level speculation (tls), in: J. N. Amaral (Ed.), Languages and Compilers for Parallel Computing, Springer Berlin Heidelberg, Berlin, Heidelberg, 2008, pp. 156–171.

[106] F. Dang, H. Yu, L. Rauchwerger, The R-LRPD Test: Speculative Parallelization of Partially Parallel Loops, in: Proceedings 16th International Parallel and Distributed Processing Symposium, 2002, pp. 10 pp–. doi:10.1109/IPDPS.2002.1015493.

[107] M. Hall, C. E. Oancea, A. C. Elster, A. Rasch, S. Joshi, A. M. Tavakkoli, R. Schulze, Scheduling language chronology: Past, present, and future, ACM Trans. Archit. Code Optim. 22 (3) (Sep. 2025). doi:10.1145/3743135.

[108] S. Donadio, J. Brodman, T. Roeder, K. Yotov, D. Barthou, A. Cohen, M. J. Garzarán, D. Padua, K. Pingali, A language for the compact representation of multiple program versions, in: E. Ayguadé, G. Baumgartner, J. Ramanujam, P. Sadayappan (Eds.), Languages and Compilers for Parallel Computing, Springer Berlin Heidelberg, Berlin, Heidelberg, 2006, pp. 136–151. doi:10.1007/978-3-540-69330-7_10.

[109] C. Chen, J. Chame, M. W. Hall, Chill : A framework for composing high-level loop transformations, Tech. rep., Technical Report 08-897, U. of Southern California (2008).

[110] S. Girbal, N. Vasilache, C. Bastoul, A. Cohen, D. Parello, M. Sigler, O. Temam, Semi-automatic composition of loop transformations for deep parallelism and memory hierarchies, International Journal of Parallel Programming 34 (Jun. 2006). doi:10.1007/s10766-006-0012-3.

[111] R. T. Mullapudi, V. Vasista, U. Bondhugula, Polymage: Automatic optimization for image processing pipelines, in: Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '15, Association for Computing Machinery, New York, NY, USA, 2015, p. 429–443. doi:10.1145/2694344.2694364.

[112] J. Hegarty, J. Brunhaver, Z. DeVito, J. Ragan-Kelley, N. Cohen, S. Bell, A. Vasilyev, M. Horowitz, P. Hanrahan, Darkroom: Compiling high-level image processing code into hardware pipelines, ACM Trans. Graph. 33 (4) (2014) 144:1–144:11. doi:10.1145/2601097.2601174.

[113] T. Nelson, A. Rivera, P. Balaprakash, M. Hall, P. D. Hovland, E. Jessup, B. Norris, Generating efficient tensor contractions for gpus, in: 2015 44th International Conference on Parallel Processing, 2015, pp. 969–978. doi:10.1109/ICPP.2015.106.

[114] A. Venkat, M. Hall, M. Strout, Loop and data transformations for sparse matrix code, in: Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '15, Association for Computing Machinery, New York, NY, USA, 2015, p. 521–532.

[115] A. Venkat, M. S. Mohammadi, J. Park, H. Rong, R. Barik, M. M. Strout, M. Hall, Automating wavefront parallelization for sparse matrix computations, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '16, IEEE Press, 2016, pp. 480–491.

[116] R. Senanayake, C. Hong, Z. Wang, A. Wilson, S. Chou, S. Kamil, S. Amarasinghe, F. Kjolstad, A sparse iteration space transformation framework for sparse tensor algebra, Proc. ACM Program. Lang. 4 (OOPSLA) (Nov. 2020). doi:10.1145/3428226.

[117] M. Bansal, O. Hsu, K. Olukotun, F. Kjolstad, Mosaic: An interoperable compiler for tensor algebra, Proc. ACM Program. Lang. 7 (PLDI) (Jun. 2023). doi:10.1145/3591236.

[118] P. Tillet, H. T. Kung, D. Cox, Triton: an intermediate language and compiler for tiled neural network computations, in: Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, MAPL 2019, Association for Computing Machinery, New York, NY, USA, 2019, p. 10–19. doi:10.1145/3315508.3329973.

[119] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, et al., {TVM}: An automated {End-to-End} optimizing compiler for deep learning, in: Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation, OSDI'18, USENIX Association, USA, 2018, p. 579–594.

[120] A. Venkat, T. Rusira, R. Barik, M. Hall, L. Truong, Swirl: High-performance many-core cpu code generation for deep neural networks, The International Journal of High Performance Computing Applications 33 (6) (2019) 1275–1289. doi:10.1177/1094342019866247.

[121] A. Paszke, D. D. Johnson, D. Duvenaud, D. Vytiniotis, A. Radul, M. J. Johnson, J. Ragan-Kelley, D. Maclaurin, Getting to the point: index sets and parallelism-preserving autodiff for pointful array programming, Proc. ACM Program. Lang. 5 (ICFP) (Aug. 2021). doi:10.1145/3473593.

[122] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, Q. Zhang, JAX: composable transformations of Python+NumPy programs (2018). URL http://github.com/jax-ml/jax

[123] J. Bezanson, S. Karpinski, V. B. Shah, A. Edelman, Julia: A fast dynamic language for technical computing, arXiv preprint arXiv:1209.5145 (2012).

[124] T. Besard, C. Foket, B. De Sutter, Effective extensible programming: Unleashing Julia on GPUs, IEEE Transactions on Parallel and Distributed Systems (2018). arXiv:1712.03112, doi:10.1109/TPDS.2018.2872064.

[125] M. Steuwer, T. Remmelg, C. Dubach, Lift: a functional data-parallel IR for high-performance GPU code generation, in: Proceedings of the 2017 International Symposium on Code Generation and Optimization, CGO '17, IEEE Press, 2017, p. 74–85.

[126] M. Steuwer, T. Koehler, B. Köpcke, F. Pizzuti, RISE & shine: Language-oriented compiler design, CoRR abs/2201.03611 (2022). arXiv:2201.03611.
URL https://arxiv.org/abs/2201.03611

[127] A. Rasch, R. Schulze, D. Shabalin, A. Elster, S. Gorlatch, M. Hall, (de/re)-compositions expressed systematically via mdh-based schedules, in: Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction, CC 2023, Association for Computing Machinery, New York, NY, USA, 2023, p. 61–72. doi:10.1145/3578360.3580269.

[128] T. Koehler, A. Goens, S. Bhat, T. Grosser, P. Trinder, M. Steuwer, Guided Equality Saturation, Proc. ACM Program. Lang. 8 (POPL) (Jan. 2024). doi:10.1145/3632900.

[129] E. Holk, Region-based memory management for expressive gpu programming, Ph.D. thesis, Indiana University (2016).

[130] S. Pennycook, J. Sewall, V. Lee, Implications of a metric for performance portability, Future Generation Computer Systems 92 (2019) 947–958. doi:https://doi.org/10.1016/j.future.2017.08.007.
URL https://www.sciencedirect.com/science/article/pii/S0167739X17300559

## Appendix A. Detailed performance figures

This appendix contains detailed performance figures that are too verbose to fit in the main text, in particular including statistical analyses. The 95% confidence intervals for the means are computed using the t-distribution method.

The reported memory usage is peak MiB of main memory for CPU execution, and exclusively peak GPU memory for GPU execution. Depending on the language, the measurement may include only application-level memory usage, and not, e.g., memory used storing code, runtime libraries, or other similar fixed costs. GPU memory usage of Accelerate and APL is missing, as these languages do not have a built-in mechanism to measure memory usage on the GPU, and we could not measure this accurately with an external tool.

*Appendix A.1. N-body*

| $n$-body, 1C, $n = 10^3$, $t = 10^5$ | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $18.94 \pm 0.003$ | $[18.94, 18.94]$ | 18.94 | 2.18 |
| Accelerate | 10 | $18.59 \pm 0.027$ | $[18.57, 18.61]$ | 18.58 | 73.53 |
| APL | 10 | $1.43 \pm 0.044$ | $[1.39, 1.46]$ | 1.45 | – |
| DaCe | 10 | $19.07 \pm 0.007$ | $[19.06, 19.07]$ | 19.07 | 203 |
| Futhark | 10 | $19.24 \pm 0.008$ | $[19.24, 19.25]$ | 19.25 | 0.15 |
| SaC | 10 | $19.32 \pm 0.016$ | $[19.30, 19.33]$ | 19.32 | 5.53 |
| $n$-body, 32C, $n = 10^3$, $t = 10^5$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $199.40 \pm 11.970$ | $[190.37, 208.43]$ | 202.84 | 2.37 |
| Accelerate | 10 | $7.03 \pm 0.015$ | $[7.02, 7.04]$ | 7.03 | 101 |
| APL | – | – | – | – | – |
| DaCe | 200 | $198.73 \pm 8.771$ | $[197.51, 199.96]$ | 201.34 | 203 |
| Futhark | 10 | $133.51 \pm 4.335$ | $[130.24, 136.77]$ | 132.27 | 0.15 |
| SaC | 10 | $126.87 \pm 6.426$ | $[122.02, 131.71]$ | 129.17 | 1.76 |
| $n$-body, GPU, $n = 10^3$, $t = 10^5$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 11 | $448.84 \pm 2.676$ | $[446.96, 450.73]$ | 449.61 | 0.08 |
| Accelerate | 10 | $12.77 \pm 0.351$ | $[12.50, 13.03]$ | 12.89 | – |
| APL | 10 | $36.20 \pm 0.450$ | $[35.86, 36.54]$ | 36.15 | – |
| DaCe | 10 | $116.81 \pm 0.002$ | $[116.81, 116.81]$ | 116.81 | 0.05 |
| Futhark | 10 | $574.31 \pm 1.238$ | $[573.38, 575.25]$ | 574.22 | 0.25 |
| SaC | 10 | $37.60 \pm 0.045$ | $[37.57, 37.64]$ | 37.61 | – |

| $n$-body, 1C, $n = 10^4$, $t = 10^3$ | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $19.03 \pm 0.004$ | $[19.03, 19.03]$ | 19.03 | 2.91 |
| Accelerate | 10 | $19.53 \pm 0.027$ | $[19.51, 19.55]$ | 19.53 | 73.55 |
| APL | 10 | $1.47 \pm 0.056$ | $[1.43, 1.51]$ | 1.48 | – |
| DaCe | 10 | $19.11 \pm 0.002$ | $[19.11, 19.12]$ | 19.11 | 204 |
| Futhark | 10 | $19.26 \pm 0.043$ | $[19.22, 19.29]$ | 19.27 | 1.53 |
| SaC | 10 | $19.36 \pm 0.025$ | $[19.34, 19.37]$ | 19.36 | 6.42 |
| $n$-body, 32C, $n = 10^4$, $t = 10^3$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $559.25 \pm 10.948$ | $[551.00, 567.51]$ | 556.84 | 2.93 |
| Accelerate | 10 | $264.48 \pm 8.391$ | $[258.15, 270.80]$ | 265.60 | 73.49 |
| APL | – | – | – | – | – |
| DaCe | 200 | $546.12 \pm 30.812$ | $[541.81, 550.43]$ | 553.57 | 203 |
| Futhark | 10 | $535.19 \pm 34.338$ | $[509.29, 561.08]$ | 550.65 | 1.53 |
| SaC | 9 | $558.51 \pm 15.493$ | $[545.88, 571.14]$ | 561.07 | 1.76 |
| $n$-body, GPU, $n = 10^4$, $t = 10^3$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 11 | $690.55 \pm 2.369$ | $[688.88, 692.22]$ | 689.40 | 0.76 |
| Accelerate | 10 | $237.47 \pm 1.617$ | $[236.25, 238.69]$ | 236.99 | – |
| APL | 10 | $40.43 \pm 0.144$ | $[40.32, 40.54]$ | 40.37 | – |
| DaCe | 10 | $972.25 \pm 0.084$ | $[972.19, 972.31]$ | 972.28 | 0.53 |
| Futhark | 10 | $1020.83 \pm 2.244$ | $[1019.14, 1022.52]$ | 1021.04 | 1.83 |
| SaC | 10 | $222.09 \pm 0.096$ | $[222.02, 222.16]$ | 222.07 | – |

| $n$-body, 1C, $n = 10^5$, $t = 10^1$ | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $18.86 \pm 0.010$ | $[18.86, 18.87]$ | 18.87 | 9.43 |
| Accelerate | 10 | $19.54 \pm 0.004$ | $[19.54, 19.54]$ | 19.54 | 73.46 |
| APL | 10 | $1.42 \pm 0.026$ | $[1.40, 1.44]$ | 1.42 | – |
| DaCe | 10 | $19.02 \pm 0.003$ | $[19.02, 19.03]$ | 19.02 | 206 |
| Futhark | 10 | $19.19 \pm 0.017$ | $[19.18, 19.21]$ | 19.19 | 15.26 |
| SaC | 10 | $19.37 \pm 0.014$ | $[19.36, 19.38]$ | 19.37 | 15.13 |
| $n$-body, 32C, $n = 10^5$, $t = 10^1$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $584.32 \pm 19.278$ | $[569.79, 598.86]$ | 588.71 | 7.68 |
| Accelerate | 10 | $557.76 \pm 29.504$ | $[535.51, 580.01]$ | 559.61 | 77.69 |
| APL | – | – | – | – | – |
| DaCe | 200 | $595.21 \pm 16.745$ | $[592.87, 597.55]$ | 603.58 | 207 |
| Futhark | 10 | $599.53 \pm 14.925$ | $[588.28, 610.79]$ | 606.43 | 15.26 |
| SaC | 10 | $596.71 \pm 21.328$ | $[580.63, 612.79]$ | 603.09 | 1.76 |
| $n$-body, GPU, $n = 10^5$, $t = 10^1$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 11 | $1407.11 \pm 0.492$ | $[1406.76, 1407.46]$ | 1407.24 | 7.63 |
| Accelerate | 10 | $245.73 \pm 0.102$ | $[245.65, 245.81]$ | 245.72 | – |
| APL | 10 | $48.38 \pm 0.271$ | $[48.17, 48.58]$ | 48.23 | – |
| DaCe | 10 | $1640.29 \pm 2.352$ | $[1638.51, 1642.06]$ | 1639.59 | 5.34 |
| Futhark | 10 | $1663.41 \pm 2.319$ | $[1661.66, 1665.16]$ | 1662.91 | 13.05 |
| SaC | 10 | $264.40 \pm 0.029$ | $[264.38, 264.42]$ | 264.41 | – |

| MG, 1C, A | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $7.02 \pm 0.037$ | $[6.99, 7.05]$ | 7.04 | 425 |
| Accelerate | 10 | $1.51 \pm 0.078$ | $[1.45, 1.57]$ | 1.53 | 1964 |
| APL | 10 | $0.51 \pm 0.007$ | $[0.51, 0.52]$ | 0.51 | – |
| DaCe | 10 | $3.07 \pm 0.005$ | $[3.07, 3.07]$ | 3.07 | 1055 |
| Futhark | 10 | $1.22 \pm 0.009$ | $[1.21, 1.23]$ | 1.22 | 928 |
| SaC | 10 | $5.33 \pm 0.069$ | $[5.28, 5.38]$ | 5.34 | 542 |

| MG, 32C, A | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $77.25 \pm 0.641$ | $[76.77, 77.74]$ | 76.99 | 426 |
| Accelerate | 13 | $6.63 \pm 0.641$ | $[6.23, 7.03]$ | 6.57 | 2026 |
| APL | – | – | – | – | – |
| DaCe | 200 | $25.83 \pm 1.180$ | $[25.67, 26.00]$ | 26.06 | 1054 |
| Futhark | 10 | $14.71 \pm 0.161$ | $[14.59, 14.83]$ | 14.66 | 928 |
| SaC | 10 | $46.50 \pm 1.095$ | $[45.67, 47.33]$ | 46.49 | 545 |

| MG, GPU, A | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $146.01 \pm 2.473$ | $[144.14, 147.87]$ | 144.92 | 1534 |
| Accelerate | 29 | $23.93 \pm 3.771$ | $[22.47, 25.39]$ | 23.68 | – |
| APL | 10 | $11.19 \pm 0.019$ | $[11.18, 11.21]$ | 11.19 | – |
| DaCe | 200 | $120.05 \pm 3.184$ | $[119.61, 120.50]$ | 120.62 | 1239 |
| Futhark | 16 | $224.05 \pm 1.133$ | $[223.43, 224.68]$ | 224.62 | 1540 |
| SaC | – | – | – | – | – |

| MG, 1C, B | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $7.45 \pm 0.060$ | $[7.41, 7.50]$ | 7.48 | 425 |
| Accelerate | 10 | $1.72 \pm 0.010$ | $[1.71, 1.72]$ | 1.72 | 1964 |
| APL | 10 | $0.53 \pm 0.009$ | $[0.53, 0.54]$ | 0.53 | – |
| DaCe | 10 | $3.39 \pm 0.003$ | $[3.38, 3.39]$ | 3.39 | 1056 |
| Futhark | 10 | $1.38 \pm 0.041$ | $[1.35, 1.41]$ | 1.39 | 928 |
| SaC | 10 | $6.44 \pm 0.065$ | $[6.39, 6.49]$ | 6.48 | 542 |
| MG, 32C, B | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $88.30 \pm 0.425$ | $[87.98, 88.62]$ | 88.37 | 426 |
| Accelerate | 10 | $7.95 \pm 0.150$ | $[7.84, 8.06]$ | 7.89 | 2145 |
| APL | – | – | – | – | – |
| DaCe | 10 | $38.83 \pm 0.192$ | $[38.69, 38.98]$ | 38.89 | 1051 |
| Futhark | 10 | $24.83 \pm 0.129$ | $[24.73, 24.93]$ | 24.88 | 928 |
| SaC | 10 | $56.33 \pm 1.162$ | $[55.46, 57.21]$ | 56.36 | 545 |
| MG, GPU, B | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $221.87 \pm 18.952$ | $[207.57, 236.16]$ | 231.77 | 1534 |
| Accelerate | 23 | $24.93 \pm 3.492$ | $[23.38, 26.47]$ | 25.04 | – |
| APL | 10 | $11.25 \pm 0.012$ | $[11.24, 11.25]$ | 11.25 | – |
| DaCe | 10 | $130.92 \pm 0.218$ | $[130.76, 131.08]$ | 130.91 | 1239 |
| Futhark | 10 | $232.44 \pm 0.473$ | $[232.08, 232.80]$ | 232.47 | 1540 |
| SaC | – | – | – | – | – |

| MG, 1C, C | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $6.67 \pm 0.091$ | $[6.60, 6.74]$ | 6.67 | 3333 |
| Accelerate | 10 | $1.14 \pm 0.009$ | $[1.13, 1.14]$ | 1.14 | 14745 |
| APL | 10 | $0.49 \pm 0.009$ | $[0.48, 0.50]$ | 0.49 | – |
| DaCe | 10 | $2.82 \pm 0.002$ | $[2.82, 2.82]$ | 2.82 | 4328 |
| Futhark | 10 | $1.39 \pm 0.009$ | $[1.38, 1.39]$ | 1.39 | 7424 |
| SaC | 10 | $5.33 \pm 0.018$ | $[5.32, 5.35]$ | 5.34 | 4295 |
| MG, 32C, C | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $44.77 \pm 0.441$ | $[44.43, 45.10]$ | 44.93 | 3333 |
| Accelerate | 10 | $13.68 \pm 0.593$ | $[13.23, 14.12]$ | 13.62 | 11819 |
| APL | – | – | – | – | – |
| DaCe | 10 | $34.01 \pm 0.255$ | $[33.82, 34.20]$ | 34.06 | 4325 |
| Futhark | 10 | $23.60 \pm 0.650$ | $[23.11, 24.09]$ | 23.77 | 7424 |
| SaC | 10 | $40.37 \pm 1.332$ | $[39.36, 41.37]$ | 40.89 | 4298 |
| MG, GPU, C | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $223.21 \pm 1.091$ | $[222.39, 224.03]$ | 223.68 | 3843 |
| Accelerate | 10 | $79.26 \pm 0.664$ | $[78.76, 79.76]$ | 79.14 | – |
| APL | 10 | $8.85 \pm 0.003$ | $[8.84, 8.85]$ | 8.85 | – |
| DaCe | 10 | $215.99 \pm 0.483$ | $[215.63, 216.36]$ | 215.93 | 7045 |
| Futhark | 10 | $233.15 \pm 0.423$ | $[232.84, 233.47]$ | 233.28 | 12308 |
| SaC | – | – | – | – | – |

| FlashAttention, 1C, $d = 128, N = 16384$ | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $67.95 \pm 0.133$ | $[67.85, 68.05]$ | 67.97 | 19.00 |
| Accelerate | 10 | $24.41 \pm 1.337$ | $[23.40, 25.42]$ | 24.29 | 2098 |
| APL | 10 | $23.78 \pm 0.068$ | $[23.73, 23.83]$ | 23.77 | – |
| DaCe | 10 | $83.45 \pm 0.112$ | $[83.37, 83.54]$ | 83.48 | 282 |
| Futhark | 10 | $2.47 \pm 0.000$ | $[2.47, 2.47]$ | 2.47 | 24.00 |
| SaC | 10 | $23.30 \pm 0.121$ | $[23.21, 23.39]$ | 23.27 | 36.90 |
| FlashAttention, 32C, $d = 128, N = 16384$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $2041.28 \pm 11.834$ | $[2032.35, 2050.20]$ | 2035.36 | 19.00 |
| Accelerate | 52 | $266.01 \pm 56.560$ | $[250.11, 281.91]$ | 244.35 | 2105 |
| APL | – | – | – | – | – |
| DaCe | 20 | $2394.63 \pm 20.851$ | $[2384.62, 2404.65]$ | 2399.42 | 282 |
| Futhark | 10 | $68.09 \pm 0.437$ | $[67.76, 68.42]$ | 68.07 | 24.00 |
| SaC | 10 | $545.64 \pm 4.157$ | $[542.51, 548.78]$ | 545.57 | 286 |
| FlashAttention, GPU, $d = 128, N = 16384$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 50 | $6116.32 \pm 4.304$ | $[6115.09, 6117.56]$ | 6116.48 | 32.12 |
| Accelerate | 10 | $15.36 \pm 0.022$ | $[15.34, 15.38]$ | 15.36 | – |
| APL | 10 | $604.56 \pm 1.150$ | $[603.69, 605.42]$ | 603.98 | – |
| DaCe | 10 | $5276.28 \pm 48.582$ | $[5239.65, 5312.91]$ | 5296.55 | 193 |
| Futhark | 17 | $4315.29 \pm 1.831$ | $[4314.32, 4316.26]$ | 4315.32 | 2080 |
| SaC | – | – | – | – | – |

| FlashAttention, 1C, $d = 128, N = 8192$ | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $73.53 \pm 0.207$ | $[73.37, 73.69]$ | 73.64 | 18.73 |
| Accelerate | 10 | $25.76 \pm 1.917$ | $[24.31, 27.20]$ | 26.48 | 550 |
| APL | 10 | $25.83 \pm 0.466$ | $[25.48, 26.18]$ | 26.07 | – |
| DaCe | 10 | $88.73 \pm 0.090$ | $[88.66, 88.80]$ | 88.76 | 282 |
| Futhark | 10 | $2.70 \pm 0.026$ | $[2.68, 2.72]$ | 2.71 | 12.00 |
| SaC | 10 | $25.89 \pm 0.211$ | $[25.73, 26.05]$ | 25.96 | 21.25 |
| FlashAttention, 32C, $d = 128, N = 8192$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $2192.89 \pm 20.785$ | $[2177.22, 2208.57]$ | 2188.95 | 18.73 |
| Accelerate | 46 | $210.28 \pm 42.073$ | $[197.65, 222.91]$ | 192.68 | 557 |
| APL | – | – | – | – | – |
| DaCe | 200 | $2536.08 \pm 28.701$ | $[2532.07, 2540.09]$ | 2543.18 | 282 |
| Futhark | 10 | $74.23 \pm 0.643$ | $[73.74, 74.71]$ | 74.20 | 12.00 |
| SaC | 10 | $541.93 \pm 9.661$ | $[534.64, 549.21]$ | 542.91 | 149 |
| FlashAttention, GPU, $d = 128, N = 8192$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 50 | $5436.95 \pm 9.108$ | $[5434.33, 5439.56]$ | 5438.43 | 16.06 |
| Accelerate | 10 | $17.50 \pm 0.001$ | $[17.50, 17.50]$ | 17.50 | – |
| APL | 10 | $559.73 \pm 8.085$ | $[553.63, 565.83]$ | 563.28 | – |
| DaCe | 10 | $4656.77 \pm 44.581$ | $[4623.16, 4690.39]$ | 4664.10 | 125 |
| Futhark | 65 | $4498.21 \pm 1.449$ | $[4497.85, 4498.57]$ | 4497.97 | 528 |
| SaC | – | – | – | – | – |

| FlashAttention, 1C, $d = 64, N = 16384$ | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $75.14 \pm 0.117$ | $[75.05, 75.22]$ | 75.17 | 18.46 |
| Accelerate | 10 | $21.63 \pm 1.428$ | $[20.55, 22.71]$ | 21.64 | 2086 |
| APL | 10 | $15.08 \pm 0.043$ | $[15.05, 15.12]$ | 15.09 | – |
| DaCe | 10 | $79.55 \pm 0.036$ | $[79.53, 79.58]$ | 79.56 | 282 |
| Futhark | 10 | $3.06 \pm 0.007$ | $[3.06, 3.07]$ | 3.06 | 12.00 |
| SaC | 10 | $24.36 \pm 0.168$ | $[24.23, 24.49]$ | 24.44 | 21.18 |
| FlashAttention, 32C, $d = 64, N = 16384$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $2274.31 \pm 10.085$ | $[2266.71, 2281.92]$ | 2271.59 | 18.46 |
| Accelerate | 95 | $150.08 \pm 43.431$ | $[141.18, 158.97]$ | 126.33 | 2093 |
| APL | – | – | – | – | – |
| DaCe | 20 | $2312.34 \pm 21.061$ | $[2302.23, 2322.45]$ | 2320.06 | 304 |
| Futhark | 10 | $83.61 \pm 1.060$ | $[82.81, 84.41]$ | 84.07 | 12.00 |
| SaC | 10 | $569.17 \pm 10.565$ | $[561.21, 577.14]$ | 568.11 | 149 |
| FlashAttention, GPU, $d = 64, N = 16384$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 50 | $5916.54 \pm 1019.260$ | $[5623.93, 6209.15]$ | 6544.76 | 16.12 |
| Accelerate | 10 | $11.49 \pm 0.001$ | $[11.49, 11.49]$ | 11.49 | – |
| APL | 10 | $293.30 \pm 1.115$ | $[292.46, 294.14]$ | 293.68 | – |
| DaCe | 10 | $4147.15 \pm 27.855$ | $[4126.15, 4168.15]$ | 4152.25 | 285 |
| Futhark | 26 | $3574.49 \pm 6.574$ | $[3571.78, 3577.20]$ | 3576.28 | 2064 |
| SaC | – | – | – | – | – |

| FlashAttention, 1C, $d = 64, N = 32768$ | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $75.35 \pm 0.109$ | $[75.27, 75.43]$ | 75.36 | 34.43 |
| Accelerate | 19 | $19.48 \pm 2.410$ | $[18.29, 20.67]$ | 19.67 | 8239 |
| APL | 10 | $14.47 \pm 0.040$ | $[14.44, 14.50]$ | 14.48 | – |
| DaCe | 10 | $80.06 \pm 0.045$ | $[80.02, 80.09]$ | 80.06 | 282 |
| Futhark | 10 | $3.38 \pm 0.348$ | $[3.12, 3.64]$ | 3.17 | 24.00 |
| SaC | 10 | $23.63 \pm 0.113$ | $[23.55, 23.72]$ | 23.61 | 36.89 |
| FlashAttention, 32C, $d = 64, N = 32768$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $2159.39 \pm 204.923$ | $[2004.87, 2313.91]$ | 2198.11 | 34.50 |
| Accelerate | 92 | $182.46 \pm 52.126$ | $[171.61, 193.32]$ | 163.79 | 8249 |
| APL | – | – | – | – | – |
| DaCe | 10 | $2325.31 \pm 17.550$ | $[2312.07, 2338.54]$ | 2330.58 | 322 |
| Futhark | 10 | $82.29 \pm 0.819$ | $[81.68, 82.91]$ | 82.65 | 24.00 |
| SaC | 10 | $572.81 \pm 1.949$ | $[571.34, 574.28]$ | 572.63 | 286 |
| FlashAttention, GPU, $d = 64, N = 32768$ | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 50 | $6579.51 \pm 5.259$ | $[6578.00, 6581.02]$ | 6579.22 | 32.25 |
| Accelerate | 10 | $9.60 \pm 0.005$ | $[9.59, 9.60]$ | 9.60 | – |
| APL | 10 | $333.03 \pm 0.655$ | $[332.53, 333.52]$ | 333.19 | – |
| DaCe | 10 | $4740.95 \pm 22.674$ | $[4723.85, 4758.05]$ | 4740.76 | 1085 |
| Futhark | 10 | $3605.73 \pm 3.402$ | $[3603.16, 3608.29]$ | 3604.84 | 8224 |
| SaC | – | – | – | – | – |

*Appendix A.4. Quickhull*

| Quickhull, 1C, Circle | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $3.14 \pm 0.017$ | $[3.13, 3.15]$ | 3.13 | 3100 |
| Accelerate | 28 | $6.98 \pm 1.080$ | $[6.55, 7.40]$ | 6.61 | 15072 |
| APL | 10 | $20.88 \pm 0.527$ | $[20.48, 21.27]$ | 20.69 | – |
| DaCe | – | – | – | – | – |
| Futhark | 10 | $6.55 \pm 0.068$ | $[6.50, 6.61]$ | 6.53 | 5422 |
| SaC | 10 | $2.16 \pm 0.013$ | $[2.15, 2.17]$ | 2.15 | – |

| Quickhull, 32C, Circle | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $0.20 \pm 0.004$ | $[0.20, 0.20]$ | 0.20 | 3478 |
| Accelerate | 57 | $3.49 \pm 0.773$ | $[3.28, 3.69]$ | 3.41 | 16993 |
| APL | – | – | – | – | – |
| DaCe | – | – | – | – | – |
| Futhark | 10 | $1.39 \pm 0.005$ | $[1.38, 1.39]$ | 1.39 | 5422 |
| SaC | – | – | – | – | – |

| Quickhull, GPU, Circle | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | – | – | – | – | – |
| Accelerate | 61 | $0.65 \pm 0.151$ | $[0.61, 0.69]$ | 0.54 | – |
| APL | 10 | $1.23 \pm 0.075$ | $[1.18, 1.29]$ | 1.26 | – |
| DaCe | – | – | – | – | – |
| Futhark | 10 | $0.06 \pm 0.000$ | $[0.06, 0.06]$ | 0.06 | 6109 |
| SaC | – | – | – | – | – |

| Quickhull, 1C, Rectangle | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $2.35 \pm 0.012$ | $[2.34, 2.36]$ | 2.35 | 3078 |
| Accelerate | 77 | $4.19 \pm 1.089$ | $[3.94, 4.43]$ | 3.74 | 13001 |
| APL | 10 | $13.85 \pm 0.403$ | $[13.54, 14.15]$ | 13.69 | – |
| DaCe | – | – | – | – | – |
| Futhark | 10 | $4.78 \pm 0.005$ | $[4.78, 4.78]$ | 4.78 | 4963 |
| SaC | 10 | $1.95 \pm 0.011$ | $[1.94, 1.96]$ | 1.94 | – |

| Quickhull, 32C, Rectangle | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $0.11 \pm 0.002$ | $[0.11, 0.12]$ | 0.11 | 3132 |
| Accelerate | 56 | $2.65 \pm 0.589$ | $[2.49, 2.81]$ | 2.78 | 15170 |
| APL | – | – | – | – | – |
| DaCe | – | – | – | – | – |
| Futhark | 10 | $1.28 \pm 0.053$ | $[1.24, 1.32]$ | 1.27 | 4963 |
| SaC | – | – | – | – | – |

| Quickhull, GPU, Rectangle | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | – | – | – | – | – |
| Accelerate | 10 | $0.14 \pm 0.004$ | $[0.14, 0.14]$ | 0.14 | – |
| APL | 10 | $0.66 \pm 0.001$ | $[0.66, 0.66]$ | 0.66 | – |
| DaCe | – | – | – | – | – |
| Futhark | 12 | $0.04 \pm 0.000$ | $[0.04, 0.04]$ | 0.04 | 6109 |
| SaC | – | – | – | – | – |

| Quickhull, 1C, Quadratic | | | | | |
|---|---|---|---|---|---|
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $27.11 \pm 0.025$ | $[27.09, 27.13]$ | 27.10 | 4149 |
| Accelerate | 13 | $78.49 \pm 7.838$ | $[73.56, 83.42]$ | 77.84 | 16958 |
| APL | 10 | $113.90 \pm 1.784$ | $[112.56, 115.25]$ | 113.35 | – |
| DaCe | – | – | – | – | – |
| Futhark | 10 | $36.33 \pm 0.047$ | $[36.29, 36.36]$ | 36.32 | 8775 |
| SaC | 10 | $7.17 \pm 0.026$ | $[7.15, 7.19]$ | 7.17 | – |
| Quickhull, 32C, Quadratic | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | 10 | $2.95 \pm 0.028$ | $[2.93, 2.97]$ | 2.95 | 5499 |
| Accelerate | 257 | $24.15 \pm 11.593$ | $[22.73, 25.58]$ | 19.37 | 23297 |
| APL | – | – | – | – | – |
| DaCe | – | – | – | – | – |
| Futhark | 10 | $4.10 \pm 0.181$ | $[3.96, 4.23]$ | 4.03 | 8775 |
| SaC | – | – | – | – | – |
| Quickhull, GPU, Quadratic | | | | | |
| Language | Runs | Mean±stddev | 95% CI | Median | Memory |
| Baseline | – | – | – | – | – |
| Accelerate | 10 | $4.48 \pm 0.041$ | $[4.45, 4.51]$ | 4.48 | – |
| APL | 10 | $7.57 \pm 0.039$ | $[7.54, 7.60]$ | 7.57 | – |
| DaCe | – | – | – | – | – |
| Futhark | 10 | $0.54 \pm 0.006$ | $[0.54, 0.55]$ | 0.54 | 9197 |
| SaC | – | – | – | – | – |

## Appendix B. N-body roofline analysis

We conduct roofline analyses of the $N$-body benchmark instances to provide a means of analyzing the performance of the different languages and their $N$-body GPU implementations. The analysis is based on metrics measured with the NVIDIA Nsight Compute (ncu) profiling tool.

The ncu tool provides roofline analysis, but only on a per-GPU-kernel basis. As $N$-body, like many other GPU programs, uses multiple kernels, we extract the ncu measurements for each kernel executed and combine them to provide a single data point for the execution of an $N$-body instance. We specifically profile a single time step for each $N$-body benchmark. For each language implementation we provide, similar to ncu, three data points, one for each level of the memory hierarchy: DRAM (GPU global memory), L2 cache (shared by all streaming multiprocessors), and L1 cache (private to each streaming multiprocessor). Some data points may partially overlap, but should be visible when zooming in.

As ncu automatically sets the GPU clocks to their base values to ensure consistency and comparability of metrics across different executions, the peak

values do not match the NVIDIA A30's advertised specifications, nor the $N$-body compute rates reported in Table 5 of Section 6. For example, the peak FP64 performance on the roofline plot is only 3.3 Tflop/s compared to the advertised 5.2 Tflop/s. Likewise, the measured values do not match the theoretical values, especially the floating-point operations (flops) executed, and we use the exact values provided by ncu. In Table B.11, we compare the number of flops for a single time step on the $N = 1,000$ dataset across the different GPU implementations, as measured by ncu. We observe that all implementations perform at least twice as many flops as theoretically expected, with the biggest outlier being APL (86× the theoretical value).

| | Theory | Baseline | Accelerate | APL | DaCe | Futhark | SaC |
|---|---|---|---|---|---|---|---|
| # Mflops | 19.012 | 41.777 | 45.007 | 1644.259 | 54.999 | 47.168 | 41.997 |

Table B.11: Number of floating-point operations in Mflops ($10^6$ flops) for 1 $N$-body time step on the $N = 1,000$ dataset across the different GPU implementations.

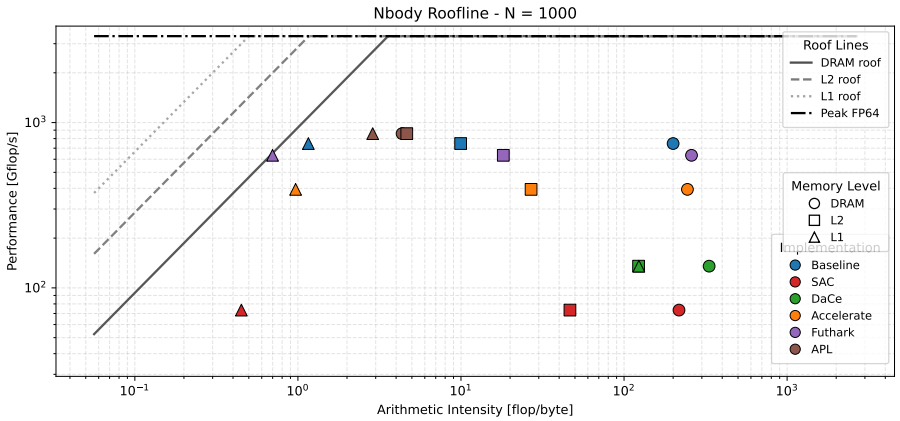



Figure B.18: $N$-body roofline plot for $N = 1,000$.

Figs. B.18, B.19, and B.20 show the roofline plots for $N = 1,000$, $N = 10,000$, and $N = 100,000$, respectively. We observe that across all datasets and languages, except APL, the GPU-kernel implementations operate clearly in the compute-bound regime relative to DRAM (GPU global memory). However, for L1 and L2 caches, the arithmetic intensity is significantly reduced, and some implementations are nearly memory-bound by L1 cache bandwidth.

We note that the performance shown in the roofline plots is not necessarily indicative of actual runtime, primarily because each implementation practically executes different numbers of flops (especially APL). Data for APL are missing for $N = 10,000$ because the profiling report is too large to process. Furthermore, for $N = 100,000$, APL's metrics have been extrapolated by profiling the execution of a partial time step.

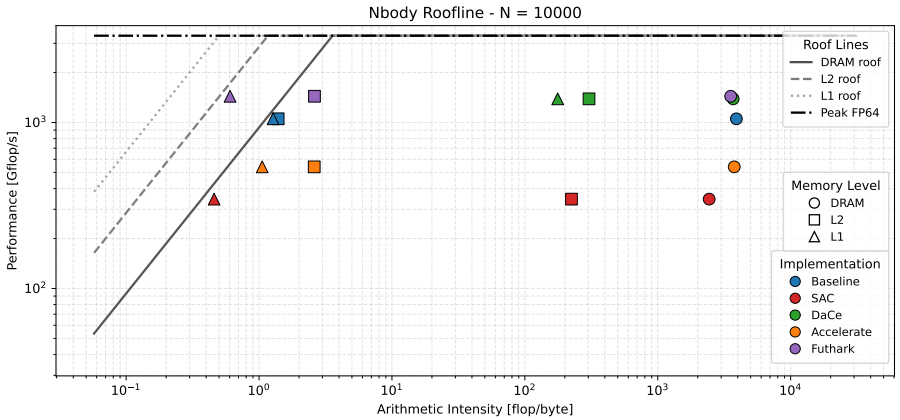



Figure B.19: $N$-body roofline plot for $N = 10,000$.

We note that although each language implementation uses a different number of GPU kernels, there is, in all cases but APL, a single kernel that performs the vast majority of flops and has by far the longest runtime. Therefore, to better understand why these implementations exhibit very similar arithmetic intensity but significantly different performance, we study in Table B.12 these main compute kernels for each language and $N$-body dataset. APL is a special case, as its GPU implementation spawns far more kernel executions than all the other languages (200+ vs $\leq 6$ per time step) and is excluded from the following analysis.

Studying these metrics, we observe that the baseline and Futhark distribute computations differently for $N = 1,000$ and $N = 10,000$ than for $N = 100,000$, resulting in overall better performance across all datasets. DaCe and SaC use the same distribution for all datasets, resulting in severe underutilization of the A30 GPU's 56 SMs for $N = 1,000$ and $N = 10,000$. Accelerate always utilizes all SMs with exactly 1 thread block per SM. Its kernel performance is significantly better than the total benchmark runtime suggests (see Table B.12), indicating increased CPU overhead relative to the other implementations. At last, DaCe and Futhark appear to have exactly the same compute distribution for $N = 100,000$ and perform slightly better than the baseline. We attribute this to their increased SM utilization since they execute 4 thread blocks per SM compared to the baseline's 2. This allows them to better hide stalls that may occur within a single thread block. However, we note that more thread blocks per SM are not always better. SaC runs 32 thread blocks per SM, the maximum possible in the Ampere GPU microarchitecture, but achieves significantly reduced compute throughput.

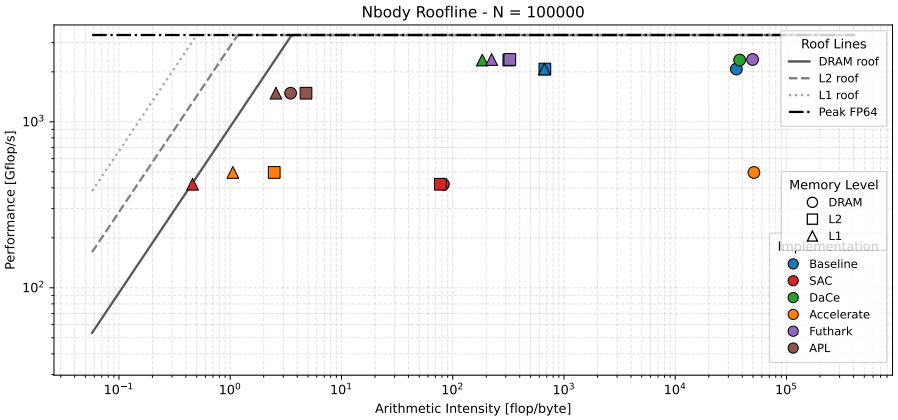



Figure B.20: $N$-body roofline plot for $N = 100,000$.

| Language | Grid Size | Waves [Full + Partial] | Blocks per SM | Kernel Runtime | Compute (SM) Throughput [%] |
|---|---|---|---|---|---|
| | | $N = 1,000$ | | | |
| Baseline | 1,000 | $2 \times 336 + 1 \times 328$ | 6 | 57.44 $\mu$s | 35.27 |
| Accelerate | 56 | $1 \times 56 + 0$ | 1 | 122.30 $\mu$s | 33.31 |
| DaCe | 4 | $0 + 1 \times 4$ | 4 | 305.92 $\mu$s | 5.22 |
| Futhark | 448 | $2 \times 168 + 1 \times 112$ | 3 | 48.26 $\mu$s | 49.28 |
| SaC | 16 | $0 + 1 \times 16$ | 16 | 589.86 $\mu$s | 2.92 |
| | | $N = 10,000$ | | | |
| Baseline | 10,000 | $178 \times 56 + 1 \times 32$ | 1 | 4.24 ms | 42.41 |
| Accelerate | 56 | $1 \times 56 + 0$ | 1 | 10.92 ms | 33.27 |
| DaCe | 40 | $0 + 1 \times 40$ | 4 | 3.02 ms | 52.92 |
| Futhark | 448 | $2 \times 168 + 1 \times 112$ | 3 | 2.84 ms | 56.07 |
| SaC | 157 | $0 + 1 \times 157$ | 32 | 13.00 ms | 12.95 |
| | | $N = 100,000$ | | | |
| Baseline | 196 | $1 \times 112 + 1 \times 84$ | 2 | 206.77 ms | 81.25 |
| Accelerate | 56 | $1 \times 56 + 0$ | 1 | 1.19 s | 34.95 |
| DaCe | 391 | $1 \times 224 + 1 \times 167$ | 4 | 174.19 ms | 89.65 |
| Futhark | 391 | $1 \times 224 + 1 \times 167$ | 4 | 172.67 ms | 90.35 |
| SaC | 1563 | $0 + 1 \times 1,563$ | 32 | 1.07s | 15.78 |

Table B.12: Main $N$-body GPU kernel performance metrics per dataset and language.